

Подготовка за държавен изпит

Алекс Трифонов

1 Множества. Декартово произведение. Релации. Функции.

1.1 Аксиоматизация на множествата: аксиоми за обема, отделянето, степенното множество и индуктивно генерираните множества.

Аксиома за обема: Ако две множества имат едни и същи елементи, то те са равни.

$$\forall x \forall y (\forall z (z \in x \iff z \in y) \Rightarrow x = y)$$

Схема за отделянето: Нека $\varphi(x, u_1, \dots, u_n)$ е теоретикомножествено свойство. Тогава за всяко множество A съществува множество B , чиито елементи са точно онези елементи от A , които имат свойство φ .

$$\forall u_1 \dots \forall u_n \forall A \exists B \forall x [x \in B \iff (x \in A \& \varphi(x, u_1, \dots, u_n))]$$

Аксиома за степенното множество: За всяко множество A съществува множество B , измежду чиито елементи за всички подмножества на A .

$$\forall A \exists B \forall x (x \subseteq A \Rightarrow x \in B)$$

Аксиома за индуктивно генерираните множества: Нека M_0 е непразно множество, а F е множество от операции. Тогава M се створи по следния начин:

1. База: $M_0 \subseteq M$
2. Индукционно предположение: Нека $M \neq \emptyset$
3. Индукционна стъпка: $M_i = M_{i-1} \cup \{f(m_i) \mid f \in F \& m_{i-1} \in M_{i-1}\}$
4. Заключение: В M няма други елементи освен базовите и включените от индукционна стъпка, т.е. $M = \langle M_0, F \rangle$

1.2 Математическа индукция.

Принцип на слабата математическа индукция: Нека $m, n \in \mathbb{N}, n \geq m$. За всяко свойство $\varphi(n)$, ако $\varphi(m)$ е истина и $(\forall k > m) (\varphi(k) \Rightarrow \varphi(k+1))$, то $\forall n \in \mathbb{N} (\varphi(n))$

Принцип на силната математическа индукция: Нека $m, n \in \mathbb{N}, n \geq m$. За всяко свойство $\varphi(n)$, ако $\varphi(m)$ е истина и $(\forall k > m) (\bigwedge_{i=m}^{k-1} \varphi(i) \Rightarrow \varphi(k))$, то $\forall n \in \mathbb{N} (\varphi(n))$

1.3 Основни операции върху множества и техните свойства

Операции: Нека A, B са множества.

- Обединение: $A \cup B = \{x \mid x \in B \vee x \in A\}$
- Сечение: $A \cap B = \{x \mid x \in B \wedge x \in A\}$
- Допълнение: $\bar{A} = \{x \mid x \notin A\}$
- Разлика: $A \setminus B = \{x \mid x \in A \wedge x \notin B\}$
- Симетрична разлика: $A \delta B = (A \setminus B) \cup (B \setminus A)$
- Степенно множество: $\mathcal{P}(A) = \{x \mid x \subseteq A\}$
- Обединение на $\{A_1, \dots, A_n\}$: $\bigcup_{i=1}^n A_i = \{x \mid \exists i (1 \leq i \leq n \& x \in A_i)\}$

- Сечение на $\{A_1, \dots, A_n\}$: $\bigcap_{i=1}^n A_i = \{x \mid \forall i(1 \leq i \leq n \& x \rightarrow A_i)\}$

Свойства Нека A, B, C са множества.

- Идемпотентност: $A \cup A = A, A \cap A = A$
- Комутативност: $A \sigma B = B \sigma A, \sigma \in \{\cup, \cap, \delta\}$
- Асоциативност: $(A \sigma B) \sigma C = A \sigma (B \sigma C), \sigma \in \{\cup, \cap, \delta\}$
- Свойства на универсума: $\Omega \cup A = \Omega, \Omega \cap A = A$
- Свойства на празното множество: $\emptyset \cup A = A, \emptyset \cap A = \emptyset$
- Свойства на допълнението: $A \cup \bar{A} = \Omega, A \cap \bar{A} = \emptyset$
- Закон на Де Морган: $A \bar{\cup} B = \bar{A} \cap \bar{B}, A \bar{\cap} B = \bar{A} \cup \bar{B}$

1.4 Наредена двойка и наредена n-орка

Наредена двойка: За два елемента a, b въвеждаме операцията наредена двойка $\langle a, b \rangle$. Наредената двойка $\langle a, b \rangle$ има следното свойство:

$$a_1 = a_2 \wedge b_1 = b_2 \iff \langle a_1, b_1 \rangle = \langle a_2, b_2 \rangle$$

Наредена двойка на Куратовски: $\langle a, b \rangle = \{\{a\}, \{a, b\}\}$

Наредена n-торка: Нека $a_1, \dots, a_n, n \geq 2$ са произволни елементи. Наредена n-орка на елементите $a_1, \dots, a_n$ наричаме $\langle a_1, \dots, a_n \rangle = \langle a_1, \langle a_2, \dots, a_n \rangle \rangle$

1.5 Декартово произведение и обобщено Декартово произведение.

Декартово произведение: За две множества A, B определяме тяхното декартово произведение като:

$$A \times B = \{\langle a, b \rangle \mid a \in A \& b \in B\}$$

Обобщено Декартово произведение: За краен брой множества $A_1, \dots, A_n$ определяме:

$$A_1 \times \dots \times A_n = \{\langle a_1, \dots, a_n \rangle \mid a_1 \in A_1, \dots, a_n \in A_n\}$$

1.6 Релация над n домейна

Релация над n домейна: Нека $A_1, \dots, A_n$ са множества. Всяко подмножество R на Декартовото произведение на $A_1, \dots, A_n$ наричаме n-местна релация над $A_1, \dots, A_n$. За $n = 2$ казваме, че R е бинарна релация.

Domain, range

$$Dom(R) = \{a \mid a \in A \& (\exists b \in B)[\langle a, b \rangle \in R]\}$$

$$Range(R) = \{b \mid b \in B \& (\exists a \in A)[\langle a, b \rangle \in R]\}$$

1.7 Свойства на бинарните релации

Нека $R \subseteq A \times A$.

- R е рефлексивна $\iff (\forall x \in A)[\langle x, x \rangle \in R]$
- R е транзитивна $\iff (\forall x, y, z \in A)[\langle x, y \rangle \in R \& \langle y, z \rangle \in R \Rightarrow \langle x, z \rangle \in R]$
- R е симетрична $\iff (\forall x, y \in A)[\langle x, y \rangle \in R \Rightarrow \langle y, x \rangle \in R]$
- R е антисиметрична $\iff (\forall x, y \in A)[\langle x, y \rangle \in R \& \langle y, x \rangle \in R \Rightarrow x = y]$
- R е силно антисиметрична $\iff (\forall x, y \in A)[\langle x, y \rangle \in R \Rightarrow \langle y, x \rangle \notin R]$

1.8 Релация на еквивалентност и класове на еквивалентност

Релация на еквивалентност: Нека $A \neq \emptyset$ е множество. Релация R над A наричаме релация на еквивалентност ако R е симетрична, рефлексивна и транзитивна.

Клас на еквивалентност: Нека A е множество, $a \in A$ и R е релация на еквивалентност над A . Класът на еквивалентност на a по отношение на релацията R е множеството:

$$[a]_R = \{b \in A \mid \langle a, b \rangle \in R\}$$

Всяка релация на еквивалентност на множеството поражда разбиване на множеството.

1.9 Релации на частична наредба

Релация на частична наредба: Релацията R над A е релация на частична наредба, ако R е рефлексивна, антисиметрична и транзитивна.

Релация на строга частична наредба: Релацията R над A е релация на строга частична наредба, ако R е антирефлексивна, антисиметрична и транзитивна.

1.10 Диаграми на Хасе

Диаграми на Хасе: графично представяне на бинарна релация $R \subseteq A \times A$, където всеки елемент на A се изобразява като връх и за всеки елемент $\langle a, b \rangle \in R$ поставяме стрелка от a към b .

1.11 Релации на пълна наредба

Пълна наредба: Нека A е множесто и R е релация над A . R е пълна наредба над A , ако R е (строга) частична наредба и всеки два различни елемента от A са сравними по отношение на R .

$$\forall a \in A \forall b \in A (a \neq b \rightarrow (\langle a, b \rangle \in R \vee \langle b, a \rangle \in R))$$

Пример за пълна наредба са релациите $\leq, <$ над естествените числа.

1.12 Минимален и максимален елемент в релация на частична наредба

Нека R е частична наредба над множеството A . Казваме, че $a \in A$ е:

- Минимален елемент по отношение на R , ако $\forall b \in A (a \neq b \Rightarrow \langle b, a \rangle \notin R)$
- Максимален елемент по отношение на R , ако $\forall b \in A (a \neq b \Rightarrow \langle a, b \rangle \notin R)$

1.13 Влагане на частична наредба в пълна наредба - топологично сортиране

Теорема: Нека $A \neq \emptyset$ е крайно множество и R е частична наредба над A . Тогава съществува пълна наредба S над A , т. че $R \subseteq S$

Доказателство: Нека за определеност R е нестрога частична наредба.

Нека $|A| = n$ (крайно, непразно). Ще докажем, че елементите на A могат да се подредят в редица. Тъй като A е крайно множество, то A има минимален и максимален елемент по отношение на R .

- База: Нека a_1 е един такъв минимален елемент на A
- И.Х. Нека сме построили редицата $a_1, \dots, a_i$ от i на брой различни елемента на A
- И.С. Ако $i = 1$, край на конструкцията и $A = \{a_1, \dots, n\}$

Иначе $A_i = A \setminus \{a_1, \dots, a_i\}$ е непразно, а $R \cap (A_i \times A_i)$ е частична наредба над A_i . Избираме минимален елемент a_{i+1} на A_i по отношение на $R \cap A_i \times A_i$

След като сме подредили елементите на A като $a_1, \dots, a_n$ дефинираме релацията $S = \{\langle a_i, a_j \rangle \mid 1 \leq i \leq j \leq n\}$. Твърдим, че S е пълна наредба над A .

- Всеки елемент на A присъства в редицата, т.е. $a = a_i$. Имаме, че $i \leq i, \langle a_i, a_i \rangle \in S$, откъдето S е рефлексивна.
- Антисиметричността и транзитивността на S следват от антисиметричността и транзитивността на $\leq$ над $\mathbb{N}$???

- Нека $a, b \in A$, тогава за някои i, j имаме, че $a = a_i, b = b_j$. Ако $i \leq j$ то $\langle a, b \rangle \in S$, иначе $\langle b, a \rangle \in S$

Оттук S е пълна наредба.

Проверка, че $R \subseteq S$. Нека $\langle a, b \rangle \in R$. Тогава имаме, че $a = a_i, b = a_j$ за някои i, j . Да допуснем, че $j < i$, т.е. $\langle b, a \rangle \in R$. Имаме, че на стъпка j в конструирането на редицата елементът b е минимален на A_j спрямо $R \cap (A_i \times A_i)$ и не се среща в $a_1, \dots, a_{j-1}$. По същият начин $a = a_i$ не се среща в $a_1, \dots, a_{j-1}$, защото $j < i$, откъдето $a_i \in A_j \setminus \{a_1, \dots, a_{j-1}\}$. Така $\langle a, b \rangle \in R \cap (A_i \times A_i)$, но това противоречи с минималността на R . Следователно $i \leq j$ и $\langle a, b \rangle \in S$

1.14 Частични и тотални функции

Частична функция: Релацията $R \subseteq A \times B$ се нарича тотална функция от A в B , ако $\forall a \in A \exists$ поне едно $b \in B$, т. че $\langle a, b \rangle \in R$

Тотална функция: Релацията $R \subseteq A \times B$ се нарича тотална функция от A в B , ако:

- $\forall a \in A \exists b \in B \langle a, b \rangle \in R$
- $\forall a \in A \forall b_1, b_2 \in B [(\langle a, b_1 \rangle \in R \wedge \langle a, b_2 \rangle \in R) \rightarrow b_1 = b_2]$

Означаваме функциите като $f : A \rightarrow B$ и пишем $f(a) = b$ вместо $\langle a, b \rangle \in f$

1.15 Инекции, биекции и сюрекции

Казваме, че функцията $f : A \rightarrow B$ е:

- Инекция (еднозначна), ако $\forall a_1, a_2 \in A [a_1 \neq a_2 \rightarrow f(a_1) \neq f(a_2)]$
- Сюрекция, ако $\forall b \in B \exists a \in A f(a) = b$
- Биекция, ако е инекция и сюрекция.

1.16 Дефиниция на крайно множество и на кардиналност на крайно множество

Крайно множество: Множеството A е крайно ако $A = \emptyset$ или $\exists n \in \mathbb{N} \setminus \{0\}$, т. че има биекция между A и $I_n = \{0, \dots, n-1\}$

Кардиналност: Броят на елементите на множеството A . Бележим с $|A|$

1.17 Дефиниция на изброимо безкрайно множество

Изброимо безкрайно множество: Нека A е множество. Казваме, че A е изброимо безкрайно множество, ако има биекция между A и $\mathbb{N}$

1.18 Принцип на Дирихле

Нека X е множество от n елемента (предмета), Y е множество от m елемента (чекмеджета) и $n > m$. Тогава за всяка тотална функция $f : X \rightarrow Y \exists a \neq b \in X$, т. че $f(a) = f(b)$

2 Основни комбинаторни принципи и конфигурации. Рекурентни уравнения.

2.1 Формулировки на принципите на изброителната комбинаторика - принцип на Дирихле, събирането, изваждането, умножението, делението, включване и изключване.

Принцип на Дирихле: Нека X, Y са крайни множества като $|X| > |Y|$. Тогава за всяка тотална функция $f : X \rightarrow Y$ съществува $a \neq b \in X : f(a) = f(b)$

Принцип на биекцията: Нека A, B са крайни множества. $|A| = |B|$ т.с.т.к. съществува биекция $f : A \rightarrow B$

Принцип на събирането: Нека A е крайно множество, а $P = \{S_1, \dots, S_k\}$ е разбиране на A . Тогава $|A| = \sum_{i=1}^k |S_i|$

Принцип на изваждането: Нека A е множество в универсум Ω . Тогава $|A| = |\Omega| - |\bar{A}|$

Принцип на умножението: Нека A, B са крайни множества и $|A| = n, |B| = m$. Тогава $|A \times B| = |A| \cdot |B| = n \cdot m$

Принцип на делението: Нека A е множество. Нека $R \subseteq A^2$ е релация на еквивалентност. Нека A има k класа на еквивалентност и всеки клас има кардиналност m . Тогава $m = \frac{|A|}{k}$

Принцип на включването и изключването: Нека $A_1, \dots, A_n$ са множества. Тогава:

$$|A_1 \cup \dots \cup A_n| = \sum_{i=1}^n |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| + \dots + (-1)^{n-1} |A_1 \cap \dots \cap A_n|$$

Доказателство: Индукция по n .

- База: При $n = 1$ имаме $|A_1| = |A_1|$
- И.Х.: Нека е вярно за $n - 1$
- И.С.: Имаме, че $|A_1 \cup \dots \cup A_{n-1} \cup A_n| = |A_1 \cup \dots \cup A_{n-1}| + |A_n| - |(A_1 \cup \dots \cup A_{n-1}) \cap A_n|$

От И.Х. знаем колко е $|A_1 \cup \dots \cup A_{n-1}|$. Разглеждаме $(A_1 \cup \dots \cup A_{n-1}) \cap A_n = (A_1 \cap A_n) \cup \dots \cup (A_{n-1} \cap A_n)$.

Но това е обединение на $n - 1$ множества следователно можем да приложим И.Х. Получаваме:

$$|(A_1 \cup \dots \cup A_{n-1}) \cap A_n| = \sum_{i=1}^{n-1} |A_i \cap A_n| - \sum_{1 \leq i < j \leq n-1} |A_i \cap A_n \cap A_j| + \dots + (-1)^{n-1-1} |A_1 \cap \dots \cap A_{n-1} \cap A_n|$$

Обединявайки с горното имаме:

$$\begin{aligned} &= \sum_{1 \leq i < j \leq n-1} (|A_i \cap A_j|) - \sum_{1 \leq i \leq n-1} (|A_i \cap A_n|) = \sum_{1 \leq i < j \leq n} (|A_i \cap A_j|) \\ &\vdots \end{aligned}$$

И получаваме търсеното.

2.2 Основните комбинаторни конфигурации: с или без наредба, с или без повтаряне. Извеждане на формулите за броя на основните комбинаторни конфигурации.

Дадени са ни n обекта и искаме да изберем k от тях. Изборът е функция $f : \{1, \dots, k\} \rightarrow \{1, \dots, n\}$. По колко начина можем да направим този избор, т.е. колко са функциите f ?

Конфигурация с повторение, с наредба: Нека A и B са множества, т.че $|A| = k, |B| = n$. Тогава броят на функциите $f : A \rightarrow B$ е n^k .

Доказателство: Индукция по k :

- $k = 0$: Тогава $A = \emptyset$ и има единствена функция $f = \emptyset : A \rightarrow B$

- $k \geq 1$ и нека $A = \{a_1, \dots, a_k\}$. Всяка функция $f : A \rightarrow B$ можем да представим еднозначно чрез вектора на нейните стойности:

$$f = (f(a_1), \dots, f(a_k)) \in B^k$$

Съгласно принципа на биекцията, броят на различните функции е същият като броя на елементите на B^k . Съгласно принципа на умножението $|B^k| = n^k$

Конфигурация с наредба, без повторение: Тъй не може да имаме повтаряне на обекти, функцията трябва да е инекция. Броят на инекциите $f : A \rightarrow B$ е $n(n-1) \dots (n-k+1) = \prod_{i=0}^{k-1} (n-i)$

Доказателство:

- Ако $k > n$, то от принципа на Дирихле няма инекция $f : A \rightarrow B$. Формулата остава вярна, защото един от множителите ще е 0.
- Ако $k = 0$, то $A = \emptyset$ и функцията $f = \emptyset : A \rightarrow B$ е инекция.
- Нека $1 \leq k \leq n$. Нека $A = \{a_1, \dots, a_k\}$. Всяка инекция $f : A \rightarrow B$ можем да представим еднозначно чрез вектора от нейните стойности: $f = (f(a_1) \dots f(a_k)) \in B^k$. Имаме, че $a_1, \dots, a_k$ са различни и съгласно принципа за умножението получаваме $\prod_{i=0}^{k-1} (n-i)$ възможности.

Конфигурации без повторение, без наредба: Броят на k -елементните подмножества на множество с n елемента е $\binom{n}{k}$

Доказателство: Ако $k \geq n$, няма k -елементни подмножества на множество с n елемента и формулата е вярна.

Нека $k \leq n$ и A е множество. Индукция по n :

- $n = 0, A = \emptyset$. Единствената възможност за k е $k = 1 = \frac{n!}{0!(n-0)!} = \binom{n}{0}$, защото единственото подмножество на празното е празното.
- Нека е вярно за всяко $k \leq n$
- Нека $A = \{a_1, \dots, a_{n+1}\}$ и $k \leq n+1$
 - $k = 0$ получаваме празното множество, което е единственото подмножество на A с 0 елемента.
 - Ако $k = n+1$ то единственото подмножество на A с $n+1$ елемента е самото A . Имаме: $1 = \binom{n+1}{n+1}$
 - $1 \leq k \leq n$. Разделяме k -елементните подмножества на две непресичащи се групи: тези, които съдържат a_{n+1} и тези, които не съдържат a_{n+1} . Съгласно И.Х. тези от втората група са $\binom{n}{k}$ на брой.

Елементите от първата група са от вида $\{a_{n+1} \cup X\}$, където X е някое $n-1$ елементно подмножество на $\{a_1, \dots, a_n\}$. Броят на елементите от тази група са колкото на X , което от И.Х ни дава $\binom{n}{k-1}$ на брой.

Събирайки двете получаваме:

$$\binom{n}{k} + \binom{n}{k-1} = \frac{n!}{k!(n-k)!} + \frac{n!}{(k-1)!(n-k-1)!} = \frac{n!}{(k-1)!(n-k)!(\frac{1}{k} + \frac{1}{n-k+1})} = \frac{(n+1)!}{k!(n+1-k)!} = \binom{n+1}{k}$$

Конфигурация без наредба, с повторение: Трябва да изберем k предмета измежду n вида, като наредбата няма значение. От принципа на биекцията задачата се свежда до начините, по които можем да изберем $k-1$ кутии, в които да поставим n обекта. Общо $\binom{n+k-1}{k-1}$

2.3 Биномни коефициенти и теорема на Нютон.

Теорема на Нютон: Нека $x, y \in \mathbb{R}, n \in \mathbb{N}$. Тогава:

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}$$

Доказателство: Индукция по n :

- $n = 0 : (x+y)^0 = 1 = \binom{0}{0} x^0 y^{0-0}$
- Нека твърдението е в сила за n

•

$$\begin{aligned}
 (x+y)^{n+1} &= (x+y)^n(x+y) = \left(\sum_{k=0}^n \binom{n}{k} x^k y^{n-k} \right) (x+y) = \\
 &= \sum_{k=0}^n \binom{n}{k} x^{k+1} y^{n-k} + \sum_{k=0}^n x^k y^{n-k+1} = \\
 &= \binom{n}{0} x y^n + \binom{n}{1} x^2 y^{n-1} \dots + \binom{n}{n} x^{n+1} + \\
 &+ \binom{n}{0} y^{n+1} + \binom{n}{1} x y^n \dots + \binom{n}{n} x^n y + \\
 &= \binom{n+1}{0} y^{n+1} + \left(\binom{n}{0} + \binom{n}{1} \right) x y^n + \dots + \binom{n+1}{n+1} x^{n+1} = \\
 &= \sum_{k=0}^{n+1} \binom{n+1}{k} x^k y^{n+1-k}
 \end{aligned}$$

2.4 Доказателства на комбинаторни тъждества чрез комбинаторни разсъждения.

Свойство:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Свойство:

$$\binom{n}{m} = \binom{n}{n-m}$$

2.5 Алгоритъм за решаване на линейни рекурентни уравнения с константни коефициенти - хомогенни и нехомогенни.

Рекурентно отношение от ред r: Нека $a_0, \dots, a_{r-1}$ са членове на редицата $a_n = f(a_{n-1}, \dots, a_{n-r}), n \geq r$ - рекурентни отношения от ред r .

Линейно рекурентно уравнение, хомогенно с константни коефициенти

Имаме дадени $a_0, a_1, \dots, a_{r-1}$ дадени начални условия. И рекурентното уравнение: $a_n = e_1 a_{n-1} + e_2 a_{n-2} + \dots + e_r a_{n-r}, e_r \neq 0$

Алгоритъм за решаване на това уравнение:

1. Намираме характеристичното уравнение на хомогенната част:

$$\begin{aligned}
 a_n - e_1 a_{n-1} - \dots - e_r a_{n-r} &= 0 \\
 x^r - e_1 x^{r-1} - \dots - e_r x_0 &= 0
 \end{aligned}$$

2. Намираме корените $x_1, \dots, x_r \in \mathbb{C}$

3.
 - Ако всички корени са различни: $a_n = x_1^n A_1 + x_2^n A_2 \dots + x_r^n A_r$
 - Ако има еднакви: Нека $\alpha = x_{i_1} = x_{i_2} = \dots x_{i_n}$

Общия вид става: $a_n = \dots + (A_{i_1} + A_{i_2} n + \dots + A_{i_k} n^{k-1}) \alpha^n$

4. Правим система за дадените:

$$\begin{cases}
 a_0 = A_1 x_1^0 + \dots + A_r x_r^0 \\
 a_1 = A_1 x_1^1 + \dots + A_r x_r^1 \\
 \vdots \\
 a_{r-1} = A_1 x_1^{r-1} + \dots + A_r x_r^{r-1}
 \end{cases}$$

5. Решаваме системата и получаваме $A_1, \dots, A_r$. Заместваме в общия вид и получаваме отговор.

Алгоритъм за решаване на нехомогенно рекурентно уравнение:

Имаме дадени $a_0, a_1, \dots, a_{r-1}$ дадени начални условия. И рекурентното уравнение: $a_n = e_1 a_{n-1} + e_2 a_{n-2} + \dots + e_r a_{n-r} + f(n)$, $e_r \neq 0$, където $f(n)$ е нехомогенната част.

За решението на нехомогенно линейно рекурентно уравнение с константни коефициенти трябва да представим нехомогенната част:

$$f(n) = e_1^n P_1(n) + \dots + e_s^n P_s(n), \deg(P_i) = d_i, i \in \{1, \dots, s\}$$

Т.е. нехомогенната част представяме като константа e_i на степен n по полиноми от степен d_i . Алгоритъмът за решаване е същият като на стъпка 2 добавяме още $(d_i + 1)$ еднакви корена $= e_i$ за всяко $i \in \{1, \dots, s\}$

3 Графи. Дървета. Обхождания на графи.

3.1 Дефиниция за краен ориентиран (мулти)граф и краен неориентиран (мулти)граф.

Краен неориентиран граф: Наричаме наредената двойка $G = (V, E)$, където $V \neq \emptyset$ е крайно множество от върхове, а $E \subseteq \{\{u, v\} \mid u, v \in V \text{ и } u \neq v\}$

Мултиграф: Наричаме наредената тройка $G = (V, E, f_G)$, където G е множество от върхове, E е множество от ребра и f_G е свързваща функция: $f_G : E \rightarrow V \times V$

Краен ориентиран граф: Наричаме наредена двойка $G = (V, E)$, където G е непразно множество от върхове, а релацията $E \subseteq V \times V$

3.2 Дефиниция за път (цикъл) в ориентиран и неориентиран мултиграф.

Път в мултиграф: Нека $G = (V, E)$ е мултиграф. Редицата от редуващи се върхове и ребра на $G : v_{i_0}, e_{l_1}, v_{i_1}, \dots, v_{k-1}, e_{l_k}, v_k$, в която $e_{l_j} = (v_{i_{j-1}}, v_{i_j}, j = 1, \dots, k-1$ наричаме път в G между v_{i_0} и v_{i_k} . Числото k наричаме дължина на пътя.

Ако в граф $G = (V, E)$ има път от u до v записваме $u \Rightarrow_G^* v$

Цикъл в мултиграф: Нека p е път в мултиграфа $G = (V, E)$. Казваме, че $p = v_{i_0}, \dots, v_{i_k}$ е цикъл, ако $v_{i_0} = v_{i_k}$. Казваме, че цикъла p е прост ако единствените повтарящи се елементи са началния и крайния връх.

3.3 Свързаност и свързани компоненти на граф.

Релация силна свързаност: В ориентираният граф $G = (V, E)$ записваме $u \leftrightarrow_G v \iff u \Rightarrow_G^* v \wedge v \Rightarrow_G^* u$. $\leftrightarrow$ е релация на еквивалентност.

Свързан граф: Неориентираният граф $G = (V, E)$ се нарича свързан, ако за $\forall u, v \in V$ $u \Rightarrow_G^* v$.

Слабо свързан граф: Ориентираният граф $G = (V, E)$ се нарича слабо свързан, ако за $\forall u, v \in V$ $u \Rightarrow_G^* v \vee v \Rightarrow_G^* u$.

Силно свързан граф: Ориентираният граф $G = (V, E)$ се нарича силно свързан, ако за $\forall u, v \in V$ $u \leftrightarrow_G^* v$

Свързани компоненти: Класовете на еквивалентност относно релацията $\leftrightarrow_G^*$

3.4 Дефиниция на дърво и кореново дърво.

Дърво: Свързан, ацикличен граф.

Кореново дърво: Индуктивна дефиниция:

- Графът $T = (\{r\}, \{\})$ с един връх и без ребра е кореново дърво.
- Нека $T = (V, E)$ е дърво с корен r и листа $L = \{l_1, \dots, l_k\}$. Нека $v \in V, u \notin V$. Тогава $T' = (V \cup \{v\}, E \cup (v, u))$ е дърво с корен r и листа $(W \setminus \{v\}) \cup \{u\}$
- Няма други коренови дървета

3.5 Доказателство, че всяко кореново дърво е дърво и $|V| = |E| + 1$

Всяко кореново дърво е дърво: Индукция по дефиниция на кореново дърво.

- $T = (\{r\}, \{\})$ е дърво.
- Нека $T = (V, E)$ е кореново дърво, което е свързан граф без цикли.
- Разглеждаме $T' = (V', E')$, където $V' = V \cup \{w\}, E' = E \cup \{(u, w)\}$ с корен r и $u \in V$.

Имаме, че T е свързан граф без цикли. Добавяйки върхът w към V , сме добавили и връх от $u \in V$ към w . Следователно T' е свързан. Тъй като u е единственият съсед на w , то w не може да участва в цикли. От И.Х. имаме, че T не съдържа цикли, откъдето T' също не съдържа цикли.

Ако $T = (V, E)$ е кореново дърво, то $|V| = |E| + 1$

Доказателство: Индукция по построение на T .

- База: $T = (\{r\}, \{\})$. Имаме, че $|V| = 1 = |E| + 1$
- Нека е вярно за $T = (V, E)$

- За $T' = (V \cup \{w\}, E \cup \{(u, w)\})$, $u \in V, w \notin V$.

$$|V \cup \{w\}| = |V| + 1 = |E| + 1 + 1 = |E \cup \{(u, w)\}| + 1$$

3.6 Покриващо дърво на граф.

Подриващо дърво: Нека $G = (V, E)$ е граф. Покриващо дърво на G наричаме дърво $T = (V, E')$, $E' \subseteq E$

Граф $G = (V, E)$ има покриващо дърво т.с.т.к е свързан.

3.7 Обхождане на граф в ширина и дълбочина.

Обхождане в ширина:

1. Избираме начален връх r . Образоваме $L_0 = \{r\}$ и нека $l = 0$. Обявяваме върха r за "обходен".
2. Ако $L_0 \cup L_1 \cup \dots \cup L_l = V$ завършваме. В противен случай отиваме към следващата стъпка.
3. Нека сме обходили нивата $L_0, \dots, L_l$. Образоваме нивото L_{l+1} от всички необходими върхове $v \notin L_0 \cup \dots \cup L_l$, които са съседни на върхове от L_l . Обявяваме върховете от L_{l+1} за "обходени" и преминаваме към стъпка две с $l = l + 1$.

Обхождане в дълбочина: Използваме текущ връх t и бяща на върха t : $p(t)$

1. "Обхождаме" началния връх r . Имаме, че $t = r$, $p(t) = \varepsilon$
2. Търсим необходим връх, който е съсед на текущия t :
 - 2.1 Ако има такъв връх v тогава обявяваме v за "обходен" и обновяваме $p(v) = t, t = v$.
 - 2.2 Ако няма такъв връх:
 - Ако $t \neq r, p(t)$ е определен. Тогава $t = p(t)$ и преминаваме към стъпка 2.
 - Ако $t = r$ обхождането е завършило.

3.8 Ойлерови обхождания на мултиграф.

Ойлеров път: В свързания мултиграф $G = (V, E)$ наирчае път, който минава еднократно през всяко ребро на мултиграфа. Ако ойлероият път има начало и край, които съвпадат, тогава той се нарича Ойлеров цикъл. Мултиграф, ребрата на който образуват Ойлеров цикъл, наричаме Ойлеров.

3.9 Теорема за обхождане на Ойлеров цикъл (с доказателство) и Ойлеров път.

Нека $G = (V, E)$ е свързан граф. G е Ойлеров $\leftrightarrow$ всеки връх в G има четна степен.

Доказателство: $\Rightarrow$: Нека G е Ойлеров и нека $v = v_0, \dots, v_k = v$ е Ойлеров цикъл в G . Всяко срещане на връх u в редицата $v_0, \dots, v_{k-1}$ съответства на две ребра през u :

- Ако $u = v_0$ ребрата са $\{v_0, v_1\}$ и v_{k-1}, v_0
- Ако $u = v_i, i > 0$ ребрата са $\{v_i, v_{i+1}\}, \{v_{i-1}, v_i\}$

Тъй като всички ребра в графа се срещат точно по веднъж в цикъла, степента на произволен връх $u \in V$ е равна на два пъти броя на срещанията на u в цикъла. Следователно $d(u)$ е четно число.

$\Leftarrow$: Нека G е свързан граф, на който всеки връх има четна степен и $u \in V$. Ще построим Ойлеров цикъл C през u .

Обявяваме u за текущ връх $t, C = (u)$

1. Докато има необходимо ребро $e = \{t, v\}$ през t , добавяме t към C , обявяваме e за обходено и $t = v$.

На всяко минаване на тази стъпка броят на необходимите ребра намалява и цикълът ще завърши след краен брой стъпки k и $C = (u = v_0, \dots, v_k)$

Да допуснем, че $u \neq v_k$. Тогава алгоритъмът завършва, всички ребра през v_k са обходени. На последната стъпка, на която v_k става текущ, обхождаме едно ребро през v_k и броят на обходените ребра през v_k е нечетен. Но алгоритъмът завършва, защото няма как да се избере следващ текущ връх и всички ребра през v_k са обходени. Но тогава v_k има нечетен брой ребра, което е противоречие.

2. Следователно $v_k = u$ и C е цикъл. Ако C съдържа всички ребра, то е Ойлеров цикъл. Ако C съдържа поне един връх v_i , който е край на необходимо ребро повтаряме цикъла на стъпка 1 с начален връх v_i , като строим нов цикъл C' с начало и край v_i .

Накрая добавяме C' към C : $C = (v_0, \dots, v_{i-1}, v_i, \dots, v_i, v_{i+1}, \dots, v_0)$ и се връщаме към стъпка 2.

Следствие: Свързаният мултиграф G съдържа Ойлеров път т.с.т.к всички върхове са от четна степен, като само два са от нечетна.

4 Характеризация на регулярните езици. Теорема на Майхил-Нероуд

4.1 Регулярен език, детерминиран краен автомат и автоматен език. Формулировка на теоремата на Клини.

Регулярен език: Нека Σ е азбука, т.е. $\{+, *, \emptyset, (,)\} \notin \Sigma$. Регулярните изрази над Σ са специални думи над азбуката $\Sigma \cup \{+, *, \emptyset, (,)\}$, които се дефинират рекурсивно:

- $\emptyset, a \in \Sigma$ са регулярни изрази.
- Ако r_1, r_2 са регулярни изрази то $(r_1 r_2)$ и $r_1 + r_2$ също са регулярни изрази.
- Ако r е регулярен израз то r^* е регулярен израз.

Езикът $L(r)$ са един регулярен израз r над Σ дефинираме чрез следната рекурсия:

- $L(\emptyset) = \emptyset, L(a) = \{a\}, a \in \Sigma$
- $L(r_1 + r_2) = L(r_1) \cup L(r_2), L(r_1 r_2) = L(r_1) L(r_2)$
- $L(r^*) = L(r)^*$

Детерминиран краен автомат: Наричаме петорката $\mathcal{A} = \langle Q, X, q_0, \delta, F \rangle$, където:

- Σ е множество от състояния
- Q е множество от преходи
- $\delta : \Sigma \rightarrow Q$ е функция на преходите (тотална)
- $q_0 \in Q$ е начално състояние
- $F \subseteq Q$ е множество от финални състояния

Разпознаване на дума: Нека $\alpha \in \Sigma^* : \alpha = a_0 \dots a_{n-1}$. Казваме, че α се разпознава от автомата $\mathcal{A}$, ако съществува редица от състояния $q_0, \dots, q_n$, т.е.:

- q_0 е начално състояние на автомата
- $\delta(q_i, a_i) = q_{i+1}, \forall i \in \{0, \dots, n-1\}$
- $q_n \in F$

Разпознаване на език: Казваме, че $\mathcal{A}$ разпознава езика L , ако $\mathcal{A}$ разпознава точно думите от L , т.е. $L = \{\alpha \in \Sigma^* \mid \mathcal{A} \text{ разпознава } \alpha\}$

Тогава казваме, че L е автоматен език.

Теорема на Клини: Множествата на регулярните и автоматните езици съвпадат.

Доказателство: 1). Индукция по дефиницията на регулярни езици:

- $\{\epsilon\}, \emptyset, \{a_i\}$ са автоматни, защото са крайни.
- Нека допуснем, че регулярните езици L_α, L_β съответни на регулярните изрази α, β са автоматни.
- Имаме, че $L_\alpha + L_\beta, L_\alpha L_\beta, L_\alpha^*$ са автоматни, защото са сума, произведение на автоматни езици съответно.

Тъй като няма други регулярни езици, то всеки регулярен език е автоматен.

2). Нека $L \subseteq \Sigma^*$ е автоматен език и нека $\mathcal{A} = \langle \Sigma, Q, q_0, \delta, F \rangle$ е краен автомат над Σ с $L(\mathcal{A}) = L$. Нека $Q = \{0, 1, 2, \dots, n-1\}$ и $q_0 = 0$. Нека

$$R_{i,j}^k = \left\{ w \in \Sigma^* \mid i \xrightarrow{w} j, \text{ минавайки само през състояния } s < k \right\}$$

Тогава:

$$R_{i,i}^0 = \{\epsilon\} \cup \{a \mid (i, a, i) \in \delta\}$$

$$R_{i,j}^0 = \{a \mid (i, a, j) \in \delta\}, i \neq j$$

Оттук за всяко $0 \leq i, j \leq n-1$ езикът $R_{i,j}^0$ е регулярен. От друга страна за всяко $0 \leq i, j \leq n-1$ и всяко $0 \leq k \leq n-1$ е в сила:

$$R_{i,j}^{k+1} = R_{i,j}^k \cup R_{ik}^k (R_{kk}^k)^* R_{kj}^k$$

Оттук езиците R_{ij}^k са регулярни за всяко $0 \leq i, j \leq n-1, 0 \leq k \leq n$. От друга страна:

$$L(\mathcal{A}) = \bigcup_{i \in F} R_{0i}^n$$

Следователно $L(\mathcal{A})$ е регулярен.

4.2 Релацията: $\approx_L$

Нека L е произволен език и нека α, β са думи. Казваме, че α и β са еквивалентни относно L и записваме $\alpha \approx \beta$,

$$\alpha \approx_L \beta \iff (\forall w \in \Sigma^*)[\alpha w \in L \iff \beta w \in L]$$

Това означава, че ако искаме да докажем, че $\alpha \not\approx_L \beta$ е достатъчно да намерим дума, за която $\alpha w \in L$ & $\beta w \notin L$ или обратното.

4.3 $\approx_L$ е релация на еквивалентност и е дясно инвариантна.

$\approx_L$ е релация на еквивалентност : Доказателство:

- Рефлексивност:

$$\begin{aligned} \alpha \approx_L \alpha &\iff \forall w \in \Sigma^* (\alpha w \in L \iff \alpha w \in L) \\ &\iff \forall w \in \Sigma^* ((\alpha w \in L \wedge \alpha w \in L) \vee (\alpha w \notin L \wedge \alpha w \notin L)) \iff \forall w \in \Sigma^* (\alpha w \in L \vee \alpha w \notin L) \end{aligned}$$

- Симетричност:

$$\alpha \approx_L \beta \iff (\forall w \in \Sigma^*)[\alpha w \in L \iff \beta w \in L] \iff (\forall w \in \Sigma^*)[\beta w \in L \iff \alpha w \in L] \iff \beta \approx_L \alpha$$

- Транзитивност:

$$\begin{aligned} \alpha \approx_L \beta \wedge \beta \approx_L \gamma &\Rightarrow \\ &(\forall w \in \Sigma^*)[\alpha w \in L \iff \beta w \in L] \wedge (\forall w \in \Sigma^*)[\beta w \in L \iff \gamma w \in L] \\ &\iff (\forall w \in \Sigma^*)[(\alpha w \in L \iff \beta w \in L) \wedge (\beta w \in L \iff \gamma w \in L)] \\ &\Rightarrow (\forall w \in \Sigma^*)[\alpha w \in L \iff \gamma w \in L] \\ &\iff \alpha \approx_L \gamma \end{aligned}$$

$\approx_L$ е дясно инвариантна: $\alpha \approx_L \beta \Rightarrow \forall x \in \Sigma^* : \alpha x \approx_L \beta x$

Доказателство: $\alpha \approx_L \beta \iff \forall w \in \Sigma^* : \alpha w \in L \iff \beta w \in L$

Нека вземем произволно x от Σ^* . Тогава:

$$(\alpha x)w = \alpha(xw), (\beta x)w = \beta(xw)$$

Понеже $xw \in \Sigma^*$:

$$\alpha(xw) \in L \iff \beta(xw) \in L \Rightarrow \alpha x \approx_L \beta x$$

□

Всеки език L над Σ^* разбива множеството Σ^* на непресичащи се класове на еквивалентност: По-точно, всеки език $L \subset \Sigma^*$ дефинира фактормножеството:

$$\Sigma^* / \approx_L = \{[w]_L \mid w \in \Sigma^*\}$$

Където $[w]_L$ е класът на еквивалентност относно релацията $\approx_L$.

4.4 Ако $\approx_L$ има индекс n , то L се разпознава от тотален краен детерминиран автомат с n състояния.

Нека $\Sigma^*/\approx_L$ е крайно. Да разгледаме крайния автомат $\mathcal{A} = (\Sigma, Q_L, q_{0_L}, \delta_L, F_L)$, където:

$$\begin{aligned} Q_L &= \Sigma^*/\approx_L \\ q_{0_L} &= [\epsilon]_L \\ \delta_L &= \{([w]_L, a, [wa]_L \mid w \in \Sigma^* \text{ \& } a \in \Sigma)\} \\ F_L &= \{[w]_L \mid w \in L\} \end{aligned}$$

Ясно е, че $\mathcal{A}_L$ е тотален. Твърдим, че $\mathcal{A}_L$ е детерминиран. Нека $q, q', q'' \in Q_L$ и $a \in \Sigma$ са такива, че:

$$(q, a, q') \in \delta_L \text{ \& } (q, a, q'') \in \delta_L$$

Тогава: $q' = [w'a]_L, q'' = [w''a]_L$, за някои $w', w'' \in \Sigma^*$, такива че:

$$[w']_L = q = [w'']_L$$

Но тогава $w' \approx_L w''$ и значи $w'a \approx_L w''a$. Оттук $q' = q''$ и следователно $\mathcal{A}_L$ е детерминиран.

4.5 Ако L се разпознава от тотален краен детерминиран автомат с n състояния, то индексът на релацията $\approx_L$ е не повече от n

Доказателство: Нека $\mathcal{A} = (\Sigma, Q, q_0, \delta, F)$ е тотален и детерминиран автомат над Σ , който разпознава L , т.е. $L(\mathcal{A}) = L$. Нека в $\mathcal{A}$ има n достижими състояния и нека те са $q_0, q_1, \dots, q_{n-1}$. Да фиксираме $w_0, \dots, w_{n-1}$ т.че:

$$q_0 \xrightarrow[\mathcal{A}]{w_i} q_i, \quad 0 \leq i \leq n-1$$

Тогава за произволна дума $w \in \Sigma^*$ е в сила:

$$q_0 \xrightarrow[\mathcal{A}]{w} q_i, \quad 0 \leq i \leq n-1$$

Знаем, че ако $w, w' \in \Sigma^*$ са такива, че $q_0 \xrightarrow[\mathcal{A}]{w} q$ и $q_0 \xrightarrow[\mathcal{A}]{w'} q$, то $w \approx_{L(\mathcal{A})} w'$.

Оттук имаме, че $w \in [w_i]$ за някое $0 \leq i \leq n-1$. Така:

$$\Sigma^*/\approx_L = \{[w_0]_L, \dots, [w_{n-1}]_L\}$$

Следователно $\Sigma^*/\approx_L$ има най-много n елемента. **Доказателство:**

4.6 Един език L е регулярен точно тогава, когато $\approx_L$ има краен индекс.

Нека $\approx_L$ има краен индекс. От предното твърдение (4.) L се разпознава от краен детерминиран автомат $\mathcal{A}$ с n състояния.

Първо ще докажем, че за произволни $u, w \in \Sigma^*$

$$[u]_L \xrightarrow[\mathcal{A}]{w} [uw]_L$$

Индукция по $|w|$:

- $|w| \leq 1$ очевидно.
- Нека за $|w| = n > 1$ твърдението е вярно за думи с дължина $n-1$
- Нека $w = w'a, |w'| = n-1, a \in \Sigma, w' \in \Sigma^*$. Тогава съгласно индукционното предположение:

$$[u]_L \xrightarrow[\mathcal{A}_L]{w'} [uw']_L$$

$$\text{От друга страна } [uw']_L \xrightarrow[\mathcal{A}_L]{a} [uw'a]_L$$

$$\text{Следователно: } [u]_L \xrightarrow[\mathcal{A}_L]{w'a=w} [uw'a]_L = [w]_L$$

Оттук за произволна дума $w \in \Sigma^*$ е в сила:

$$q_{0_L} \xrightarrow[\mathcal{A}_L]{w} [w]_L$$

Следователно $w \in L(\mathcal{A}_L) \iff [w]_L \in F_L$. От друга страна $[w]_L \in F_L$ т.с.т.к $w \approx_L w', w' \in L$. Но от $w \approx_L w'$ следва:

$$w' \in L \iff w'\epsilon \in L \iff w\epsilon \in L \iff w \in L$$

Откъдето: $w \in L(\mathcal{A}_L) \iff w \in L$

5 Лема за разрастването за контекстносвободни езици. Незатвореност на класа на контекстносвободните езици относно сечение и допълнение

5.1 Контекстносвободна (граматика):

Контекстносвободна граматика (КСГ) наричаме всяка четворка $G = (\Gamma, \Sigma, R, S)$, където $\Sigma \subseteq \Gamma$ е крайна азбука, $S \in \Gamma \setminus \Sigma$, а $R \subseteq (\Gamma \setminus \Sigma) \times \Gamma^*$ е крайно множество. Γ наричаме азбука на граматиката, Σ терминални символи на граматиката, $\Gamma \setminus \Sigma$ - нетерминални символи на граматиката, S начален символ на граматиката, а R правила на граматиката.

5.2 Извод в КСГ:

Ако $(A, \alpha) \in R$ ще пишем $A \rightarrow_G \alpha$ или $A \rightarrow \alpha \in R$ и казваме, че можем да заместим символа A с думата α

За всеки избор на думи $w_1, w_2 \in \Gamma^*$ и правило $A \rightarrow_G \alpha$ ще пишем:

$$w_1 A w_2 \Rightarrow_G w_1 \alpha w_2$$

G преобразува (за $n \geq 0$ стъпки) думата $u \in \Gamma^*$ до дума $u' \in \Gamma^*$ и пишем: $u \Rightarrow_G^* u'$, ако:

$$u \Rightarrow_G u_1 \Rightarrow_G u_2 \cdots \Rightarrow_G u', \quad u_1, \dots, u_{n-1} \in \Gamma^* \text{ (при } n = 0 \text{ получавме } u \Rightarrow_G^* u)$$

5.3 Контекстносвободен език:

ако G е КСГ, то под език на G разбираме множеството:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow_G^* w\}$$

5.4 Дърво на извод в КСГ:

Нека $G = (\Gamma, \Sigma, R, S)$ е КСГ. Дърво на извода (дърво на синтактичен анализ) с корен A и резултат W по G дефинираме индуктивно:

- $\forall a \in \Sigma, a$ е дърво на извода с корен a и резултат a
- S е дърво с корен S и резултат ϵ
- Ако A е правило в G : $A \rightarrow A_1 \dots A_n \in R$ в G и за $A_1, \dots, A_n$ имаме дефинирани дървета на извода $T_{A_1}, \dots, T_{A_n}$ с резултати съответно $w_1, \dots, w_n$, то дървото с корен A и ребра към $A_1, \dots, A_n$ е дърво на извода с корен A и резултат $w_1, \dots, w_n$

5.5 Затвореност на контекстносвободните езици относно регулярните операции (конструкции, без доказателства)

Нека L_1, L_2 са КСЕ над азбуката Σ и нека $G = (\Gamma_1, \Sigma, R_1, S_1), G_2 = (\Gamma_2, \Sigma, R_2, S_2)$ са КСГ с $L(G_1) = L_1, L(G_2) = L_2$ и такива, че $\Gamma_1 \cap \Gamma_2 = \Sigma$

Безконтекстните езици са затворени относно операцията конкатенация:

Доказателство: Да разгледаме безконтекстната граматика: $G = (\Gamma, \Sigma, R, S)$, където:

$$\begin{aligned} \Gamma &= \Gamma_1 \cup \Gamma_2 \cup \{S\} \\ S &\notin \Gamma_1 \cup \Gamma_2 \\ R &= R_1 \cup R_2 \cup \{S \rightarrow S_1 S_2\} \end{aligned}$$

Имаме $L_1 L_2 \subseteq L(G)$. Нека $w_1 \in L_1, w_2 \in L_2$. Тогава

$$S_1 \Rightarrow_{G_1}^* w_1 \text{ \& } S_2 \Rightarrow_{G_2}^* w_2$$

Оттук и $R_1 \cup R_2 \subseteq R$ следва, че:

$$S_1 \Rightarrow_G^* w_1 \text{ \& } S_2 \Rightarrow_G^* w_2$$

Откъдето:

$$S \Rightarrow_G^* S_1 S_2 \Rightarrow_G^* w_1 S_2 \Rightarrow_G^* S_1 S_2$$

Коего означава, че $w_1 w_2 \in L$

За обратното включване имаме, че ако $u \Rightarrow_G^* v, u \in \Gamma_i^*$, то $v \in \Gamma_i^*$ и следователно $u \Rightarrow_{G_i}^* v, u \in \Gamma_i^*$

Нека сега $w \in L(G)$ и да разгледаме един най-ляв извод на w от S . Той има вида:

$$S \Rightarrow_G^* S_1 S_2 \Rightarrow_G^* w_1 S_2 \Rightarrow_G^* w_1 w_2$$

Където $w_1 w_2 = w$ и

$$S_1 \Rightarrow_G^* w_1 \text{ \& } S_2 \Rightarrow_G^* w_2$$

Но $S_1 \in \Gamma_1^*, S_2 \in \Gamma_2^*$ и значи:

$$S_1 \Rightarrow_{G_1}^* w_1 \text{ \& } S_2 \Rightarrow_{G_2}^* w_2$$

Откъдето $w_1 \in L_1, w_2 \in L_2$. Следователно $w \in L_1 L_2$ □

Безконтекстните езици са затворени относно операцията обединение:

Доказателство: Да разгледаме безконтекстната граматика: $G = (\Gamma, \Sigma, R, S)$, където:

$$\begin{aligned}\Gamma &= \Gamma_1 \cup \Gamma_2 \cup \{S\} \\ S &\notin \Gamma_1 \cup \Gamma_2 \\ R &= R_1 \cup R_2 \cup \{S \rightarrow S_1 | S_2\}\end{aligned}$$

Имаме $L_1 \cup L_2 \subseteq L(G)$. Нека $w_1 \in L_1$. Тогава

$$S_1 \Rightarrow_{G_1}^* w_1$$

Оттук и $R_1 \subseteq R$ следва, че:

$$S_1 \Rightarrow_G^* w_1$$

От друга страна:

$$S \Rightarrow_G S_1$$

Откъдето:

$$S \Rightarrow_G^* w_1$$

Коего означава, че $w_1 \in L$

За обратното включване, забелязваме, че от дефиницията на R и от това, че $\Gamma_1 \cap \Gamma_2 = \Sigma$ (нямат общи нетерминални символи), следва че ако $u \Rightarrow_G v$, за някое $u \in \Gamma_1^*$, то $v \in \Gamma_1^*$ и $u \Rightarrow_{G_1}^* v$.

Оттук ако $u \Rightarrow_G^* v$ за някое $u \in \Gamma_2^*$, то $v \in \Gamma_2^*$ и $u \Rightarrow_{G_2}^* v$.

Нека сега $w \in L(G)$. Тогава $S \Rightarrow_G^* w$ и значи:

$$S \Rightarrow_G S_1 \text{ и } S_1 \Rightarrow_G^* w$$

или:

$$S \Rightarrow_G S_2 \text{ и } S_2 \Rightarrow_G^* w$$

Но $S_1 \in \Gamma_1^*, S_2 \in \Gamma_2^*$ и значи:

$$S_1 \Rightarrow_{G_1}^* w \text{ или } S_2 \Rightarrow_{G_2}^* w$$

Откъдето: $w \in L_1$ или $w \in L_2$ □

Безконтекстните езици са затворени относно операцията звезда на Клини:

Доказателство: Нека L е безконтекстен език над азбуката Σ и нека $G = (\Gamma, \Sigma, R, S)$ е КСТ с $L(G) = L$.

Да разгледаме безконтекстната граматика: $G' = (\Gamma', \Sigma, R', S')$, където:

$$\begin{aligned}\Gamma' &= \Gamma \cup \{S'\} \\ S' &\notin \Gamma \\ R' &= R \cup \{S' \rightarrow \epsilon | S' S\}\end{aligned}$$

Имаме $L^* \subseteq L(G)$. Действително: $S' \Rightarrow_{G'} \epsilon$ и значи $\epsilon \in L(G)$. От друга страна, ако $w_1, \dots, w_n \in L$, то

$$S \Rightarrow_G^* w_i, 1 \leq i \leq n$$

Откъдето, заедно с $R \subseteq R'$ имаме:

$$S \Rightarrow_{G'}^* w_i, 1 \leq i \leq n$$

Оттук:

$$S' \Rightarrow_{G'}^* S' S \dots S \Rightarrow_{G'} S \dots S \Rightarrow_{G'}^* w_1 S \dots S \dots \Rightarrow_{G'}^* w_1 \dots w_n$$

И значи $w_1 \dots w_n \in L(G')$.

За обратното включване, както в предните две твърдения имаме, че ако $u \Rightarrow_{G'}^* v$ за някое $u \in \Gamma^*$, то $v \in \Gamma^*$ и $u \Rightarrow_G^* v$

Нека сега $w \in L(G)$ и да разгледаме един най-ляв извод на w от S . Той има вида

$$S' \Rightarrow_{G'}^* S' S \dots S \Rightarrow S \dots S \Rightarrow_{G'}^* w_1 S \dots S \dots \Rightarrow_{G'}^* w_1 \dots w_n$$

Където $n \geq 0$, $w_1 \dots w_n = w$ и $S \Rightarrow_{G'}^* w_i, 1 \leq i \leq n$. Но $S \in \Gamma^*$ и значи: $S \Rightarrow_G^* w_i, 1 \leq i \leq n$, откъдето $w_i \in L, 1 \leq i \leq n$. Следователно $w \in L^*$ $\square$

5.6 Лема за разрастването за контекстносвободни езици

Нормална форма на Чомски: Една безконтекстна граматика G е в нормална форма на Чомски (НФЧ), ако за всяко правило $A \rightarrow_G \alpha$ е в сила $|\alpha| = 2$

За всяка КСГ G съществува КСГ G' в НФЧ, т.че за всяка дума $w : |w| = 2$ е изпълнено $w \in L(G) \iff w \in L(G')$

Поддърво: T' е поддърво на дървото на извода T и пишем $T' \preceq T$ ако T' участва в изграждането на T или $T' \equiv T$

Твърдение: Нека G е КСГ в НФЧ. Нека T е дърво на извода в G с връх A и извод w . Нека T' е поддърво на T с връх B и извод α . Тогава съществуват думи w', w'' , т.че:

$$w = w' \alpha w'' \text{ и } A \Rightarrow_G^* w' B w''$$

При това, ако $T \not\equiv T'$, то $|w' w''| \geq 1$

Лема за разрастването: Нека L е безконтекстен език. Тогава съществува число $n_0 > 0$, т.че за всяка $w \in L, |w| > n_0$, съществуват думи x, y, z, u, v , такива че $w = xyzuiv, |yzu| \leq n_0, |yu| \geq 1$ и за всяко $i \geq 0$ е изпълнено $xy^i zu^i v \in L$

Доказателство: Нека G е КСГ в НФЧ, която генерира точно думите от L с дължина поне 2. Нека броят на нетерминалните символи на G е m . Полагаме $n_0 = 2^m$. Нека сега $w \in L$ и $|w| > n_0$. Тогава $|w| > 2$ (защото в G има поне един нетерминален символ) и следователно в G има дърво на извода T с връх S - началния символ на граматиката, и извод w .

Тъй като G е в НФЧ, то ако едно дърво на извода има височина q , то извода на това дърво е с дължина най-много 2^q (максималния брой на листата). Заедно с $|w| > 2^m$ получаваме, че височината на дървото T е $h > m$. Образоваме редица от поддървета на T :

$$T = T_0 \prec T_1 \prec \dots \prec T_h$$

Нека T_i е с връх A_i и извод $w_i, 0 \leq i \leq h$. Височината на T_i е $h - i$. Редицата

$$A_{h-m-1}, A_{h-m}, \dots, A_{h-1}$$

съдържа $m + 1$ нетерминални символа и значи поне един от тях се повтаря два пъти. Нека $h - m - 1 \leq j < k \leq h - 1$ са такива, че $A_j = A_k$. От $T \prec T_j$ и горното твърдение следва, че съществуват думи x, v :

$$w = x w_j v \text{ и } S \Rightarrow_G^* x A_j v$$

От друга страна $T_j \succ T_k$, откъдето съществуват думи y, u , т.че:

$$w_j = y w_k u \text{ и } A_j \Rightarrow_G^* y A_j u$$

като при това $|yu| \geq 1$. Оттук, полагайки $z = w_k$ и използвайки $A_j = A_k$ получаваме:

$$S \Rightarrow_G^* xA_jv \Rightarrow_G^* xyA_juv \Rightarrow_G^* xyxA_juuv \Rightarrow_G^* \dots \Rightarrow_G^* xy^iA_jv^iz \Rightarrow_G^* xy^izu^iv$$

Освен това $yzu = w_j$ и тъй като височината на T_j е най-много m , то $|yzu| \leq 2^m = n_0$

5.7 Пример с доказателство за неконтекстносвободен език

Използвайки лемата за разрастването ще докажем, че езикът:

$$L = \{0^k1^k2^k \mid n \geq 0\}$$

не е контекстносвободен. Нека $n > 0$. Да разгледаме думата

$$w = 0^n1^n2^n$$

Тогава $w \in L$ и $|w| = 3n > n$. Нека x, y, z, u, v са думи такива, че: $w = xyzuv$, $|yzu| \leq n$, $|yu| \geq 1$. Тогава yzu не съдържа нули или не съдържа двойки (може и двете). Б.О.О нека yzu не съдържа двойки. Да разгледаме:

$$w_0 = xy^0zu^0v = xzv$$

Тогава w_0 също съдържа точно n на брой двойки, но тъй като $|yu| \geq 1$, то $|w_0| < 3n$ и следователно:

$$w_0 \notin L$$

Оттук и лемата за разрастването следва, че L не е безконтекстен език.

5.8 Контекстносвободните езици не са затворени относно сечение и допълнение

Безконтекстните езици не са затворени относно сечение: Ще докажем с пример:

Доказателство: Нека

$$\begin{aligned} L_1 &= \{0^k1^k2^s \mid k, s, \geq 0\} \\ L_2 &= \{0^k1^s2^s \mid k, s, \geq 0\} \end{aligned}$$

Езикът L_1 е безконтекстен, тъй като е конкатенация на безконтекстния $\{0^k1^k \mid k \geq 0\}$ и регулярния $2^s \mid s \geq 0$. Аналогично за L_2 . От друга страна:

$$L_1 \cap L_2 = \{0^k1^k2^k \mid k \geq 0\}$$

който не е безконтекстен език.

Безконтекстните езици не са затворени относно операцията допълнение

Доказателство: По Де Морган:

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

От това, че КСЕ са затворени относно обединение, но не относно сечение, следва, че КСЕ не са затворени относно допълнение.

6 Сортиране чрез сравнения във време $O(n \lg n)$

6.1 Двоична пирамида определение и свойства.

Двоична пирамида: Двоична пирамида е попълнено двоично дърво, в което или ключът на всеки връх е по-голям или равен на ключовете на неговите деца и тогава пирамидата е максимална пирамида (max heap), или ключът на всеки връх е по-малък или равен на ключовете на неговите деца (min heap).

Свойство: Попълнено двоично дърво с ключове е пирамида тогава и само тогава, когато за всяко листо u , по уникалния път, свързващ корена с u , стойностите на ключовете са ненарастващи (или ненамаляващи) в посока от корена към u .

Свойство: Нека T е пирамида с n елемента и представена чрез масив $A[1..n]$. Тогава за всеки елемент $A[i]$:

1. Елементът, който отговаря на родителя на $A[i]$ е $A[\lfloor \frac{i}{2} \rfloor]$, ако върхът на T , отговарящ на $A[i]$, не е коренът.
2. Елементът, който отговаря на лявото дете на $A[i]$, е $A[2i]$, ако върхът на T , отговарящ на $A[i]$ има ляво дете.
3. Елементът, който отговаря на дясното дете на $A[i]$, е $A[2i + 1]$, ако върхът на T , отговарящ на $A[i]$ има дясно дете.

6.2 Наивно построяване на двоична пирамида: доказателство за коректност и изследване на сложността по време.

Най-естественият начин да бъде превърнат произволен масив от числа в пирамида чрез разместване е чрез последователно добавяне на елементи в частично построена пирамида.

NaiveBuildHeap($A[1..n]$: array of integers)

```
@1. for  $i = 1$  to  $n$  do Heapify( $A[1..n]$ ,  $i$ )
@2. end for
```

Heapify($A[1..n]$: array of integers, l : index in A)

```
@1.  $p \leftarrow \lfloor \frac{l}{2} \rfloor$ 
@2. while  $p > 0$  &  $A[p] > A[l]$  do
@3.    $swap(A, p, l)$ 
@4.    $l \leftarrow p$ 
@5.    $p \leftarrow \lfloor \frac{l}{2} \rfloor$ 
@6. end while
```

Твърдение: Нека $A[1..l - 1]$ е представяне на двоична пирамида в $A[1..n]$.

Тогава $Heapify(A, l)$ модифицира A до A' , т. че:

1. $Time_{Heapify}(A, i) \in O(\log_2(i + 1))$
2. $A'[1..l]$ е вариант на $A[1..l]$ и е представяне на двоична пирамида
3. $A'[l + 1..n] = A[l + 1..n]$

Доказателство: Разглеждаме следния инвариант:

$$I(k) \stackrel{def}{\iff} \forall j \neq 1, i_k (j \leq i_k \rightarrow A_k[\lfloor \frac{j}{2} \rfloor] \leq A_k[j]) \ \& \ ((2i_k \leq i \rightarrow A_k[p_k] \leq A_k[2i_k]) \ \& \ (2i_k + 1 \leq i \rightarrow A_k[p_k] \leq A_k[2i_k + 1]))$$

// (1) Ни казва, че пирамидалното свойство се запазва за елементите "над" i_k

// (2) Ни казва, че родителят на i_k е по-малък от лявото дете на i_k

// (3) Ни казва, че родителят на i_k е по-малък от дясното дете на i_k

Нека p_k, i_k, A_k са състоянията на p, i, A преди k -тото изпълнение на @2

1) $I(1)(1)$ от $A[1..i - 1]$ е двоична пирамида.

2) Нека $I(k), p_{k+1}, i_{k+1}, A_{k+1}$ са дефинирани $\Rightarrow p_k > 0, A_k[p_k] > A_k[i_k], j \neq p_k, i_k$

От @3 имаме:

$$A_{k+1}[j] = \begin{cases} A_k[j], & j \neq i_k, p_k \\ A_k[p_k], & j = i_k \\ A_k[i_k], & j = p_k \end{cases}$$

3) (1) Нека $j \leq i, j \neq i_{k+1} = p_k$

$$(1.1) \lfloor \frac{j}{2} \rfloor \neq i_k, p_k. \text{ Тогава } A_{k+1}[\lfloor \frac{j}{2} \rfloor] = A_k[\lfloor \frac{j}{2} \rfloor] \stackrel{\text{и.х.}}{\leq} A_k[j]$$

$$\text{Ако } j \neq i_k, \text{ то } A_{k+1}[j] = A_k[j] \geq A_{k+1}[\lfloor \frac{j}{2} \rfloor]$$

$$\text{Ако } j = i_k, \text{ то } \lfloor \frac{j}{2} \rfloor = p_k = i_{k+1}. \text{ Но } A_{k+1}[\lfloor \frac{j}{2} \rfloor] = A_{k+1}[p_k] = A_k[i_k] < A_k[p_k] = A_{k+1}[i_k] = A_{k+1}[j]$$

$$(1.2) \lfloor \frac{j}{2} \rfloor = i_k. \text{ Тогава } j \in \{2i_k, 2i_k + 1\} \Rightarrow A_{k+1}[j] = A_k[j]$$

$$\text{Оттук } A_{k+1}[2i_k] = A_k[2i_k] \stackrel{\text{и.х.}}{\geq} A_k[p_k] = A_{k+1}[i_k] = A_{k+1}[\lfloor \frac{j}{2} \rfloor]$$

$$\text{Аналогично } A_{k+1}[2i_k + 1] \stackrel{\text{и.х.}}{\geq} A_k[p_k] = A_{k+1}[\lfloor \frac{j}{2} \rfloor]$$

Докажем (1) с индукция.

(2) Нека $2i_{k+1} \leq i$. Тогава:

$$A_{k+1}[2i_{k+1}] = \begin{cases} A_{k+1}[i_k] = A_k[p_k], & 2i_k = p_k \\ A_k[2i_{k+1}], & \text{иначе} \end{cases}$$

$$\text{Но } p_{k+1} = \lfloor \frac{p_k}{2} \rfloor \text{ и от } I(k) \Rightarrow A_{k+1}[p_{k+1}] = A_k[p_{k+1}] \leq A_k[p_k] \leq A_k[p_{k+1}]$$

Докажем (2) с индукция. (3) е аналогично.

По индукция $I(k)$ е вярно за всяко k и следователно ако p_{k+1} не е дефинирано, то $p_k = 0$ или $A[i_k] \geq A[p_k]$.

Ако $A[i_k] \geq A[p_k]$, от $I(k)$, (1) получаваме, че $A[1..i]$ е пирамида.

Ако $p_k = 0, p_k = \lfloor \frac{j_k}{2} \rfloor$, т.е. $i_k = 1$ и от $I(k)$, (1) получаваме, че $A[1..i]$ е пирамида.

Коректност на NaiveBuildHeap: За всеки вход A , *NaiveBuildHeap* построява пирамида от него.

Доказателство: След i -тата итерация $A[1..i]$ е валидна двоична пирамида.

- База: $A[1]$ след първата итерация едноелементният $A[1]$ е двоична пирамида.
- Нека е вярно, че $A[1..i]$ реализира пирамида. Разглеждаме $A[1..i+1]$. На @2 се извиква *Heapify*($A[1..n], i+1$). От коректността на *Heapify* имаме, че $A[1..i+1]$ е двоична пирамида, а $A[i+2..n]$ е непроменено.
- Терминация: За $i = n+1$ имаме, че след n -тата итерация $A[1..n]$ е двоична пирамида.

Времева сложност на BuildNaiveHeap:

1. Знаем, че $\text{Time}_{\text{Heapify}}(A, i) \in O(\log i)$
2. Имаме, че общото време за *BuildNaiveHeap* е сумата от времената на всяко извикване:

$$T(n) = \sum_{i=2}^n O(\log i)$$

3. $\sum_{i=2}^n \log i = \log(n!) \approx n \log n$, откъдето: $\text{Time}_{\text{BuildNaiveHeap}}(A[1..n]) \in O(n \log n)$

6.3 Бързо построяване на двоична пирамида, използващо функция *Heapify*: доказателство за коректност и изследване на сложността по време.

Идеята тук е ако един връх нарушава пирамидалното свойство, то да го разменим с по-малкото (или с по-голямото при Max-heap) дете, стига и двете деца (ако съществуват) да са пирамиди.

Heapify($A[1..n]$: array of integers, i : index in A)

- @1. $j \leftarrow i$
- @2. **while** $j \leq \lfloor \frac{n}{2} \rfloor$ **do**
- @3. $\text{left} \leftarrow \text{Left}(j)$

```

@4.   right ← Right(j)
@5.   if  $A[\textit{left}] > A[j]$  then
@6.     largest ← left
@7.   else
@8.     largest ← j
@9.   end if
@10.  if  $\textit{right} \leq n$  and  $A[\textit{right}] > A[\textit{largest}]$  then
@11.    largest ← right
@12.  end if
@13.  if  $\textit{largest} \neq j$  then
@14.    j ← largest
@15.  else
@16.    break
@17.  end if
@18. end while

```

Коректност на Heapify: При допускането, че $A[1..n]$ и i са такива, че ако децата на $A[i] : [L(i), A[R(i)]]$ са пирамиди, то след изпълнение на *Heapify* $A[i]$ също е пирамида.

Доказателство: При всяко достигане **след първото** на @3, $A[L(j)], A[R(j)]$ са пирамиди. Освен това, всички пирамидални инверсии в $A[i]$ се намират в $A[j]$.

База: Разглеждаме първото достигане на @3. В този момент $i = j$. Твърдението е вярно предвид допускането, че децата на $A[i]$ са пирамиди.

И.Х.: Допускаме, че инвариантът е в сила при някое достигане на @3, което не е последното. Оттук @16 няма да бъде изпълнен, както и $j \leq \lfloor \frac{n}{2} \rfloor$. Следователно $L(j) \leq n$, което означава, че пирамидата $A[L(j)]$ съществува. Б.О.О ще допуснем, че $A[R(j)]$ също съществува, т.е. $R(j) \leq n$.

Разглеждаме следните случай в следващото изпълнение на цикъла, които са взаимно изключващи се:

Случай 1: $A[j] < A[L(j)] < A[R(j)]$

При изпълняване на цикъла имаме, че се разменят $\textit{right} = \textit{largest}$ и j . Размяната не засяга $A[L(j)]$, откъдето то продължава да бъде пирамида. Нопирамидална инверсия вече може да има в $A[R(j)]$. От И.Х. имаме, че $A[L(R(j))]$ и $A[R(R(j))]$ са пирамиди. Изпълнява се @14 и $j = \textit{largest} = \textit{right}$. При следващото достигане на @3 имаме, че $A[L(j)]$ и $A[R(j)]$ са пирамиди и всички пирамидални инверсии в пирамидата с връх $A[i]$ се намират в подпирамидата с връх $A[j]$.

Случай 2: $A[L(j)] \leq A[j] < A[R(j)]$

Случай 3: $A[j] \leq A[R(j)] < A[L(j)]$

Случай 4: $A[R(j)] \leq A[j] < A[L(j)]$

Аналогични на **Случай 1**

Случай 5: $A[\textit{left}] \not\leq A[j], A[\textit{right}] \not\leq A[j]$

Но това е невъзможно при текущите допускания, защото очевидно тогава @16 би се изпълнил, откъдето няма как да се върнем на @3.

Терминация: Цикълът може да бъде напуснат по два начина: или през @3 ($j > \lfloor \frac{n}{2} \rfloor$), или през @16.

- $j > \lfloor \frac{n}{2} \rfloor$: Тогава $A[j]$ е листо и не може да има пирамидални инверсии. От инварианта следва, че в $A[i]$ няма пирамидални инверсии и следователно е пирамида.
- Цикълът е приключил на @16. За да бъде достигнат @16, трябва да бъде вярно, че $\textit{largest} = j$ на @12. За това трябва да е в сила и $A[\textit{left}] \leq A[j], A[\textit{right}] \leq A[j]$, защото това е единствения начин @8 да бъде достигнат и @11 да не бъде достигнат. Но ако $A[\textit{left}] < A[j]$ и $A[\textit{right}] < A[j]$, то съгласно инварианта, пирамидални инверсии в $A[i]$ няма и следователно е пирамида.

Сложност по време: $\Theta(\log m)$, където m е броят на елементите на $A[i]$. Това следва от факта, че височината на $A[i]$ е $\Theta(\log m)$, а коренът $A[i]$ може да измине най-много път с дължина, равна на височината.

Идеята на бързия алгоритъм за построяване на пирамида е да сканира масива отдясно наляво, иначе казано, от листата към корена, започвайки от първото не-листо. Листата са тривиални пирамиди. За всяко не-листо $A[i]$ в указания ред, при допускането, че и двете му деца (или едното му дете, ако друго дете няма) са корени на подпирамиди, ако $A[i]$ е по-голям или равен от двете си деца (или едното си дете), то $A[i]$ е подпирамида. Ако това не е вярно, ще направим $A[i]$ пирамида чрез серия от размени на елементи в нея, разменяйки

корен с по-голямото дете, избутвайки малкия елемент, който образува пирамидални инверсии, в посока към листата, докато той или не стане листо, или не се окаже по-голям или равен на децата си (детето си).

BuildHeap($A[1..n]$: array of integers)

```
@1. for  $i \leftarrow \lfloor \frac{n}{2} \rfloor$  downto 1 do
@2.   Heapify( $A, i$ )
@3. end for
```

Коректност на BuildHeap: За всеки вход A , BuildHeap построява пирамида от него.

Доказателство: Всеки път, когато изпълнението е на @1, $A[i+1], A[i+2], \dots, A[n]$ са пирамиди.

База: При първото достигане на @1, $i = \lfloor \frac{n}{2} \rfloor$. Тогава $A[i+1], A[i+2], \dots, A[n]$ са точно листата на попълненото дърво, чието представяне като масив е A . Всяко листо е пирамида.

Поддръжка: Да разгледаме някое достигане на @1, което не е последното. $A[Left(i)], A[Right(i)]$ (ако съществува) са пирамиди съгласно И.Х., понеже $i < Left(i), i < Right(i)$. Тогава използваме доказаната коректност на Heapify, откъдето $A[i]$ става пирамида. След това i намалява с 1. Спрямо новата стойност на i , инвариантът е в сила.

Терминация: Когато изпълнението е на @2 за последен път, $i = 0$. Тогава $A[0+1], A[0+2], \dots, A[n]$ са пирамиди. В частност $A[1]$ е пирамида. Но $A[1]$ е A . $\square$

Бързото построяване на пирамида става в линейно време:

Асимптотична горна графница за сложността на BuildHeap е сумата по всички височини от 1 (прескачаме листата) до $\lfloor \lg n \rfloor$ (височината на дървото) от произведението от броя на върховете с дадената височина и самта височина. (Един връх може да се придвижва само надолу)

Имаме, че броят на върховете с височина k е $\left\lfloor \frac{n}{2^k} \right\rfloor$.

Ако $T(n)$ е функция на сложността, то в най-лошия случай:

$$T(n) = \sum_{k=1}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^k} \right\rfloor \Theta(k) = \sum_{k=1}^{\lfloor \lg n \rfloor} \Theta\left(\frac{n}{2^k} k\right)$$

В сила е:

$$\sum_{k=1}^{\lfloor \lg n \rfloor} \frac{n}{2^k} k \leq \sum_{k=1}^{\infty} \frac{n}{2^k} k = n \sum_{k=1}^{\infty} \frac{k}{2^k}$$

Лесно се доказва, че $\sum_{k=1}^{\infty} \frac{k}{2^k}$ е сходящ:

$$\lim_{k \rightarrow \infty} \frac{\frac{k+1}{2^{k+1}}}{\frac{k}{2^k}} = \frac{1}{2} < 1$$

Щом $\sum_{k=1}^{\infty} \frac{k}{2^k}$ е ограничен от константа, то $n \sum_{k=1}^{\infty} \frac{k}{2^k} \approx n$ и очевидно $\sum_{k=1}^{\lfloor \lg n \rfloor} \frac{n}{2^k} k \approx n$.

Тогава $T(n) \approx n$ $\square$

6.4 Сортиращ алгоритъм HEAPSORT: псевдокод, доказателство за коректност и изследване на сложността по време

Heapsort($A[1..n]$)

```
@1. BuildHeap( $A$ )
@2. for  $i \leftarrow n$  downto 2 do
@3.   swap( $A[1], A[i]$ )
@4.   Heapify( $A[1..i-1], 1$ )
@5. end for
```

Коректност на Heapsort: Heapsort е сортиращ алгоритъм

Доказателство: Нека $A_0[1..n]$ означава първоначалния масив. Следното твърдение е инвариант за цикъла:

Всеки път, когато изпълнението на Heapsort е на @2:

- Текущият подмасив $A[i+1..n]$ се състои от $n-i$ на брой най-големи елементи на $A_0[1..n]$, в сортиран вид.

- Текущия $A[1..i]$ е пирамида.

База: При първото достигане на @2 $i = n$. Подмасивът $A[i+1..n]$ е празен. След изпълнението на BuildHeap имаме, че $A[1..n]$ е пирамида.

Поддръжка: Да допуснем, че твърдението е в сила при някакво достигане на @2, което не е последното. Имаме, че $A[i+1..n]$ се състои от $n-i$ на брой елементи на A_0 в сортиран вид. Съгласно допускането имаме също че $A[1]$ е максимален елемент на $A[1..i]$ (max-heap).

След изпълнението на @3 имаме, че $A[i..n]$ съдържа $n-i+1$ на брой най-големи елемента на A_0 в сортиран вид. За $i-1$ първата част е изпълнена.

След @3 имаме, че $A[1..i]$ може да не е пирамида. Но от това, че $A[1..i]$ е пирамида, следва, че $A[2], A[3]$ са пирамиди, защото разместването не ги засяга. На @4 прилагаме Heapify върху $A[1..i-1]$, откъдето $A[1..i-1]$ е пирамида след изпълнението на Heapify. В следващата итерация i се декраментира, откъдето имаме именно, че $A[1..i-1]$ е пирамида.

Терминация: При $i = 1$ имаме, че $A[2..n]$ се състои от $n-1$ най-големи елемента на $A_0[1..n]$ в сортиран вид. Оттук следва, че $A[1]$ е минимален елемент на $A[1..n]$. Следователно $A[1..n]$ се състои от n на брой най-големи елементи на $A_0[1..n]$ в сортиран вид.

Сложност по време на Heapsort: Сложността на BuildHeap е $\Theta(n)$, а сложността на for-цикълъ е $\Theta(n \lg n)$, откъдето сложността на Heapsort е $\Theta(n \lg n)$.

6.5 Сортиращ алгоритъм MERGESORT като пример за алгоритъм по схемата Разделяй и Владей

Ако размерът на входа n е по-голямо от някаква прагова стойност n_0 , във фазата Разделяй, Mergesort дели входа на две равни части, всяка с размер $\frac{n}{2}$, без да променя наредбата на елементите. Във фазата Владей прави по едно рекурсивно викане върху всяка от двете части. Във фазата Комбинирай всяка от двете части вече е сортирана, така че алгоритъмът слива двете сортирани части в една окончателна сортирана редица. Акцентът е върху третата фаза, където се извършва истинската работа по сортирането.

Ако размерът на входа не надхвърля n_0 , то се задейства спирачката на рекурсията и входът бива сортиран по някакъв елементарен начин, примерно с Insertion Sort.

6.6 Псевдокод и доказателство за коректност на MERGESORT

Merge($A[1..n]$: array of integers; l, m, r : indices in A ; $B[1..n]$: array of integers)

```

@1.  $j \leftarrow l$ 
@2.  $k \leftarrow m$ 
@3. for  $i = l$  to  $r-1$  do
@4.   if  $(k = r)$  or  $(k < r \ \& \ j < m \ \& \ A[j] \leq A[k])$  then
@5.      $B[i] \leftarrow A[j]$ 
@6.      $j \leftarrow j + 1$ 
@7.   else
@8.      $B[i] \leftarrow A[k]$ 
@9.      $k \leftarrow k + 1$ 
@10.  end if
@11. end for

```

Твърдение: Ако $A[1..n]$ е масив от числа, $1 \leq l \leq m \leq r \leq n+1$ и $A[l..m-1]$ & $A[m..r-1]$ са сортирани възходящо и още масивите A, B не споделят обща памет, то $Merge(A, l, m, r, B)$ модифицира $B[l..r-1]$ до $B'[l..r-1]$ - сортиран вариант на $A[l..r-1]$. При това $A, B[1..l-1], B[r..n]$ не се променят.

Доказателство: 1) A не се променя, защото няма обща памет с B .

2) Променя се само B , при това само $B[i]$, $l \leq i \leq r-1 \implies B[1..l-1], B[r..n]$ не се променят.

Нека B_s, j_s, k_s, i_s са състоянията на B, j, k, i непосредствено преди s -тата итерация на @3.

Ред @3 се изпълнява $r-l-1+1 = r-l$ пъти.

$$\begin{aligned}
& (1) B_s[l..i_s - 1] \text{ е сортиран вариант на } A[l..j_s - 1] \circ A[m..k_s - 1] \text{ \&} \\
& (2) j_s \leq m \text{ \&} \\
I(s) \iff & (3) k_s \leq r \text{ \&} \\
& (4) j_s < m, i_s > l \Rightarrow B_s[i_s - 1] \leq A[j_s] \text{ \&} \\
& (5) k_s < r, i_s > l \Rightarrow B_s[i_s - 1] \leq A[k_s]
\end{aligned}$$

С индукция по s показваме, че $I(s)$ е истина за $1 \leq s \leq r - l$.

$i = 1$: $I(1) \checkmark$ ($j_1 = l, k_1 = m, i_1 = l$)

$s \rightarrow s + 1$: Нека $I(s)$ е вярно.

Тъй като при всяко изпълнение на тялото на цикъла се увеличава точно една от променливите j_s или k_s , то $(k_s - m) + (j_s - l) = i_s - l$

$$\Rightarrow \text{Ако } k_s = r, \text{ то } r - m + j_s - l = i_s - l \Rightarrow r - i_s = m - j_s$$

$r > i_s$, защото се изпълнява тялото на цикъла, откъдето $m > j_s$.

1сл. if-ът на @4 е истина.

$$\begin{aligned}
\Rightarrow B_{s+1}[i_s] &= A[j_s] \stackrel{(1)}{\geq} B_{s+1}[i_{s-1}] \\
B_{s+1}[i_s] &\leq A[j_s + 1] \text{ (4)} \\
B_{s+1}[i_s] &= A[j_s] < A[k_s] \text{ (5)}
\end{aligned}$$

2сл. Аналогично.

Забележка: $Time_{Merge}(A, l, m, r, B) \in O(r - l + 1)$.

MergeSort($A[1..n]$: array of integers; l, r : indices in A ; $B[1..n]$: array of integers)

@1. $m \leftarrow \lfloor \frac{l+r}{2} \rfloor$
@2. if $r - l \leq 1$ return
@3. $MergeSort(A, l, m, B)$
@4. $MergeSort(A, m, r, B)$
@5. $Merge(A, l, m, r, B)$
@6. $Copy(B, l, r, A)$

Copy($B[1..n]$: array of integers; l, r : indices in B ; $A[1..n]$: array of integers)

@1. for $i = l$ to $r - 1$ do $A[i] = B[i]$
@2. end for

Твърдение: $MergeSort(A, l, r, B)$ завършва за всеки $l \leq r$ и ако A и B не споделят обща памет $MergeSort$ сортира $A[l..r - 1]$ без да променя останалите елементи на A .

Доказателство: Индукция по $r - l$:

$r - l \leq 1$: MergeSort завършва на @1 и очевидно $A[l..r - 1] \in \{A[l], \varepsilon\}$ е сортиран.

$r - l > 1$. Нека $r - l > 1$. Тогава $m = \lfloor \frac{l+r}{2} \rfloor \leq \frac{l+r}{2} < \frac{2r}{2} = r$.

Следователно $m - l < r - l$ и също имаме, че: $m = \lfloor \frac{l+r}{2} \rfloor \geq \frac{l+r-1}{2} > \frac{l+l+1-1}{2} = l$

Откъдето: $r - m < r - l$.

От И.Х. за $m - l$, $MergeSort(A, l, m, b)$ сортира $A[l..m - 1]$ без да променя останалите елементи на A .

Сега от свойството на $Merge$ след @5 $B[l..r - 1]$ е сортиран вариант на $A[l..r - 1]$ без да се променят останалите елементи на A .

След @6 $A[l..r - 1] = B[l..r - 1]$ ще е сортиран вариант на входния $A[l..r - 1]$ без да се променят останалите елементи на A . $\square$

6.7 Изследване на сложността по време на MERGESORT

Твърдение: $Time_{MergeSort}(A, l, r, B) = O((r - l) \log_2(r - l))$

Доказателство: Нека $Time(m) = \max\{Time_{MergeSort}(A, l, r, B) \mid r - l \leq m\}$.

Тогава $Time(m)$ е монотонно растяща функция. Имаме:

$$Time(2^s) \geq Time(m), \quad m \leq 2^s$$

Ако $r - l \leq 2^s$, тогава:

$$r - \left\lfloor \frac{r+l}{2} \right\rfloor \leq 2^{s-1} \text{ \& } \left\lfloor \frac{r+l}{2} \right\rfloor - l \leq 2^{s-1}$$

Тогава:

$$Time(2^s) \leq Time(2^{s-1}) + Time(2^{s-1}) + c2^s, \quad c > 0$$

$$Time(2^s) \leq 2Time(2^{s-1}) + c2^s$$

Нека a_s е редицата:

$$a_0 = Time(2^0) = Time(1)$$

$$a_s = 2a_{s-1} + c2^s$$

Тогава по индукция:

$$1) \quad a_0 \geq Time(2^0)$$

$$2) \quad \text{Ако } a_s \geq Time(2^s), \text{ то:}$$

$$a_{s+1} = 2a_s + c2^{s+1} \geq 2Time(2^s) + c2^{s+1} \geq Time(2^{s+1})$$

Решението на $a_s = 2a_{s-1} + c2^s$ има общ вид $a_s = 2^s(\alpha_s + \beta)$.

Нека $n \in \mathbb{N}$. Избираме $s : 2^s \geq n \geq 2^{s-1}$.

$$\begin{aligned} \text{Тогава: } Time(n) &\leq Time(2^s) \leq 2^s(\alpha_s + \beta) = 2 \cdot 2^{s-1}(\alpha(s-1) + \alpha + \beta) = 2n(\alpha \log_2 n + \alpha + \beta) \\ \Rightarrow Time(n) &\in O(n \log_2 n) \Rightarrow Time_{MergeSort}(A, l, r, B) \in O((r-l) \log_2(r-l)) \end{aligned}$$

Твърдение: Нека P е алгоритъм, който детерминирано сортира всеки свой вход от цели числа $A[1..n]$ като единствените операции на P върху A са $A[i] \ A[j]$, $\in \{<, \leq, =, \geq, >\}$.

Тогава за всяко достатъчно голямо n има вход $A[1..n]$ за P , върху когото P извършва $c \cdot n \log n$ сравнения, $c > 0$.

Доказателство: Да забележим, че за всеки вход $\pi \in S_n$ на P сортиращата пермутация π е уникална.

Това означим, че ако за пермутация $\pi \in S_n$ с $s(\pi)$ означим броя сравнения, които извършва P върху π , а с $\varepsilon_1(\pi) \circ \dots \circ \varepsilon_s(\pi) \in \{0, 1\}^{S(\pi)}$ означим отговорите на тези сравнения, то:

$$\pi_1 \neq \pi_2 \Rightarrow \varepsilon_1(\pi_1) \circ \dots \circ \varepsilon_s(\pi_1) \neq \varepsilon_1(\pi_2) \circ \dots \circ \varepsilon_s(\pi_2)$$

Тогава от една страна всички пермутации са $|S_n| = n!$, а от друга думите $\in \{0, 1\}$ и дължина $\leq f(n) = \max\{s(\pi) \mid \pi \in S_n\}$ са:

$$\sum_{k=0}^{f(n)} 2^k = 2^{f(n)+1}$$

Имаме: $n! < 2^{f(n)+1} \Rightarrow \log_2 n! < f(n) + 1$

$$\log_2 n! = \sum_{k=1}^n \log_2 k = \log_2 e \sum_{k=2}^n \ln k$$

$$\sum_{k=2}^n \ln k = \sum_{k=1}^{n-1} \int_k^{k+1} \ln(k+1) dx \geq \sum_{k=1}^{n-1} \int_k^{k+1} \ln x dx = \int_1^n \ln x dx$$

, защото $\ln x$ е растяща функция. Интегрираме по части:

$$\begin{aligned} \int_1^n \ln x dx &= x \ln x \Big|_1^n - \int_1^n x d \ln x = n \ln n - n + 1 \\ \Rightarrow \log_2 e(n \ln n - n + 1) &\leq \log_2 n! < f(n) \end{aligned}$$

6.8 Приложение на MERGESORT за намиране на инверсиите в масив от числа, с доказателство за коректност

Merge-Modified($A[1..n]$:int, l , mid , h : $1 \leq l < mid < h \leq n$)

```

@1.  $n_1 \leftarrow mid - l + 1$ 
@2.  $n_2 \leftarrow h - mid$ 
@3. създаваме  $L[1..n_1 + 1], R[1..n_2 + 1]$ 
@4.  $L[1..n_1] \leftarrow A[l..mid]$ 
@5.  $R[1..n_2] \leftarrow A[mid + 1..h]$ 
@6.  $L[n_1 + 1] \leftarrow \infty$ 
@7.  $R[n_2 + 1] \leftarrow \infty$ 
@8.  $i \leftarrow 1$ 
@9.  $j \leftarrow 1$ 
@10.  $c \leftarrow 0$ 
@11. for  $k \leftarrow l$  to  $h$  do
@12.   if  $L[i] \leq R[j]$  then
@13.      $A[k] \leftarrow L[i]$ 
@14.      $i++$ 
@15.   else
@16.      $A[k] \leftarrow R[j]$ 
@17.      $c \leftarrow c + n_1 - i + 1$ 
@18.      $j++$ 
@19.   end if
@20. end for
@21. return  $c$ 

```

[1]

Обосновка за коректност на Merge-Modified: При подадени на вход $A[1..n], l, mid, h$, т.че $A[l..mid], A[mid + 1..h]$ са сортирани Merge-Modified връща броя на инверсиите между $A[l..mid]$ и $A[mid + 1..h]$.

- Ако $L[i] \leq R[j]$, то $\langle L[i], R[j] \rangle$ не е инверсия. Тъй като R е сортиран, $L[i]$ не е в инверсия с никое $R[j'], j \leq j' \leq n_2$. Броят на инверсиите в $L[i..n_1] \circ R[j..n_2]$ е равен на броя на инверсиите в $L[i + 1..n_1] \circ R[j..n_2]$
- Ако $L[i] > R[j]$, то $\langle L[i], R[j] \rangle$ е инверсия. Тъй като L е сортиран $\langle L[i + 1], R[j] \rangle, \dots, \langle L[n_1], R[j] \rangle$ също са инверсии. Тогава броят на инверсиите в $L[i..n_1] \circ R[j..n_2]$ е равен на броя на инверсиите в $L[i + 1..n_1] \circ R[j..n_2] + n_1 - i + 1$ (броят на елементите на $L[i..n_1]$).

7 Минимални покриващи дървета.

7.1 Дефиниция на задачата - какъв тип графи се разглеждат и какво се иска.

Тегловен граф: Наредена двойка G, w от граф $G = (V, E)$ и тегловна функция $w : E \rightarrow \mathbb{R}$

Покриващ граф: Покриващ граф на $G = (V, E)$ е всеки подграф $G' = (V, E')$ на G . Покриващо дърво на G (ПД) е подкриващ подграф на G , който е дърво.

Тегло на ПД: Нека G е свързан тегловен граф с тегловна функция w . Нека T е ПД на G . Теглото на T е $w(T) = \sum_{e \in E(T)} w(e)$. Минимално покриващо дърво (МПД) е подкриващо дърво $T : w(T) \leq w(T')$ за всяко друго ПД T' .

Задачата за МПД: При подадена наредена двойка (G, w) от неориентиран свързан граф G и тегловна функция върху ребрата му w да се намери МПД T на G .

7.2 Теорема за съгласуваното множество (условия за нарастване на подмножество на МПД) с доказателство.

Сигурно ребро: Нека G е тегловен граф и $A \subseteq E$ е, такова че съществува МПД T , такова че $A \subseteq E(T)$. Нека e е ребро за $G : e \notin A$. Казваме, че e е сигурно за A , ако съществува МПД T' , такова че $A \cup \{e\} \subseteq E(T')$.

Обща схема на алгоритъм за МПД:

МПД(G : граф, w : теглова функция):

- @1. $A \leftarrow \emptyset$
- @2. **while** A не образува ПД **do**
- @3. намери ребро e , което е сигурно за A
- @4. $A \leftarrow A \cup \{e\}$
- @5. **end while**
- @6. **return** A

Срез в граф: Нека G е граф. Срез в граф G е всяко разбиване на $V(G)$ на два дяла. Нека $S = \{V_1, V_2\}$ е срез в G . Ребро $e = (u, v)$ прекосява S , ако $u \in V_1, v \in V_2$. Нека $E' \subseteq E(G)$. S е съобразен с E' , ако нито едно ребро от E' не го прекосява. Ако G е тегловен и реброто e прекосява S , казваме че реброто e е леко, ако е има минимално тегло измежду всички ребра, които прекосяват S .

МПД теоремата: Нека $G = (V, E)$ е свързан тегловен граф с тегловна функция w . Нека $A \subseteq E$ е, такова че съществува МПД $T : A \subseteq E(T)$. Нека $S = \{V_1, V_2\}$ е срез в G , който е съобразен с A . Нека $e = (u, v)$ е произволно леко ребро, прекосяващо S . Тогава e е сигурно за A .

Доказателство: Нека T е МПД, такова че $A \subseteq E(T)$. Ако $e \in E(T)$, то няма какво да докажем.

Нека $e \notin E(T)$. Ще покажем, че съществува МПД T' , такова че $A \cup \{e\} \subseteq E(T')$. Добавяме e към T и получаваме покриващ граф U , който има единствен цикъл C . Очевидно $e \in E(C)$ и e прекосява среза S . Но съществува ребро $e' \in E(C)$, такова че $e' \neq e$, което също прекосява среза S . Изтриваме e' от U и получаваме ПД $T' \neq T$. Да сравним $w(T)$ и $w(T')$. T' се получи от T с добавяне на e и изтриване на e' :

$$w(T') = w(T) + w(e) - w(e')$$

Но e е леко по конструкция, следователно $w(e) \leq w(e')$, откъдето $w(e) - w(e') \leq 0$. Тогава $w(T') \leq w(T)$. Но T е МПД, така че $w(T) \leq w(T')$ по дефиниция. Следователно $w(T) = w(T')$. Оттук T' също е МПД, което съдържа както ребрата от A , така и e .

7.3 Алгоритъм на Прим: псевдокод и доказателство за коректност. Изследване на сложността по време при използване на различни структури от данни.

Алгоритъмът на Prim за намиране на МПД далечно прилича на обхождане в ширина. Той започва от един стартов връх s , намира най-леко ребро, инцидентно с s , да го наречем (s, u) , слага това ребро в дървото, после намира най-леко ребро, единият край на който е s или u , а другият край е връх, различен от s и u , слага го в дървото, и така нататък.

Ще разгледаме вариант на алгоритъма на Прим, който прави бърз избор на минимално ребро, което да бъде добавен към временното дърво. Това ще постигнем чрез приоритетна опашка Q . Ключът на всеки връх v в Q е минималното тегло на реброто, свързващо този връх с връх от временното дърво. Ако такова ребро няма, ключът е ∞ . В Q поначало се слагат всички върхове на графа с ключове ∞ , като само s има ключ 0 . След това на всяка итерация Q съдържа точно върховете, които не са в дървото; тоест, Q съдържа точно

граничните и неизвестните върхове. Граничните върхове са тези с ключове-числа, а неизвестните върхове са тези с ключове ∞ . Ключовете пишем така: ако върхът е u , неговият ключ е $u.key$. Самото МПД се реализира с масив на предшествията.

Prim(G: неориентиран свързан граф, w: теглова функция, s: стартов връх):

```

@1. for  $u \in V$  do
@2.    $u.key \leftarrow \infty$ 
@3.    $\pi[u] \leftarrow Nil$ 
@4. end for
@5.  $s.key \leftarrow 0$ 
@6.  $Q \leftarrow min\_priority\_queue(V)$ 
@7. while  $Q$  is not empty do
@8.    $x \leftarrow Extract - Min(Q)$ 
@9.   for  $y \in adj[x]$  do
@10.    if  $y \in Q$  and  $w((x, y)) < y.key$  then
@11.       $y.key \leftarrow w((x, y))$ 
@12.       $\pi[y] \leftarrow x$ 
@13.    end if
@14.  end for
@15. end while
@16. return  $\pi[1..n]$ 

```

Доказателство за while-цикъла: При всяко достигане на @7 имаме, че:

1. $Q \neq \emptyset \rightarrow (\exists u \in Q : u.key < \infty)$
2. $\forall u \in Q \setminus \{s\}$: ако $u.key < \infty$, то:
 - 2.1 $\pi[u] \neq Nil$
 - 2.2 $\pi[u] \notin Q$
 - 2.3 $u.key$ е минималното тегло на ребро, инцидентно с u и прекосяващо $\{V \setminus Q, Q\}$
3. $\forall u \in Q$, ако $u.key = \infty$, то $\exists v \in Q : v.key < \infty$ и има път между u и v , чиито върхове са само от Q .

Доказателство: База: Разглеждаме първото достигане на @7. Тогава $Q = V$, така че $Q \neq \emptyset$. Имаме, че $s.key = 0 < \infty$ следователно 1. е вярно. Тъй като s е единствения връх, за който $s.key < \infty$, то $\{u \mid u \in Q \setminus \{s\} \ \& \ u.key < \infty\} = \emptyset$, откъдето импликацията на 2. е истина.

3. е вярно, понеже $V = Q, s.key < \infty, \forall u \in V \setminus \{s\} : u.key = \infty$ и G е свързан.

Поддръжка: Да допуснем, че твърдението е вярно за някое достигане на @7, което не е последното. Нека моментът на това достигане е t , а следващото достигане да е в момент t' . Нека x е върхът изваден от опашката след момента t и преди t' .

Разглеждаме 1. Ако $Q = \emptyset$ след изваждането на x от Q (@8), то импликацията на 1. е вярна.

Нека $Q \neq \emptyset$ след изваждането на x от Q . Ако в момента $t \exists z \in Q : z \neq x \ \& \ z.key < \infty$ доказателството е готово.

Да допуснем, че няма такова z , т.е. $\forall x \in Q : z \neq x \rightarrow z.key = \infty$. Но в индуктивното предположение 3. ни дава, че x има поне един съсед z' в Q . На @11 имаме, че $z'.key = w((x, z'))$, поради което в момент $t' \exists u' \in Q : u'.key < \infty$, т.е. 1. е вярно.

Разглеждаме 2. Нека в момента $t, Q_f = \{v \in Q \mid v.key < \infty\}, Q_\infty = \{v \in Q \mid v.key = \infty\}$. Ще разгледаме три множества: $Q_f \setminus N(x), Q_f \cap N(x), Q_\infty \cap N(x)$, където $N(x)$ е множеството от съседни върхове на x . Върховете от $Q_f \setminus N(x)$ не биват засегнати от изваждането на x от Q и за тях 2.1, 2.2 и 2.3 са изпълнени в момента t' .

Разглеждаме $v \in Q_f \cap N(x)$. Ако в момента $t, w((x, v)) \geq v.key$, то редове @11-12 не се изпълняват и в момента t' 2.1, 2.2 и 2.3 остават в сила за v от И.Х.

Нека в момента $t \ w((x, v)) < v.key$. Тогава $\pi[v] = x \neq Nil, \pi[v] = x \notin Q$. Оттук 2.1 и 2.2 са в сила. На @11 $v.key = w((x, v))$ и очевидно (x, v) е най-лекото ребро, прекосяващо ребро, свързано с v , така че и 2.3 е в сила.

Разглеждаме $v \in Q_\infty \cap N(x)$. Когато y стане v (@9), @10 връща истина, понеже $v \in Q$ и $w((x, v)) < \infty$. Тогава $\pi[v] = x \neq Nil, \pi[v] = x \notin Q$. Оттук 2.1 и 2.2 са в сила. На @11 $v.key = w((x, v))$ и очевидно (x, v) е най-лекото (и единствено) ребро, прекосяващо ребро, свързано с v , така че и 2.3 е в сила. Разглеждаме 3.

След изваждането на x от Q , за всеки $v \in Q_\infty \setminus N(x)$ твърдението остава в сила заради свързаността на графа. $\square$

Коректността на алгоритъма на Прим: Масивът $\pi[1..n]$, който Prim връща, реализира МПД на G .

Доказателство: Нека във всеки момент от работата на алгоритъма G' е подграфът на G , чиито върхове на са в Q . Твърдим, че съществува МПД T на G , за което при всяко достигане на @7

$$\{(u, \pi[u]) \mid u \in V \setminus (\{s\} \cup Q)\} = E(T) \cap E(G')$$

База: Разглеждаме първото достигане на @7. В този момент $\{s\} \cup Q = V$, така че $V \setminus (\{s\} \cup Q) = \emptyset$ и множеството на лявата страна е празно. Със сигурност такова МПД съществува.

Поддръжка: Да допуснем, че инвариантът е в сила в момент t , в който изпълнението на @7 не е последното - Q не е празно. Съгласно миналото твърдение 1. имаме, че $\exists u \in Q : u.key < \infty$

Твърди се, че в момента t за всеки връх $v \in Q$, такъв че $v.key$ е минимален, е вярно че реброто $(v, \pi[v])$ е леко ребро прекосяващо срез $\{V \setminus Q, Q\}$. Но това следва директно от миналото твърдение 2.3. Съгласно МПД теоремата, в момента t за всеки връх $v \in Q$, т.че v е минимален, е вярно, че реброто $(v, \pi[v])$ е сигурно за множеството $E(T) \cap E(G')$, защото това множество е съобразено със срез $\{V \setminus Q, Q\}$.

На @8 върхът, който е изваден от опашката и съхранен в x , е в момента t в Q и е с минимална key стойност. Отгук, след изваждането на този връх от Q множеството $\{(u, \pi[u]) \mid u \in V \setminus (\{s\} \cup Q)\}$ продължава да е равно на $E(T) \cap E(G')$.

Терминация: При последното достигане на @7, опашката Q е празна. Тогава $E(G') = E(G)$, така че $E(T) \cap E(G') = E(T)$, освен това $V \setminus (\{s\} \cup Q) = V \setminus \{s\}$ и заключаваме, че $\{(u, \pi[u]) \mid u \in V \setminus \{s\}\} = E(T)$. С други думи, Prim е построил МПД чрез $p[1..n]$. $\square$

Сложност по време: Допускаме, че Q е реализирана чрез двоична пирамида. В израза за сложността няма множител n . Вместо да броим колко пъти се “извърта” while-а, ще разсъждаваме колко пъти се изпълнява for-цикълът на редове 9–12 за цялото изпълнение на алгоритъма. @9 се изпълнява $\Theta(|V| + |E|)$ пъти, защото това е просто минаване през списъците на съседство. Само че тялото на този for се изпълнява в най-лошия случай във време $\Theta(\lg |V|)$ заради имплицитното викане на Decrease-Key на ред 11. Отгук имаме оценка за сложността $\Theta((n + m) \lg n)$, което при свързан граф е $\Theta(m \lg n)$.

Сложност по време при различни структури от данни: Ако Q бъде реализирана не чрез двоична пирамида, а чрез пирамида на Fibonacci може сложността да се подобри, поне в асимптотичния смисъл. При пирамидите на Fibonacci, функцията Decrease-Key работи в амортизирано време $\Theta(1)$. Функцията Extract-Min остава със сложност $\Theta(\lg |V|)$ в най-лошия случай, поради което асимптотичната сложност става $\Theta(|E| + |V| \lg |V|)$. Събираемостта $|V| \lg |V|$ отчита работата на всички Extract-Min по време на изпълнението на алгоритъма. Пирамида на Fibonacci обаче е много сложна структура, ползването на която води до големи скрити мултипликативни константи (скрити в Θ -нотацията). Трябва данните наистина да са големи, за да има осезаема практическа полза от ползването на пирамиди на Fibonacci вместо двоични пирамиди в алгоритъма на Prim.

7.4 Алгоритъм на Крускал: псевдокод и доказателство за коректност. Изследване на сложността по време при използване на различни структури от данни. ОСТАВЯМ ЗА СЛЕД КОНСУЛТАЦИЯТА.

Алгоритъмът на Kruskal е ефикасен алгоритъм за намиране на МПД, който е основан на съвсем различна идея от тази на алгоритъма на Prim. Най-общо казано, сортираме ребрата по тегло, образуваме гора от тривиални дървета, по едно за всеки връх на графа, и след това разглеждаме ребрата в ненамаляващия ред на теглата, като за всяко ребро, ако краищата му са от различни дървета на гората, сливаме тези (две) дървета чрез това ребро в едно дърво, а ако краищата му са от едно и също дърво на гората, прескачаме това ребро, и правим това, докато гората има повече от едно дърво.

8 Най-къси пътища в теглови графи

ISS(G : ориентиран тегловен граф, s : връх в G)

```
@1. for  $v \in V(G)$  do
@2.    $v.d \leftarrow \infty$ 
@3.    $v.\pi \leftarrow Nil$ 
@4. end for
@5.  $s.d \leftarrow 0$ 
```

RELAX($(x, y) \in E(G)$)

```
@1. if  $y.d > x.d + w((x, y))$  then
@2.    $y.d \leftarrow x.d + w((x, y))$ 
@3.    $y.\pi \leftarrow x$ 
@4. end if
```

Лема: Неравенство на триъгълника:

За всяко ребро $(x, y) \in E(G)$ е изпълнено

$$\delta(s, y) \leq \delta(s, x) + w((x, y))$$

Доказателство. Доказателството е с разглеждане на случаи и подслучаи.

- **Няма $s \rightsquigarrow x$.** Тогава $\delta(s, x) = \infty$, така че дясната страна на неравенството е ∞ .
 - Няма $s \rightsquigarrow y$, така че и лявата страна на неравенството е ∞ . Неравенството е в сила.
 - Съществува $s \rightsquigarrow y$. Тогава лявата страна на неравенството е или число, или $-\infty$. Неравенството е в сила.
- **Съществува $s \rightsquigarrow x$.**
 - Съществува поне един $s \rightsquigarrow x$, който съдържа отрицателен цикъл. Тогава $\delta(s, x) = -\infty$. Но в този случай очевидно съществува $s \rightsquigarrow y$, който съдържа отрицателен цикъл, така че и $\delta(s, y) = -\infty$. Тогава неравенството е в сила.
 - Не съществува нито един $s \rightsquigarrow x$, който съдържа отрицателен цикъл. Тогава е изпълнено $-\infty < \delta(s, x) < \infty$. Нека p е най-къс път $s \rightsquigarrow x$. Тогава $w(p) = \delta(s, x)$.

□

Лема: Горна граница и точна горна граница: Нека G е инициализиран с ISS(G, s). Тогава, след произволна редица от релаксации на ребрата на G , е изпълнено

$$\forall v \in V(G) \quad v.d \geq \delta(s, v)$$

Нещо повече, при достигане на равенство за даден връх, то не се променя за този връх след произволна редица от последващи релаксации на ребрата на G .

Доказателство. Доказателството е по индукция по броя на релаксациите. Забележете, че “произволна редица от релаксации на ребрата на G ” означава буквално това, което е написано: релаксациите се вършат върху произволни ребра, в произволен ред; не се иска тези релаксации да се вършат с цел намиране на най-къси пътища!

База: За нула релаксации, т.е., d -стойностите са както са генерирани от ISS. Разглеждаме произволен $v \in V(G)$.

- Нека $v = s$. В сила е $s.d = 0$ заради ISS. От друга страна, или $\delta(s, s) = 0$, ако s не е върху отрицателен цикъл, или $\delta(s, s) = -\infty$, ако s е върху отрицателен цикъл. И в двата случая е вярно, че $s.d \geq \delta(s, s)$.
- Ако $v \neq s$, то $v.d = \infty$ заради ISS. От друга страна, $\delta(s, v) \in \mathbb{R} \cup \{-\infty, \infty\}$. Във всеки случай неравенството е в сила.

Индукционна стъпка: Да допуснем, че неравенството е в сила след някаква произволна серия от релаксации. Разглеждаме RELAX((u, v)), за някое $(u, v) \in E(G)$, веднага след тази серия. Единствената d -стойност, която може да се промени от RELAX((u, v)), е $v.d$. Ако тя се промени, новото $v.d$ е $u.d + w((u, v))$.

И така, след промяната е в сила

$$\begin{aligned} v.d &= u.d + w((u, v)) \quad (\text{защото RELAX работи така}) \\ &\geq \delta(s, u) + w((u, v)) \quad (\text{от индуктивното предположение за } u.d) \\ &\geq \delta(s, v) \quad (\text{от Лема 45}) \end{aligned}$$

Доказахме, че неравенството е в сила след RELAX((u, v)).

Запазване на равенството: За да се убедим, че ако веднъж се постигне равенство $v.d = \delta(s, v)$, то остава в сила във всеки момент след това, да съобразим, че

- няма как да се получи $v.d < \delta(s, v)$ заради вече доказанния факт, и
- няма как да се получи $v.d > \delta(s, v)$ след $v.d = \delta(s, v)$, защото RELAX никога не увеличава d -стойността на края на релаксираното ребро.

□

8.1 Алгоритъм на Дийкстра: вариант на задачата

Ще разгледаме два варианта на алгоритъма на Dijkstra: базов и изтънчен. Базовият вариант намира следващия връх x с последователно търсене, а изтънченият вариант ползва АТД приоритетна опашка, от която бързо изважда следващия връх x . Допускаме без ограничение на общността, че целият граф G е достижим от s .

```
@1. Dijkstra_slow( $G$ : ориентиран тегловен граф,  $w$ : тегл. ф-я върху  $G$ ,  $s$ : стартов връх)
@2. ISS( $G, s$ )
@3.  $D \leftarrow \emptyset$ 
@4. while  $V \setminus D \neq \emptyset$  do
@5.   избери  $x \in V \setminus D$ , такъв че  $x.d$  е минимална
@6.    $D \leftarrow D \cup \{x\}$ 
@7.   for  $y \in \text{adj}[x]$  do
@8.     RELAX( $(x, y)$ )
@9.   end for
@10. end while
```

```
@1. Dijkstra( $G$ : ориентиран тегловен граф,  $w$ : тегл. ф-я върху  $G$ ,  $s$ : стартов връх)
@2. ISS( $G, s$ )
@3.  $D \leftarrow \emptyset$ 
@4. създай празна приоритетна min опашка  $Q$ 
@5.  $Q \leftarrow V$ 
@6. while not isempty( $Q$ ) do
@7.    $x \leftarrow \text{EXTRACT-MIN}(Q)$ 
@8.    $D \leftarrow D \cup \{x\}$ 
@9.   for  $y \in \text{adj}[x]$  do
@10.    RELAX( $(x, y)$ )
@11.  end for
@12. end while
```

8.1.1 Доказателство за коректност

Преди да докажем коректността на алгоритъма на Dijkstra, ще формулираме няколко важни лема.

Твърдение: Нека v е връх, такъв че няма $s \rightsquigarrow v$. Тогава, след произволна редица от RELAX върху ребрата на графа е вярно, че $v.d = \infty$.

Доказателство: Съгласно предното твърдение, $v.d \geq \delta(s, v)$. Но $\delta(s, v) = \infty$ (по дефиниция). □

Твърдение: Нека $v \neq s$ и p е най-къс път $s \rightsquigarrow v$. Нека $u \neq v$ е предпоследният връх в p . Ако $u.d = \delta(s, u)$ в някакъв момент преди релаксирането на реброто (u, v) , то $v.d = \delta(s, v)$ във всеки момент след това релаксиране.

Доказателство: Нека q е подпътят на p от s до u . Тогава q е най-къс път от s до u , така че $w(q) = \delta(s, u)$. След релаксирането на (u, v) е в сила

$$v.d \leq u.d + w((u, v)) = \delta(s, u) + w((u, v)) = \delta(s, v).$$

От първото твърдение имаме $v.d \geq \delta(s, v)$, следователно $v.d = \delta(s, v)$. Това равенство се запазва. $\square$

Сега преминаваме към основната теорема за коректност.

Твърдение: Теорема 101 (Коректност на алгоритъма на Dijkstra): За всеки ориентиран граф G , за всяка тегловна функция w с неотрицателни тегла, за всеки избор на стартов връх s е вярно, че след термилирането на $\text{DIJKSTRA}(G, w, s)$:

$$\forall v \in V(G) \quad v.d = \delta(s, v).$$

Доказателство: Ще докажем, че за всеки връх $v \in V(G)$, в момента, в който v влиза в D (множеството от върхове, чиито най-къси разстояния вече са фиксирани), е вярно, че $v.d = \delta(s, v)$. Това влече верността на теоремата заради Лема 46.

Да допуснем противното: $\exists v \in V(G)$ такъв, че $v.d \neq \delta(s, v)$ в момента, в който v влиза в D . Нека v е **първият** връх, който влиза в D с d -стойност, различна от разстоянието от s до него. От Лема 46 следва, че $v.d < \delta(s, v)$ е невъзможно. Следователно допускането ни става:

$$\exists v \in V(G) \quad v.d > \delta(s, v).$$

Очевидно $v \neq s$, защото $s.d = 0$ и при положителни тегла $\delta(s, s) = 0$.

Разглеждаме итерацията, по време на която v влиза в D . Нека t е моментът точно преди v да влезе в D . В момента t е вярно, че $v.d > \delta(s, v)$; ако допуснем, че $v.d = \delta(s, v)$ в момента t , то $v.d = \delta(s, v)$ във всеки момент след t до края на алгоритъма (Лема 46).

Тъй като $v \neq s$, то $D \neq \emptyset$ в момента t . По допускане съществува път от s до v . Разглеждаме най-къс $s \rightsquigarrow v$ път p . Понеже теглата са положителни, такъв съществува.

В момента t първият връх на p (който е s) е в D , а последният връх (v) не е в D . Дефинираме u като най-близкия до s връх в p , който в момента t **не е** в D . Нека върхът преди u в p се казва α (възможно е $\alpha = s$). Забележете, че в момента t всички върхове на p от s до α включително са в D , а u е първият връх в p (посока от s към v), който не е в D .

Ще докажем, че $u.d = \delta(s, u)$ в момента t . Знаем, че в момента t връх α е в D . Той е бил сложен в D в някакъв по-ранен момент $t' < t$. Тъй като v е **първият** грешен връх, в момента t' е изпълнено $\alpha.d = \delta(s, \alpha)$. Съгласно Лема 46, във всеки момент след t' продължава да е вярно, че $\alpha.d = \delta(s, \alpha)$. На итерацията, когато α е бил сложен в D , реброто (α, u) е било релаксирано. Прилагаме Лема 48 към момента на релаксиране и заключаваме, че

$$u.d = \delta(s, u) \quad (11.6)$$

е изпълнено във всеки момент след това релаксиране, включително в момента t . Оттук $u \neq v$, защото за v знаем, че $v.d > \delta(s, v)$ в момента t .

Съгласно Теорема 100, $\delta(s, u) < \delta(s, v)$, тъй като теглата са положителни и върховете s, u, v са наредени в този ред върху най-къс път от s до v . По допускане $\delta(s, v) < v.d$, така че

$$\delta(s, u) < \delta(s, v) < v.d \quad (11.7)$$

От (11.6) и (11.7) получаваме в момента t :

$$u.d = \delta(s, u) < \delta(s, v) < v.d \quad (11.8)$$

Ключовото наблюдение е, че и u , и v са извън D в момента t . Причината алгоритъмът да избере v , а не u , за следващ връх в D е, че

$$v.d \leq u.d \quad (11.9)$$

(тъй като v е изваден от опашката с минимален ключ).

От (11.8) и (11.9) получаваме в момента t :

$$v.d \leq u.d = \delta(s, u) < \delta(s, v) < v.d \quad (11.10)$$

Коего е невъзможно. Следователно първоначалното допускане е грешно и за всеки връх v е вярно, че $v.d = \delta(s, v)$ в момента на влизането му в D . Това доказва теоремата.

8.1.2 Сложност при използване на различни структури от данни

Сложността по време е същата, в асимптотичния смисъл, като на съответните имплементации на алгоритъма на Prim за минимално покриващо дърво.

- Базовата имплементация (с линейно търсене на минимума) има сложност $\Theta(n^2)$ в най-лошия случай.
- Изтънчената имплементация (с приоритетна опашка, реализирана като двоична пирамида) има сложност $\Theta((n + m) \log n)$ в най-лошия случай.

Ако не всички върхове са достижими от s , в базовия вариант може да модифицираме условието на цикъла така, че да спрем, когато всички останали върхове имат $d = \infty$. В изтънчения вариант аналогично се проверява дали Q съдържа само безкрайности. Така върховете, недостижими от s , остават с $d = \infty$.

8.2 Алгоритъм за намиране на най-къси пътища в ДАГ: вариант на задачата

Ако G е тегловен даг, може да използваме по-ефикасен алгоритъм от алгоритъма на Dijkstra за задачата SSSP (Single-Source Shortest Paths).

```

@1. DAG_SSSP( $G$ : даг,  $w$ : тегловна функция върху  $G$ ,  $s$ : стартов връх)
@2.  $T \leftarrow \text{TOPOSORT}(G)$ 
@3.  $\text{ISS}(G, s)$ 
@4. for  $x \in V(G)$  в топологичната наредба do
@5.   for  $y \in \text{adj}[x]$  do
@6.      $\text{RELAX}((x, y))$ 
@7.   end for
@8. end for

```

8.2.1 Доказателство за коректност

Ще покажем коректността на алгоритъма с две доказателства: първото е “традиционното” доказателство (по индукция по топологичната наредба), а второто е с инвариант на цикъла.

Твърдение: Теорема 103 (Коректност на DAG_SSSP): За всеки даг G , за всяка тегловна функция w над него, за всеки избор на стартов връх s е вярно, че след термилирането на $\text{DAG_SSSP}(G, w, s)$:

$$\forall v \in V(G) \quad v.d = \delta(s, v).$$

Доказателство: [Първо доказателство] Нека T е резултатът от топологичното сортиране.

Първо, (11.12) е в сила за върховете вляво от s в T : от една страна, за всеки връх v вляво от s е вярно, че $\delta(s, v) = \infty$, понеже няма път $s \rightsquigarrow v$ в дага; от друга страна, във всеки момент от работата на алгоритъма е вярно, че $v.d = \infty$ (Лема 47).

Сега, (11.12) е вярно за s . Наистина, $\delta(s, s) = 0$ (при липса на отрицателни цикли), а $s.d$ става 0 след изпълнението на ISS и остава така до края (Лема 46).

Накрая разглеждаме произволен връх v вдясно от s в T . Важно наблюдение: за всеки $p \in \text{Paths}(s, v)$, вътрешните върхове на p са строго между s и v в T . Нека

$$U = \{u \in V(G) \mid \exists p \in \text{Paths}(s, v) \text{ и } u \text{ е предпоследният връх на } p\}.$$

Забележете, че е възможно $s \in U$ и че v е достижим от s т.с.т.к. $U \neq \emptyset$.

Когато x взема стойностите от $\{s, \dots, v'\}$ (където v' е върхът непосредствено вляво от v в T), извикванията на RELAX реализират

$$v.d = \min_{u \in U} \{u.d + w((u, v))\} \quad (\text{от алгоритмични съображения}) \quad (11.13)$$

От граф-теоретични съображения пък имаме

$$\delta(s, v) = \min_{u \in U} \{\delta(s, u) + w((u, v))\}. \quad (11.14)$$

При допускането, че $\forall u \in U : u.d = \delta(s, u)$, от (11.13) и (11.14) заключаваме, че $v.d = \delta(s, v)$ в момента, в който x стане v . Това остава вярно дори когато $U = \emptyset$ (т.е. v не е достижим от s): тогава $v.d = \infty$ и $\delta(s, v) = \infty$. $\square$

Доказателство: [Второ доказателство (с инвариант)] По отношение на цикъла (редове 3–5), за всеки $u \in V(G) \setminus \{s\}$ нека $P(u)$ означава множеството от пътищата от s до u , чийто предпоследен връх е вляво от текущия x (в топологичната наредба). Следното твърдение е инвариант:

Инвариант: При всяко достигане на реда за x (преди вътрешния цикъл), за всеки $v \in V(G) \setminus \{s\}$ е вярно, че $v.d$ съдържа дължината на най-къс път от $P(v)$.

База: При първото достигане, $P(v) = \emptyset$ за всяко v (няма върхове вляво от първия x). Тогава $v.d = \infty$ (от ISS), което е дължината на най-къс път в празното множество.

Поддръжка: Допускаме, че инвариантът е верен за текущо x . Разглеждаме два случая:

1. $P(x) = \emptyset$ (т.е. x е недостижим от s). Тогава $x.d = \infty$ и $\text{RELAX}((x, y))$ не променя никое $y.d$. След тялото на цикъла, за всеки $y \in \text{adj}[x]$, $y.d$ съдържа дължината на най-къс път с предпоследен връх вляво от, или съвпадащ с, x . За всички други върхове d -стойностите не се променят. Когато x премине към следващия връх в наредбата, инвариантът се запазва.

2. $P(x) \neq \emptyset$ (т.е. x е достижим от s). Преди тялото, $y.d$ вече съдържа дължината на най-къс път с предпоследен връх вляво от x . По време на тялото, RELAX прави

$$y.d \leftarrow \min(y.d, x.d + w(x, y)),$$

като $x.d + w(x, y)$ е дължината на най-къс път с предпоследен връх точно x . Така след тялото, $y.d$ съдържа дължината на най-къс път с предпоследен връх вляво от, или съвпадащ с, x . Отново, при преминаване към следващия x инвариантът се запазва.

Терминация: При последното достигане, x е най-десният връх в топологичната наредба. Тогава за всеки v , $P(v)$ е всички пътища от s до v (тъй като всеки предпоследен връх е вляво от най-десния). Инвариантът твърди, че $v.d$ е дължината на най-къс път от s до v , т.е. $v.d = \delta(s, v)$. $\square$

8.2.2 Сложност при използване на различни структури от данни

Сложността по време е $\Theta(n + m)$, защото:

- топологичното сортиране се изпълнява за $\Theta(n + m)$;
- вложеният **for**-цикъл обхожда всяко ребро точно веднъж, като всяка операция RELAX е с константна сложност.

Тази сложност е асимптотично оптимална.

8.2.3 Относителни предимства пред алгоритъма на Дийкстра

Предимствата на DAG_SSSP пред алгоритъма на Dijkstra върху даг са следните:

- **По-бърз:** $\Theta(n + m)$ е по-добра от сложностите на Dijkstra ($\Theta(n^2)$ за базовия вариант или $\Theta((n + m) \log n)$ с двоична пирамида, дори и с Fibonacci heap).
- **Работи с отрицателни тегла:** В даговете няма цикли по дефиниция, така че няма опасност от отрицателен цикъл. Dijkstra не е приложим при отрицателни тегла; Bellman-Ford е драстично по-бавен.
- **Намиране на най-дълги пътища:** С една тривиална модификация (обръщане на неравенството в RELAX) алгоритъмът намира най-дълги пътища в даг. Dijkstra не може да бъде трансформиран по този начин, дори графът да е даг.

8.3 Алгоритъм на Белман-Форд: вариант на задачата

Този алгоритъм намира най-къси пътища в тегловни графи, в които може да има отрицателни тегла, стига да няма отрицателни цикли. Ако има поне един отрицателен цикъл, алгоритъмът установява това и връща съответна индикация FALSE; в такъв случай намерените дължини на най-къси пътища следва да се игнорират. Ако върне TRUE, то отрицателни цикли няма и намерените дължини на най-къси пътища са коректни.

Без ограничение на общността, приемаме че целият граф G е достижим от стартовия връх s .

8.3.1 Псевдокод

```
@1. Bellman-Ford( $G$ : ориентиран тегловен граф,  $w$ : тегловна функция върху  $G$ ,  $s$ : стартов връх)
@2. ISS( $G, s$ )
@3. for  $i \leftarrow 1$  to  $n - 1$  do
@4.   for  $(x, y) \in E(G)$  do
@5.     RELAX( $(x, y)$ )
@6.   end for
@7. end for
@8. for  $(x, y) \in E(G)$  do
@9.   if  $y.d > x.d + w((x, y))$  then
@10.    return FALSE
@11.  end if
@12. end for
@13. return TRUE
```

8.3.2 Доказателство за коректност

Ще докажем коректността на алгоритъма в два случая: когато няма отрицателни цикли и когато има поне един отрицателен цикъл.

Твърдение: Лема 49 (for-цикълът на Bellman-Ford): За всеки ориентиран граф G без отрицателни цикли, за всяка тегловна функция w над него, за всеки избор на стартов връх s , в края на изпълнението на for-цикъла (редове 2–4) на Bellman-Ford(G, w, s) е вярно, че:

$$\forall v \in V(G) \quad v.d = \delta(s, v).$$

Доказателство: Щом няма отрицателни цикли, “дърво на най-късите пътища с корен s ” е добре дефинирано. Нека T е такова дърво. Дълбочината на връх в ориентирано кореново дърво е ориентираното разстояние като брой ребра от корена до него. Нека T_k (за $k \in \{0, 1, \dots, n - 1\}$) е поддървото на T , индуцирано от върховете от нива $0, 1, \dots, k$.

Разглеждаме следния инвариант:

Инвариант: При всяко достигане на ред 2 (началото на for-цикъла), за всеки връх v от T_{i-1} е изпълнено $v.d = \delta(s, v)$.

База: При първото достигане на ред 2, $i = 1$, така че $i - 1 = 0$. T_0 съдържа само корена s . От ISS имаме $s.d = 0$, а $\delta(s, s) = 0$ (понеже няма отрицателни цикли).

Поддръжка: Допускаме, че инвариантът е в сила в даден момент, като ниво i не е празно. За всеки връх v от ниво $i - 1$ е изпълнено $v.d = \delta(s, v)$. Всеки връх t от ниво i има родител в ниво $i - 1$. При изпълнението на вътрешния foreach-цикъл (редове 3–4), реброто (v, t) бива релаксирано. От Лема 48 следва, че след релаксирането $t.d = \delta(s, t)$ и това равенство се запазва. Следователно, след завършване на вътрешния цикъл, за всеки връх v от T_i е изпълнено $v.d = \delta(s, v)$. След това i се инкрементира и инвариантът се запазва.

Терминация: Когато достигнем ред 2 с празно ниво i , имаме $T_{i-1} = T$ (цялото дърво). Тогава за всеки връх $v \in V(G)$ (понеже целият граф е достижим от s) е изпълнено $v.d = \delta(s, v)$. $\square$

Твърдение: Следствие 30: За всеки ориентиран граф G без отрицателни цикли, по време на изпълнението на foreach-цикъла (редове 5–7) условието на ред 6 винаги е лъжа.

Доказателство: От Лема 49, при първото достигане на ред 5, $\forall v \in V(G) : v.d = \delta(s, v)$. Тогава за всяко ребро (x, y) имаме $y.d \leq x.d + w((x, y))$ (от лемата за неравенство на триъгълника), следователно условието $y.d > x.d + w((x, y))$ е невъзможно. $\square$

Твърдение: Теорема 104 (Коректност при липса на отрицателни цикли): За всеки ориентиран граф G без отрицателни цикли, за всяка тегловна функция w над него, за всеки избор на стартов връх s , Bellman-Ford(G, w, s) връща TRUE и при това $\forall v \in V(G) : v.d = \delta(s, v)$.

Доказателство: Следва директно от Лема 49 и Следствие 30. $\square$

Твърдение: Лема 51: Нека G е ориентиран тегловен граф с поне един отрицателен цикъл. Тогава, по време на изпълнението на foreach-цикъла (редове 5–7), за поне едно ребро (x, y) е вярно, че $y.d > x.d + w((x, y))$.

Доказателство: Допускаме противното: след изпълнението на for-цикъла (редове 2–4) е в сила

$$\forall (x, y) \in E(G) : y.d \leq x.d + w((x, y)). \quad (11.16)$$

Нека $c = v_1, e_1, v_2, e_2, \dots, v_k, e_k, v_1$ е произволен отрицателен цикъл в G ($k \geq 1$). От (11.16) получаваме:

$$\begin{aligned} v_2.d &\leq v_1.d + w(e_1) \\ v_3.d &\leq v_2.d + w(e_2) \\ &\vdots \\ v_k.d &\leq v_{k-1}.d + w(e_{k-1}) \\ v_1.d &\leq v_k.d + w(e_k) \end{aligned}$$

Събираме тези неравенства:

$$\begin{aligned} v_1.d + v_2.d + \dots + v_k.d &\leq v_1.d + v_2.d + \dots + v_k.d + \sum_{j=1}^k w(e_j) \\ 0 &\leq \sum_{j=1}^k w(e_j) \end{aligned}$$

Това е невъзможно, защото c е отрицателен цикъл (сумата от теглата е отрицателна). Следователно допускането е грешно и съществува ребро (x, y) , за което $y.d > x.d + w((x, y))$. $\square$

Твърдение: Теорема 105 (Коректност при наличие на отрицателни цикли): Ако G има поне един отрицателен цикъл, изпълнението на Bellman-Ford(G, w, s) достига ред 7, където алгоритъмът връща FALSE.

Доказателство: Следва директно от Лема 51. $\square$

8.3.3 Сложност по време

Сложността по време на алгоритъма на Bellman-Ford е $\Theta(n \cdot m)$, което в най-лошия случай е $\Theta(n^3)$ (при пътен граф, където $m = \Theta(n^2)$).

- Външният for-цикъл се изпълнява $n - 1$ пъти.
- Вътрешният foreach-цикъл обхожда всички m ребра на всяка итерация.
- Проверяващият цикъл (редове 5–7) също обхожда всички m ребра.

Това прави Bellman-Ford значително по-бавен от алгоритъма на Dijkstra ($\Theta((n + m) \log n)$) и от алгоритъма за DAG ($\Theta(n + m)$), но той има предимството да работи с отрицателни тегла и да открива отрицателни цикли.

8.3.4 Установяване на наличие на цикли с отрицателно тегло

Алгоритъмът установява наличие на отрицателен цикъл чрез втората фаза (редове 5–7). Идеята е следната:

Ако в графа няма отрицателни цикли, след $n - 1$ релаксации на всички ребра, d -стойностите на всички върхове са фиксирани към истинските най-къси разстояния $\delta(s, v)$. Тогава за всяко ребро (x, y) е изпълнено $y.d \leq x.d + w((x, y))$ (неравенството на триъгълника).

Ако обаче съществува отрицателен цикъл, той може да бъде достигнат от s (понеже приемаме, че целият граф е достижим). Тогава, колкото и пъти да релаксираме ребрата, d -стойностите на върховете в цикъла могат да намаляват неограничено. След $n - 1$ итерации обаче, ако се опитаме да направим още една релаксация на някое ребро от цикъла, ще открием, че условието $y.d > x.d + w((x, y))$ е изпълнено. Това е индикация, че в графа съществува отрицателен цикъл.

С малка модификация, алгоритъмът може да върне и самия отрицателен цикъл (например чрез проследяване на π -стойностите), но в основния си вариант той само сигнализира за наличието му чрез връщане на FALSE.

8.4 Алгоритъм на Флойд-Уоршал: вариант на задачата

Алгоритъмът на Флойд-Уоршал решава задачата за всички двойки най-къси пътища (APSP — All-Pairs Shortest Paths) в тегловен ориентиран граф. Той работи коректно при липса на отрицателни цикли, но може да открива наличието им. Характерно за алгоритъма е, че използва динамично програмиране с рекурсивна декомпозиция, базирана на максималния номер на вътрешен връх в пътя.

8.4.1 Псевдокод

Нека върховете на графа са номерирани с $1, 2, \dots, n$. Нека W е матрица $n \times n$, където $W[i, j]$ е теглото на реброто (i, j) , ако такова съществува, или ∞ , ако няма ребро. Приемаме, че $W[i, i] = 0$ за всички i .

```
@1. Floyd-Warshall( $W$ : матрица  $n \times n$ , представляваща тегловен ориентиран граф)
@2.  $D^{(0)} \leftarrow W$ 
@3. for  $k \leftarrow 1$  to  $n$  do
@4.   for  $i \leftarrow 1$  to  $n$  do
@5.     for  $j \leftarrow 1$  to  $n$  do
@6.        $D^{(k)}[i, j] \leftarrow \min(D^{(k-1)}[i, j], D^{(k-1)}[i, k] + D^{(k-1)}[k, j])$ 
@7.     end for
@8.   end for
@9. end for
@10. return  $D^{(n)}$ 
```

Ако освен дължините на пътищата искаме да възстановяваме и самите пътища, използваме матрица на предшествениците Π :

```
@1. Floyd-Warshall_Solution( $W$ : матрица  $n \times n$ )
@2. for  $i \leftarrow 1$  to  $n$  do
@3.   for  $j \leftarrow 1$  to  $n$  do
@4.      $D^{(0)}[i, j] \leftarrow W[i, j]$ 
@5.     if  $i = j$  or  $W[i, j] = \infty$  then
@6.        $\Pi^{(0)}[i, j] \leftarrow \text{Nil}$ 
@7.     else
@8.        $\Pi^{(0)}[i, j] \leftarrow i$ 
@9.     end if
@10.   end for
@11. end for
@12. for  $k \leftarrow 1$  to  $n$  do
@13.   for  $i \leftarrow 1$  to  $n$  do
@14.     for  $j \leftarrow 1$  to  $n$  do
@15.       if  $D^{(k-1)}[i, j] \leq D^{(k-1)}[i, k] + D^{(k-1)}[k, j]$  then
@16.          $D^{(k)}[i, j] \leftarrow D^{(k-1)}[i, j]$ 
@17.          $\Pi^{(k)}[i, j] \leftarrow \Pi^{(k-1)}[i, j]$ 
@18.       else
@19.          $D^{(k)}[i, j] \leftarrow D^{(k-1)}[i, k] + D^{(k-1)}[k, j]$ 
@20.          $\Pi^{(k)}[i, j] \leftarrow \Pi^{(k-1)}[k, j]$ 
@21.       end if
@22.     end for
@23.   end for
@24. end for
@25. return  $(D^{(n)}, \Pi^{(n)})$ 
```

Възстановяването на самия път от i до j става рекурсивно:

```
@1. Print-Path( $\Pi, i, j$ )
@2. if  $i = j$  then
@3.   print  $i$ 
@4. else
@5.   if  $\Pi[i, j] = \text{Nil}$  then
@6.     print "няма път"
@7.   else
@8.     Print-Path( $\Pi, i, \Pi[i, j]$ )
@9.     print  $j$ 
@10.   end if
@11. end if
```

8.4.2 Доказателство за коректност

Нека върховете на графа са номерирани с $1, 2, \dots, n$. За един път $p = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_l$ дефинираме множеството от вътрешните му върхове като $\{v_2, v_3, \dots, v_{l-1}\}$. Ако $l \leq 2$, множеството от вътрешни върхове е празно.

За всеки път $i \rightsquigarrow j$ дефинираме неговия **максимален вътрешен връх** като най-големия номер на връх, който се среща вътре в пътя. Ако пътят няма вътрешни върхове, приемаме, че максималният вътрешен връх е 0.

Нека $D^{(k)}[i, j]$ е дължината на най-къс път от i до j , чиито вътрешни върхове са подмножество на $\{1, 2, \dots, k\}$. Тогава:

- За $k = 0$: $D^{(0)}[i, j] = W[i, j]$, защото път без вътрешни върхове е или ребро (i, j) , или (ако $i = j$) пътят с дължина 0.
- За $k \geq 1$: разглеждаме най-къс път от i до j с вътрешни върхове от $\{1, \dots, k\}$.
 - Ако k **не** е вътрешен връх в този път, то пътят е и най-къс път с вътрешни върхове от $\{1, \dots, k-1\}$, следователно $D^{(k)}[i, j] = D^{(k-1)}[i, j]$.
 - Ако k **е** вътрешен връх в пътя, то пътят може да се раздели на два подпътя: $i \rightsquigarrow k$ и $k \rightsquigarrow j$, всеки от които има вътрешни върхове от $\{1, \dots, k-1\}$. Тогава $D^{(k)}[i, j] = D^{(k-1)}[i, k] + D^{(k-1)}[k, j]$.

Така получаваме рекурентната формула:

$$D^{(k)}[i, j] = \min(D^{(k-1)}[i, j], D^{(k-1)}[i, k] + D^{(k-1)}[k, j]).$$

Твърдение: Теорема (Коректност на Floyd-Warshall): След термилирането на алгоритъма, за всички $i, j \in \{1, \dots, n\}$ е изпълнено:

$$D^{(n)}[i, j] = \delta(i, j),$$

където $\delta(i, j)$ е дължината на най-къс път от i до j (или ∞ , ако такъв път не съществува).

Доказателство: Доказателството следва директно от рекурентната формула и индукция по k . Базата $k = 0$ е вярна по дефиниция. Индукционната стъпка запазва свойството, че $D^{(k)}[i, j]$ е дължината на най-къс път с вътрешни върхове от $\{1, \dots, k\}$. При $k = n$ множеството от разрешени вътрешни върхове е цялото $V(G)$, следователно $D^{(n)}[i, j] = \delta(i, j)$. $\square$

8.4.3 Сложност по време

Алгоритъмът на Флойд-Уоршал има три вложени цикъла, всеки от които се изпълнява n пъти. Следователно сложността по време е:

$$\Theta(n^3).$$

Това е независимо от плътността на графа — алгоритъмът винаги изпълнява точно n^3 операции за минимум и събиране. За сравнение:

- Алгоритъмът на Джонсън (който използва n пъти алгоритъма на Белман-Форд и след това n пъти алгоритъма на Дейкстра) има сложност $\Theta(nm \log n)$, което за гъсти графи ($m = \Theta(n^2)$) е $\Theta(n^3 \log n)$ — по-лошо.
- Алгоритъмът с повторно умножение на матрици има сложност $\Theta(n^3 \log n)$.

За гъсти графи алгоритъмът на Флойд-Уоршал е асимптотично оптимален, тъй като $\Omega(n^3)$ е долна граница за APSSP при гъсти графи (изходът сам по себе си има n^2 числа, но $\Theta(n^3)$ е необходима за обработка на всички тройки върхове).

8.4.4 Сложност по памет

Алгоритъмът има няколко варианта по отношение на паметта:

- **Наивна реализация:** Запазваме всички матрици $D^{(0)}, D^{(1)}, \dots, D^{(n)}$, което изисква $\Theta(n^3)$ памет.
- **Оптимизирана реализация:** Тъй като $D^{(k)}$ зависи само от $D^{(k-1)}$, можем да пазим само две матрици — текущата и предишната. Това намалява паметта до $\Theta(n^2)$.

- **Реализация на място (in-place):** Оказва се, че можем да използваме само една матрица D и да обновяваме директно в нея:

$$D[i, j] \leftarrow \min(D[i, j], D[i, k] + D[k, j]).$$

Това е коректно, защото:

- Ако $k \neq i$ и $k \neq j$, стойностите $D[i, k]$ и $D[k, j]$ не са били променяни в текущата итерация на k .
- Ако $k = i$, тогава $D[i, j] \leftarrow \min(D[i, j], D[i, i] + D[i, j]) = \min(D[i, j], 0 + D[i, j]) = D[i, j]$ — няма промяна.
- Ако $k = j$, аналогично $D[i, j] \leftarrow \min(D[i, j], D[i, j] + D[j, j]) = D[i, j]$ — няма промяна.

Така реализацията на място изисква само $\Theta(1)$ допълнителна памет върху входната матрица (но входната матрица бива унищожена).

Ето псевдокода на версията с памет $\Theta(1)$ допълнително:

```

@1. Floyd-Warshall_InPlace( $D$ : матрица  $n \times n$ )
@2. for  $k \leftarrow 1$  to  $n$  do
@3.   for  $i \leftarrow 1$  to  $n$  do
@4.     for  $j \leftarrow 1$  to  $n$  do
@5.        $D[i, j] \leftarrow \min(D[i, j], D[i, k] + D[k, j])$ 
@6.     end for
@7.   end for
@8. end for
@9. return  $D$ 

```

Тази версия работи коректно при липса на отрицателни цикли и използва само $\Theta(n^2)$ памет за входната матрица и $\Theta(1)$ допълнителна памет.

8.4.5 Откриване на отрицателни цикли

Алгоритъмът на Флойд-Уоршал може да открива наличие на отрицателен цикъл: след изпълнението на алгоритъма, ако за някое i е изгълнено $D[i, i] < 0$, то в графа съществува отрицателен цикъл, достижим от i (и i е част от него). Това е така, защото $D[i, i]$ е дължината на най-къс цикъл, минаващ през i ; ако тази дължина е отрицателна, значи имаме отрицателен цикъл.

9 Процедурно програмиране - основни конструкции.

9.1 Принципи на структурното програмиране.

Процедурното програмиране представлява изграждане на програми с отделни компоненти с ясно дефинирани задачи и граници. Такива структури са например функциите. Метод, който ни помага да получим таква структура е декомпозицията.

Основен принцип на структурното програмиране е принципът за модулност - разпределяне на функционалността на програмата на независими, взаимнозаменяеми модули, така че всеки от тях да съдържа всичко необходимо за изпълнението само на един аспект от желаната програма.

9.2 Управление на изчислителния процес. Основни управляващи конструкции - условни оператори, оператори за цикъл.

Изчислителният процес преминава през стъпки в определен ред, с ясно определени начална и крайна точка. В C++, изпълнението на програмата започва от `main` функцията. От своя страна тя може да извиква други функции, а нормалното изпълнение на програмата завършва с края на изпълнението на `main` функцията. Тя има следната дефиниция:

$$\text{int main} \left(\underbrace{\text{int argc}}_{\text{размер на масива argc}}, \underbrace{\text{char} * \text{argv}}_{\text{параметрите подадени на входа на програмата}} \right)$$

C++ предоставя няколко оператора за контрол на потока на изпълнение, които позволяват промяната на пътя на изпълнение на програмата. Такива оператори са условните оператори и операторите за цикъл.

Условен оператор: Синтаксис:

```
if (condition_0) {
    code_0;
} else if (condition_i) {
    code_i;
} else {
    code_else;
}
```

Забележка: Можем да имаме произволен брой **else if** блокове, както и да нямаме **else** блок.

Операторът се изпълнява по следния начин:

- Ако `condition_0` се оценява булево като истина, то се изпълнява `code_0` и останалата част от условния оператор се прескача, иначе:
- Ако `condition_i` се оценява булево като истина, то се изпълнява `code_i` и останалата част от условния оператор се прескача, иначе се проверява `condition_i+1` ако има такова.
- Ако всички условия са се оценили булеви като лъжа, то се изпълнява `else`.

Тернарен оператор

```
result = (condition) ? expr_1 : expr_2;
```

Представява триместна операция. Пресмята се **condition**:

- При оценка **true** се пресмята **expr_1** и се връща резултатът.
- При оценка **false** се пресмята **expr_2** и се връща резултатът.

Оператор за многозначен избор switch:

```
switch(expression) {
    case value_i:
        expr_i;
    default:
        expr_default;
}
```

Служи за проверка на променлива или израз за равенство спрямо набор от различни стойности. Ако оценената стойност `expression` е равна на някоя от стойностите след `case`, то изпълнението започва от съответния за `case_i`: `expr_i` и продължава до настъпване на някое от следните условия за прекъсване:

- Достигнат е края на `switch` блока.
- Друг оператор за контрол на потока на изпълнение е срещнат (`break` или `return`).
- Нещо друго прекъсне нормалното изпълнение на програмата (ОС).

Ако се изпълни `expr_i` за някой `case_i` (непоследен) и в `expr_i` няма `break` или `return`, то изпълнението няма да терминира `case`-блока, а ще продължи с проверка за следващия `case_i+1`.

Ако не е намерена съответстваща стойност на `expression` и съществува `default` етикет, то се изпълнява `expr_default`.

Ползата на `switch-case` спрямо `if-else if-else` е, че зададеният израз се оценява само веднъж и се проверява за равенство всеки път.

Оператор for

```
for (initialization; condition; step) { body; }
```

Първо се изпълнява `initialization`, което се случва само веднъж, когато цикълът е инициализиран. Обикновено се използва за декларация и инициализация като създадените променливи имат област на действие само в цикъла.

Второ, за всяка итерация на цикъла се оценява `condition`. Ако резултатът е истина, тялото на цикъла се изпълнява. В противен случай изпълнението на цикъла приключва.

След всяко изпълнение на `body` се изпълнява `step` - израз, променящ променливите на цикъла, дефинирани в `initialization`. След това отново се изпълнява стъпка две.

Оператор while

```
while (condition) {body;}
```

При изпълнение на оператора `while`, първо се оценява стойността на `condition`. Ако оценката е истина, то се изпълнява `body` и след това се връща в началото на оператора, като процесът се повтаря. Ако оценката е лъжа, изпълнение на `while` приключва.

Операторът `do while` е сходен на `while`, но първо се изпълнява `body` и после се проверява условие.

9.3 Променливи - видове: локални променливи, глобални променливи; инициализация на променлива; оператор за присвояване.

Променливи: Променливата представлява именувана област в паметта. С нея се свързва:

- Име (идентификатор)
- Място в паметта (адрес)
- Тип
- Стойност

Областта на действие на променливата започва от мястото на нейното дефиниране до края на блока, в който е дефинирана.

Дефиниция на променлива:

```
int x;
```

Тук сме създали променлива от даден тип (`int` в случая).

Казваме, че сме инициализирали променливата, ако сме задали начална стойност за нея:

```
int x;  
x = 4;
```

Можем да извършим дефиниция и инициализация в една стъпка, използвайки следния синтаксис:

```
int x{5};  
int y(6);
```

Така директно се инициализира, докато като при използване на `=` първо се дефинира променливата и след това и се присвоява стойност. (сору инициализация)

Ако не инициализираме променлива, то нейната стойност е неопределена - това, което се е съдържало в адреса, който заема.

Оператор за присвояване:

```
int x = 0;  
int y = 1;  
x = 7*8;  
y = x;
```

Запазваме дадена стойност (оценката на някой израз, в случая $7*8$) в променлива с посочен идентификатор (в случая `x`). Като резултат от присвояването се връща стойността на променливата.

Лявата променлива от страна на равенството е място в паметта със стойност, която може да се променя (променлива), докато дясната страна на равенството е временна стойност, без специално място в паметта (константа, литерал, резултат от пресмятане).

Присвояването е дясноасоциативна операция. $a = b = c = 2$ се възприема като $a = (b = (c = 2))$.

Видове променливи спрямо областта на действие: Областта на действие на променлива определя къде в програмата тя може да бъде достъпвана.

Локалните променливи имат блокова област на действие, което означава, че са в област на действие от мястото на дефинирането им до края на блока, в който са дефинирани.

Глобалните променливи са променливи, които са дефинирани извън тялото на коя да е функция. Областта им на действие е от мястото на дефиниция до края на файла, в който са дефинирани.

9.4 Функции и процедури. Параметри - видове параметри. Подаване на параметри - по име и по стойност. Типове и проверка за съответствие на тип.

Функцията е самостоятелен фрагмент от програмата, представляващ редица от команди, които могат да се използват многократно и са предназначени за изпълнение на определена задача.

Дефиниране на функция

```
type function_name(parameters){  
    body;  
}
```

Тук `type` е типът на резултатът, който функцията връща. Ако функцията не връща нищо, то типът е `void`.

Формалните параметри на функцията могат да бъдат:

- `void` - функцията не приема параметри
- `type parameter_1, ..., type parameter_n`

Извикване на функция:

- `function_name(parameter_1, ..., parameter_n)`, като аргументите са вече-дефинирани.
- `function_name()`, ако няма параметри.

Типът на фактическите аргументи (параметри) трябва да се съпоставя с типа на формалните параметри.

Връщане на резултат:

- `return result;`
- `return;`

Типът на `result` трябва да се съпоставя с типа на резултата на функцията. След `return` работата на функцията прекратява незабавно.

При извикване на функция се заделя нова стекова рамка, в която се пазят фактическите параметри, адрес за връщане и локалните променливи на функцията. Тази стекова рамка се намира на върха на програмния стек.

Функцията може само да се декларира като не се зададе тяло на функция. Декларацията представлява "обещание" за дефиниция на функцията.

Предаване на параметри - по име и по стойност:

При предаването по стойност се пресмята стойността на фактическия параметр и в стековата рамка на функцията се създава копие на стойността. Таква всяка промяна на стойността остава локална за функцията. При завършване на функцията, предадената стойност и всички промени над нея изчезват.

Предаване с препратка/референция се използва, когато искаме промените в параметрите на функцията да се отразят върху фактическите параметри (аргументите, които подаваме). Също се използва, когато обектите са тежки за копиране.

Пример:

```
int add5(int x) {x += 5; return x;}
int add6(int& x) {x+= 6; return x;}

int a = 3;
cout << add5(a) << " " << a;
int b = 3;
cout << add6(b) << " " << b;
```

Изходът от програмта ще бъде: "8 3" и "9 9"

10 Процедурно програмиране - указатели, масиви и рекурсия.

10.1 Указатели и указателна аритметика.

Оператор за рефериране: Операторът & ни позволява да достъпим адрес на променлива. Той е унарн и връща адреса на операнда си.

```
int x{5};
cout << &x;
```

Вторият ред ще принтира адреса на променливата x в паметта.

Оператор за дерефериране: Операторът * е унарн, приема адрес и връща стойността, съхранявана в подадения адрес в паметта.

```
int x{5};
cout << *(&x);
```

Вторият ред ще принтира стойността, съхранявана в адреса на x , а именно самия x .

Указател: Указателят е обект, който съдържа като своя стойност адрес в паметта. Подобно на референциите, указателите се дефинират с *: `type* name (= expr);`

```
int x{5};
int *ptr{&x};
```

Указателят съхранява адреса на началото на дадена стойност в паметта, а типът на указателя дава информация какъв тип данни се намира на този адрес и съответно какъв е техният размер. Например `sizeof(char)` е 1 байт, затова `char*` указва, че на този адрес се намира стойност с размер 1 байт. Аналогично, `int*` указва, че на адреса се намира `int`, който обикновено е 4 байта.

Това е причината указателите да имат различни типове, въпреки че всички реално съхраняват числена стойност — адрес в паметта. Типът е необходим, за да знае компилаторът как да интерпретира данните и как да извършва операции като разименуване (`*p`) и аритметика с указатели (`p + 1`).

Стойностите от тип указател са с размер на машинна дума съответно 32 или 64 бита, в зависимост от архитектурата. Физическото представяне на указател е цяло число, указващо адреса на lvalue в паметта.

Ако не инициализираме променлива с тип указател, то тя ще съдържа случайна стойност - подобно на останалите променливи.

`nullptr` е специална стойност за указатели, които не сочат към никакъв адрес.

Операции с указатели: Операторът за рефериране връща като резултат указател. Операторът за дерефериране приема като аргумент указател.

Като операции имаме:

- Сравнение (`==`, `!=`, `<`, `>`, `<=`, `>=`)
- Извеждане (`«`)
- Не можем да въведем указател с оператора за въвеждане (`»`)

Указателна аритметика: Позволява чрез даден указател да достъпваме съседни клетки в паметта. За целта трябва да укажем колко клетки напред или назад в паметта да прескочим. Размерите на клетките зависят от типа, който указателя реферира. Ако имаме `type * p`:

- `p + i` прескача `i * sizeof(p)` байта напред
- `p - i` прескача `i * sizeof(p)` байта назад

Указатели и константи

Константен указател: Не се променя накъде сочи. `<type> * const.`

Пример:

```
int x, *p = &x;
int *const q = p;
*q = 5; // x = 5
q = p + 2; // error: cannot assign to variable 'q' with const-qualified type 'int *
const'
```

- Можем да променяме стойността, към която указателят сочи.
- Не можем да променяме самият указател, т.е. адресът му.

Указател към константа: Сочи към константа. `const <type> * ⇔ <type> const *`

Пример:

```
int x, *p = &x;
const int * q = &x;
q++; // q = q + 4
p = q; // error: assigning to 'int *' from 'const int *' discards qualifier
s
*q = 5;
```

- Можем да променим самият указател, т.е. адресът му.
- Не можем да променяме стойността, към която указателят сочи.

Забележка: Ако `x` е константа, то `&x` е указател към константа.

10.2 Едномерни и многомерни масиви. Основни операции с масиви - индексирание.

Масивът е съставен тип данни, представляващ крайна редица от елементи с един и същи тип, с фиксирана дължина. Позволява произволен достъп до всеки негов елемент по индекс - цяло число започващо от 0 до дължината на масива -1.

Пример:

- `bool b[10];` // масив с 10 елемента от тип `bool`. Съдържа произволни стойности, защото не е инициализиран.
- `double x[3] = 0.5, 0.1, 0.4;` // Последният елемент е стойност по подразбиране за `double`.
- `int a[] = 3 + 2, 2 * 4;` // Пропускайки дължината, компилаторът я определя от дължината на инициализиращия списък.
- `float f[1]2.3, 4.5;` // Директно инициализиране.

Физическо представяне: Елементите са разположени последователно един след друг в паметта. Името на масив е константен указател към първия му елемент. Следователно: `int * p = a;` Сочи към първия елемент на масива `a`.

Операции за работа с масиви:

- Достъп до елемент по индекс с оператор `[]`: `array[int]`

Важно за достъп до елемент е, че не е извършва никаква проверка дали индексът е валиден.

Многомерен масив: Масив, чиито елементи са едномерни масиви е двумерен масив. N-мерен масив е масив, чиито елементи са (N-1)-мерни масиви.

Синтаксис за дефиниране на N-мерен масив:

```
int arr[size_1][size_2]...[size_n];
```

Първата размерност може да бъде изпусната ако е даден инициализиращ списък.

Пример:

```
int a[2][3] = {{1,2,3}, {4, 5, 6}}
```

10.3 Сортиране и търсене в едномерен масив - основни алгоритми.

Selection sort: Разделя входния масив на две части - сортиран подписък от елементи, който се изгражда от ляво надясно и останалите елементи. Намира най-малкия елемент от несортираната част и го разменя с най-левия елемент от несортираната част.

```

void selectionSort(int *a, int n) {
    for (int i = 0; i < n - 1; ++i) {
        int j_min = i;
        for (int j = i + 1; j < n; ++j) {
            if (a[j] < a[j_min]){
                j_min = j;
            }
        }
        std::swap(&a[i], &a[j_min]);
    }
}

```

Insertion sort: На всяка итерация намира локацията на един елемент и го слага там. Повтаря докато няма повече елементи.

```

void insertion_sort(int *a, int n){
    int key;
    for (int i = 1; i < n; ++i) {
        key = a[i];
        int j = i - 1;
        while (j >= 0 & key < a[j]) {
            a[j+1] = a[j];
            --j;
        }
        a[j+1] = key;
    }
}

```

Merge sort: Разделя несортирания списък на два подсписъка. Повтаря това докато не получи едноелементни списъци, след което обединява списъците като ги сортира.

```

void merge(int* a, int l, int m, int r, int* tmp){
    int i = l, j = m+1, k = 0;
    while (i <= m && j <= r) {
        if (a[i] <= a[j]) {
            tmp[k++] = a[i++];
        } else {
            tmp[k++] = a[j++];
        }
    }
    while (i <= m) tmp[k++] = a[i++];
    while (j <= r) tmp[k++] = a[j++];
    memcpy(a + l, tmp, k*sizeof(int));
}

void mergeSort(int* a, int l, int r, int * tmp){
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(a, l, m, tmp);
        mergeSort(a, m+1, r, tmp);
        merge(a, l, m, r, tmp);
    }
}

```

Двоично търсене: търсене на индекс на елемент в сортиран масив.

```

int binary_search(int *a, int n, int x) {
    int l = 0, r = n - 1, m = (l+r)/2;
    while (l <= r) {
        m = (l + r)/2;
        if (a[m] == x) {
            return m;
        }

        if (a[m] < x){
            l = m + 1;
        }
    }
}

```



```

    } else {
        r = m-1;
    }
}
return -1;
}

```

10.4 Символни низове. Представяне в паметта. Основни операции със символни низове.

Символен низ: наричаме последователност от символи. В C++ се представя като масив от символи (char), в който след последния символ е записан терминиращият символ '\0' - първият символ в ASCII таблицата.

Пример:

- char word[] = {'H','e','l','l','o','\0'};
- char word[100] = "Hello"; Ще постави '\0' след 'o', но ще заделни памет за общо 100 символа.

Основни операции със символни низове:

- Вход:
 1. » : въвежда до разделител (интервал, табулация, нов ред)
 2. cin.getline(str, num) : въвежда до нов ред, но не повече от num-1 символа.
- Изход: cout « "Hello";
- Индексиране: Достъпване на i-тия символ в низа.

Вградени функции за низове:

- strlen(str): връща дължината на низ
- strcpy(dest, str): прехвърля всички символи от str в dest и връща dest
- strcmp(a, b): Сравнява лексикографски два низа
- strcat(a, b): Конкатенира два низа, като записва символите на b в края на низа a. Терминиращия символ на низа a се изтрива. Връща низа a. (Условие е низът a да има достатъчно място за всички символи на низ b).
- strchr(str, symb): Търси символ в низ. Намира адреса на първото срещане на symb в str, ако има такова. В противен случай връща nullptr.
- strstr(str, substr): Търси подниз в низ. Намира адреса на първото срещане на substr в str, ако има такова. В противен случай връща nullptr.

10.5 Рекурсия - пряка и косвена рекурсия, линейна и разклонена рекурсия.

Рекурсивна функция: Функция, която извиква себе си пряко или косвено. Функцията е пряко рекурсивна, ако в тялото си се извиква. Функцията е косвено рекурсивна, ако извиква друга функция, в чието изпълнение се извиква обратно първата: $F_1 \rightarrow F_2 \rightarrow \dots \rightarrow F_1$

Линейна рекурсия е такава, при която функция прави само едно рекурсивно извикване на себе си в кода.

Пример:

```

int factorial(int n){
    if (n == 0)
        return 1;
    else
        return n * factorial(n-1);
}

```

Разклонена рекурсия е такава, при която функцията извиква себе си два или повече пъти, за да реши един проблем. Пример: MergeSort.

Рекурсията е удобна за работа с рекурсивни структури, обхождане на графи, търсене с връщане назад, алгоритми от тип "разделяй и владей".

Недостатък на рекурсията е скритото използване на памет за стекови рамки.

11 Обектно-ориентирано програмиране. Основни принципи. Класове и обекти. Наследяване и капсулация.

11.1 Абстракция със структури от данни. Класове и обекти. Декларация на клас и декларация на обект. Основни видове конструктори. Управление на динамичната памет и ресурсите ("RAII"). Методи - декларация, предаване на параметри, връщане на резултат.

Абстракция със структури от данни. Класове и обекти: Абстракцията със структури от данни е основен принцип на ООП, според който представянето на данните е отделено от използването им.

ООП се фокусира над създаването на програмно-дефинирани типове данни, които съдържат свойства и поведение.

Класовете в обектно-ориентирания език представляват шаблони, по които са създават инстанции на класа. Инстанциите наричаме обекти. Всеки клас дефинира член-функции (методи) и член-данни (полета). Полетата са променливи, които са асоциирани с конкретен обект на класа (с изключение на `static`). Методите определят какво е поведението на обектите на класа по време на живота им в процеса.

Декларация на клас и декларация на обект:

Декларацията на клас се състои от име на класа и тяло, съдържащо декларация на член-данни (полета) и член-функции (методи).

Член-данните и член-функциите могат да бъдат декларираны с различни модификатори за достъп - `private`, `protected` и `public`.

- `Private` забранява външен достъп до данните и методите, декларираны в класа.
- `Protected` забранява външен достъп до данните и методите, декларираны в класа, с изключение на класовете-наследници.
- `Public` позволява външен достъп до данните и методите, декларираны в класа - член-данните и член-функциите на един клас са `private` по подразбиране. (На `struct` са `public` по подразбиране - единствената разлика между двете в `C++`).

След като един клас е дефиниран, може да се създават негови инстанции. Връз като между клас и обект в `C++` е като тази между тип и променлива, но обектът се състои от множество компоненти.

Създаването на обекти е свързано с инициализиране на паметта на обекта, задаване на начални стойности и инициализация - това се изпълнява от специален вид член-функции на класовете - **конструктори**.

Конструкторът има следните особености:

- Името му съвпада с името на класа.
- Конструкторът няма тип на връщане, дори `void`.
- Изпълнява се автоматично при създаване на обекти
- Конструкторът не може да бъде извикан като обикновена член-функция върху вече съществуващ обект.

Във всеки клас може да се дефинира един или няколко конструктора. Техните видове са:

- Обикновен конструктор с параметри.
- Конструктор с параметри по подразбиране.
- Системно генериран конструктор по подразбиране - Когато няма дефиниран конструктор.
- Конструктор за копиране - Когато искам да инициализираме един обект чрез друг. Приема параметър от тип `class_name const &`
- Конструктор за преобразуване на тип - Специални конструктори, които позволяват правилно да се конструира обект от клас А, при подаден обект от тип или клас В.
- Преместващ конструктор - Приема параметър от тип `class_name &&`, като цели да премести съдържанието на подадения обект в новосъздаваният се. След това обектът-параметър остава празен. Компиляторът системно генерира такъв конструктор по подразбиране.

Деструктор: Деструкторът е функция, която се извиква автоматично при унищожаване на обекта в края на неговата област на действие или при извикване на `delete`. Той е противоположното на конструктора и служи за освобождаване на използваните ресурси. Синтаксис:

```
class_name :: ~class_name() {}
```

Управление на динамичната памет и ресурсите ("RAII"):

Областта за динамична памет (heap) представлява набор от свободни блокове памет. Динамичната памет може да бъде заделяна и освобождавана по всяко време по време на изпълнение на програмата. Програмата може да заявява блокове памет с произволна големина, а операционната система се грижи за управлението ѝ.

Заделената динамична памет остава заета до нейното освобождаване с delete или до приключване на програмата. Отговорността за правилното управление на динамичната памет е на програмиста.

RAII (Resource Acquisition Is Initialization) е принцип, при който заделянето на ресурс се извършва при създаването (инициализацията) на обекта, а освобождаването му — при унищожаването на обекта чрез деструктора.

C++ реализира RAII чрез т.нар. „голяма четворка“: конструктор, деструктор, копиращ конструктор и оператор =. Тези функции могат да бъдат генерирани автоматично от компилатора, но когато класът управлява външни ресурси (например динамична памет), е необходимо те да бъдат дефинирани от програмиста.

Методи - декларация, предаване на параметри, връщане на резултат:

При извикване на метод, имаме достъп до полетата без да се указва обекта. Това се случва защото при извикване на метод се създава автоматично специален константен указател с име this.

```
class\_name * const this
```

Той сочи към обекта, за който е извикан метода. Компилаторът скито от нас превежда методите, така че да получават this като първи параметър и всяко поле на обекта в тялото се достъпва чрез this.

Съществува специален тип методи - константни методи, при които методът гарантира, че няма да се променя състоянието на обекта или да извиква не-константни методи. Синтаксис: [тип] [име][операция](параметри)[const] {тяло}

C++ позволява повечето вградени операции да бъдат предефинирани, така че да работят с обект от произволен клас. Не могат да бъдат предефинирани следните операции: ?, ::, sizeof, #, ##

Пример:

```
Pair operator*(Pair const& p) const{
    return Pair(x*p.x, y*p.y)
}
```

11.2 Неследяване. Производни и вложени класове. Достъп до наследените компоненти. Капсулация и скриване на информацията. Статични полета и методи.

Наследяване. Производни и вложени класове. Достъп до наследените компоненти:

Основна идея на наследяването е създаване на нови класове чрез използване на атрибути и поведение на съществуване на класове.

Класът, който наследяваме, се нарича родителски или базов. Класът, който наследя, се нарича наследник или подклас.

Синтаксис на наследяването:

```
class derived: public base{, ...} {body}
```

Видимостта може да има 3 стойности: private, protected и public. Чрез видимостта се определя достъпът до функции, които не са методи на класа. Private по подразбиране.

При наследяване наследникът получава от основния клас всички негови полета, методи и достъп до неговите public и protected компоненти.

Наследникът не получава от родителя си достъп до private компоненти, но ги съдържа.

Наследникът може да дефинира свои методи и полета, дори и те да са със същите имена като в родителя - това наричаме предефиниране на компоненти. При наследяване, наследникът съдържа указател към началото на базовия клас.

Когато се инстанцира обект от произволен клас се случват следните неща:

- Първо се конструира обект от всички базови класове, които се наследяват. Използва се конструктор по подразбиране ако не е специфициран.
- Заделя се памет за наследникът.
- Извиква се подходящият конструктор за наследника.
- Инициализират се променливите от инициализацията списък
- Изпълнява се тялото на конструктора.

Капсулации и скриване на информация: Принципът на капсулацията е, че разделя (абстрахира) описанието на типа данни от конкретната му реализация. Обектите скриват вътрешното си състояние - данните от които се състоят, връзките между тях и как те си взаимодействат. Само самите обекти могат да модифицират състоянието директно, по този начин се грижат за запазване на инвариантата на структурата от данни, представлява състоянието. Осъществява се чрез групиране на данни и функции в клас и с модификаторите за достъп `private`, `protected` и `public`.

Статични полета и методи: При създаване на инстанция, всеки обект получава свое копие на всички полета.

Статичните полета са споделени между всички обекти на дадения клас. Те не се асоциират с конкретен обект и съществуват дори и да няма инстанцирани обект от конкретния клас. Полетата на класа могат да бъдат направени статични, използвайки думата `static`.

Статичните обекти са на практика глобални променливи, които живеят в областта на класа. Можем да ги инициализираме, използвайки оператора `::` или при декларацията им, директно инициализиране с `{}`.

Пример:

```
class Foo{
    public:
        static const int sc_value{4};
        static int s_value;
};

int Foo::s_value{1};
```

Методите също могат да бъдат статични и тогава те не са асоциирани с конкретен обект и могат да бъдат извиквани директно, използвайки името на класа и оператора за достъп `::`.

```
class Foo{
    static int func(){};
};

int Foo::func();
```

Те могат да бъдат извиквани и от инстанции на класа. Тъй като те не са асоциирани с конкретен обект, нямат `this` указател. Те могат да достъпват други статични компоненти (полета и методи), но не и нормални такива.

12 Обектно-ориентирано програмиране. Подтипов и параметричен полиморфизъм. Множествено наследяване.

12.1 Виртуални функции и подтипов полиморфизъм. Динамично свързване. Абстрактни методи и класове. Масиви от обекти и указатели към обекти.

Полиморфизъм: свойството един елемент да има различно поведение в различни ситуации.

Подтипов полиморфизъм: Подтиповият полиморфизъм позволява обект от произведен клас да бъде използван чрез интерфейса на базовия клас. Реализира се чрез предефиниране (overriding) на виртуални функции.

Класът-дете дефинира функция със същото име, тип и параметри като класа родител, но дава различна имплементация.

Виртуални функции: Виртуалната функция е член-функция, декларирана с `virtual` в базов клас, която позволява предефиниране в производни класове и динамичен избор на изпълняваната функция по време на изпълнение чрез механизма на динамично свързване.

Пример:

```
class Base{
public:
    virtual void func() {
        cout << "Base" << endl;
    }
};

class Child: public Base{
public:
    void func() override {
        cout << "Child" << endl;
    }
};

Base *c = new Child;
c->func(); // prints "Child"
Base *b = new Base;
b->func(); // prints "Base"
```

Чрез `virtual` се постига подтипов полиморфизъм. Така указателя към `Base` има различно поведение в различните случаи.

Динамично свързване: Възможно е да се използват функции с еднакви имена, в това число и методи на класове – така нареченото предефиниране на функции. За да може компилаторът да избере кой метод или функция трябва да бъде извикана има два типа свързване:

При статичното свързване изборът на функция става по време на компилация, като пример за това са неvirtуални функции и `overload`-нати функции.

Вторият вид свързване е динамично. При него определянето на извикващия метод става по време на изпълнение и се използва при виртуални функции или указатели към базов клас. Извиква се методът на подклас, от който е обекта, към който сочи указателя.

```
Base *c = new Child;
c->func(); // calls the func method of Child
```

Начинът, по който това става в `C++` е чрез запазената дума `virtual`.

Абстрактни методи:

- Чисто виртуални.
- Няма имплементация в родителя.
- Разчитат, че ще бъдат имплементирани от наследник.

Абстрактни класове:

- Съдържат поне една чисто виртуална функция. Сигнатура: `virtual void foo() = 0;`

- Не може да имаме инстанция от техния тип. (Можем да имаме указател).
- Ако наследник не имплементира виртуалните функции, то също става абстрактен клас.
- В C++ интерфейс обикновено наричаме абстрактен клас, съдържащ само чисто виртуални функции и без състояние (без член-данни).

Масиви от обекти и указатели към обекти: Масивите са хомогенни контейнери, но чрез наследяване и полиморфизъм можем да създадем масив от указатели от тип родител, които да сочат към наследници. Така елементите на този масив ще имат полиморфно поведение.

12.2 Параметричен полиморфизъм. Шабини на функция и на клас.

Параметричен полиморфизъм: Възможността обект да има повече от една "форма" се постига чрез въвеждане на типов параметер. В C++ параметричният полиморфизъм се постига чрез шабини - правила за генериране на код. Те позволяват дефинирането на "общи" функции и класове, които работят с неопределени типове по общ унифициран начин.

Пример:

```
template <typename T> T myMax(T x, T y){
    return (x > y) ? x : y;
}
cout << myMax<int>(3, 7) << endl;
cout << myMax<double>(3.5, 7.5) << endl;
cout << myMax<char>('g', 'e') << endl;
```

Типовите параметри могат да участват в:

- Тялото на функцията.
- Типът на връщания резултат.
- Типовете на параметрите.

Шаблонът не се компилира. При всяко използване с различни типове се генерира нова функция, която се компилира. Такава функция се нарича шаблонна.

Шаблоните могат да се ползват и за класове. Типовите параметри могат да използват в типовете на член-данните, типовете на параметрите на член функциите, тип на връщания резултат на член функциите, в тялото. Шабини на класове се използват чрез явно указване на параметрите. Както при функциите, шабини на класове не се компилират. При всяко използване на шаблон с различни параметри се генерира нов шаблонен клас. Компилират се само член-функциите и затова не можем да правим обекти от шабини на класове, а само обекти от шаблонни класове.

```
template <typename T1, typename T2> class Foo {
public:
    T1 x;
    T2 y;

    Foo(T1 val1, T2 val2) : x(val1), y(val2) {}

    void getValues(){
        cout << x << " " << y;
    }
};

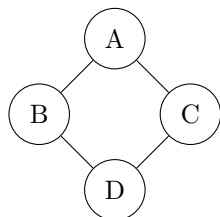
Foo<double, string> doubleString(3.14, "Hello");
Foo<float, bool> floatBool(5.67f, true);

doubleString.getValues();
floatBool.getValues();
}
```

12.3 Множествено наследяване

В C++ един клас може да наследява няколко други класове. Конструкторите се изпълняват в реда на деклариране на наследяването, а деструкторите обратно. Наследникът приема всички полета и методи от всичките си родители.

При реализиране на йерархии от класове с множествено наследяване е възможно един производен клас да наследи повече от един път даден базов клас.



Като производни на класа A, класовете B и C наследяват неговите компоненти. От друга страна, класовете B и C са базови за класа D, следователно класът D ще наследи два пъти компонентите на A – веднъж чрез класа B и веднъж чрез класа C. За член-функциите двойното наследяване не е от значение, тъй като за всяка член-функция се съхранява само едно копие. Член-данните, обаче, се дублират и обект на D ще наследи двукратно всяка член-данна, дефинирана в класа A. Многократното наследяване на клас води от една страна до затруднен достъп до многократно наследените членове, а от друга страна до поддържане на множество копия на член-данните на многократно наследения клас, което е неефективно. Преодоляването на тези недостатъци се осъществява чрез използването на виртуални базови класове. Чрез тях се дава възможност да се поделят базови класове. Когато един клас е виртуален, независимо от многократното му участие като базов клас (пряк или косвен), се създава само едно негово копие.

13 Структури от данни. Стек, опашка, дърво. Основни операции върху тях. Реализация.

За изпита ще има само две структури за описване.

13.1 Структури от данни - дефиниране на понятието

: Под структура от данни се разбира организирана информация, която може да бъде описана, създадена и обработена с помощта на програма. За да се определи една структура от данни е необходимо да се направи:

- Логическо описание на структурата, което я описва на базата на декомпозицията и на по-прости структури, а също на декомпозиция на операциите над структурата на по-прости операции.
- Физическо представяне на структурата, което дава метода за представяне на структурата в паметта на компютъра.

13.2 Списък. Логическо описание. Списък с една и две връзки. Характеристики на реализациите с една и две връзки. Сложност на операциите по добавяне, премахване и намиране на елемент. Дефиниране на клас за списък, използващ една от реализациите.

Списъкът е крайна редица от елементи от един и същ тип. Операциите включване и изключване на елемент са допустими на произволно място в редицата. Възможен е достъп до всеки елемент на редицата. Има два основни начина за представяне на списък в паметта на компютъра: свързано представяне с една връзка или свързано представяне с две връзки.

При свързаното представяне с една връзка (singly linked list) последователните елементи на списъка се съхраняват на различни места в оперативната памет, а не в последователно разположени полета. Връзката между отделните елементи се осъществява чрез указател към следващия елемент. Ако елементът е последен в списъка, стойността на този указател трябва да бъде някаква различима стойност (например NULL). За задаване на списъка е достатъчен указател към неговото начало. За удобство при реализирането на операциите включване, изключване и обхождане се въвеждат още указатели към края и към текущ елемент на списъка. В общия случай елементите на списъка при свързаното представяне с една връзка се състоят от две полета – data (данните, записани в елемента) и next (указател към следващия елемент).

При свързаното представяне с две връзки (doubly linked list) последователните елементи на списъка се съхраняват на различни места в оперативната памет, а не в последователно разположени полета. Връзките между отделните елементи се осъществяват чрез два указателя към следващия и към предходния елемент. Ако елементът е последен в списъка, стойността на указателя към следващия елемент трябва да бъде някаква различима стойност (например NULL), аналогично за указателя към предходния елемент за елемента в началото на списъка. За задаване на списъка е достатъчен указател към неговото начало. В общия случай елементите на списъка при свързаното представяне с две връзки се състоят от три полета – data (данните, записани в елемента), next (указател към следващия елемент) и prev (указател към предходния елемент).

Характеристики на двете реализации:

- Свързаният списък с една връзка използва по-малко памет, тъй като всеки елемент съдържа само един указател. Недостатък е, че списъкът може да се обхожда само в посока от началото към края. При премахване на елемент е необходимо да се знае предходният елемент.
- Свързаният списък с две връзки позволява двупосочно обхождане на елементите. Операциите по добавяне и премахване на елемент са по-удобни, тъй като всеки елемент пази указатели както към следващия, така и към предходния елемент. Недостатък е по-големият разход на памет заради допълнителния указател.

Сложност на операциите:

- Добавяне на елемент в началото: $O(1)$
- Добавяне на елемент в произволна позиция: $O(n)$
- Премахване в началото: $O(1)$
- Премахване в произволна позиция: $O(n)$
- Намиране на произволен елемент: $O(n)$

Намирането на определен елемент е това, което изисква линейно време. За разлика от масивите, където вмъкването на елемент отнема линейно време (скъпо при тежки типове данни), при списъците самото вмъкване на елемент е константна операция, ако е известна позицията за вмъкване.

Дефиниране на клас за списък с една връзка:

```
template <class T>
struct Node {
    T data;
    Node<T>* next;
    Node(const T& value, Node<T>* nxt = nullptr)
        : data(value), next(nxt) {}
};

template <class T>
class LinkedList {
private:
    Node<T>* head;

    void clear() {
        Node<T>* current = head;

        while (current != nullptr) {
            Node<T>* toDelete = current;
            current = current->next;
            delete toDelete;
        }

        head = nullptr;
    }

public:
    LinkedList() : head(nullptr) {}

    ~LinkedList() {
        clear();
    }

    bool empty() const {
        return head == nullptr;
    }

    void pushFront(const T& value) {
        Node<T>* newNode = new Node<T>(value, head);
        head = newNode;
    }

    void pushBack(const T& value) {
        Node<T>* newNode = new Node<T>(value);

        if (empty()) {
            head = newNode;
            return;
        }

        Node<T>* current = head;
        while (current->next != nullptr)
            current = current->next;

        current->next = newNode;
    }

    bool contains(const T& value) const {
        Node<T>* current = head;

        while (current != nullptr) {
            if (current->data == value) {
```

```

        return true;
    }

    current = current->next;
}

return false;
}

void remove(const T& value) {
    if (empty()) {
        return;
    }

    if (head->data == value) {
        Node<T>* toDelete = head;
        head = head->next;

        delete toDelete;
        return;
    }

    Node<T>* current = head;

    while (current->next != nullptr && current->next->data != value) {
        current = current->next;
    }

    if (current->next != nullptr) {
        Node<T>* toDelete = current->next;

        current->next = toDelete->next;

        delete toDelete;
    }
}

void insert(const T& value, const int& index) {
    if (index == 0){
        pushFront(value);
        return;
    }
    int i = 0;
    Node<T> * current = head;

    while (current->next != nullptr && i != index){
        current = current->next;
        i++;
    }

    Node<T>* new_node = new Node<T>(value);

    new_node->next = current->next;
    current->next = new_node;
}

void print() const {
    Node<T>* current = head;

    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->next;
    }

    std::cout << std::endl;
}
};

```

13.3 Стек. Логическо описание. Характеристики на статичната, динамичната и свързаната реализация. Сложност на операциите по добавяне и премахване на елемент. Дефиниране на клас за стек, използващ една от реализациите.

Стекът е линейна структура от данни, представляваща крайна редица от елементи от един и същи тип. Операциите включване и изключване на елемент са допустими само за единия край на редицата, който се нарича връх на стека. Възможен е пряк достъп само до елемента, който се намира на върха на стека. При тази организация на логическите операции, последният включен елемент се изключва пръв. Затова стекът се определя още като структура — последен влязъл – пръв излязъл (last in – first out, LIFO).

Широко се използват два основни начина за представяне на стек: последователно и свързано. При последователното (статичното) представяне, предварително в паметта се запазва блок, вътре в който се помещава стекът и той там расте и се съкращава. При включване на елементи в стека, те се помещават в последователни адреси в неизползваната част веднага след върха на стека.

При свързаното (динамично) представяне последователните елементи се съхраняват на различни места в оперативната памет, а не в последователно разположени полета. Връзката между отделните елементи се осъществява чрез указател към следващия елемент. Ако елементът е последен в стека, стойността на този указател трябва да бъде някаква различима стойност (например NULL). За задаване на стека е достатъчен указател към върха на стека. В общия случай елементите на стека при свързаното представяне се състоят от две полета – data (данните, записани в елемента) и link (указател към следващия елемент).

Сложност за добавяне и премахване:

- Добавяне на елемент: $O(1)$
- Премахване на най-горен елемент: $O(1)$

Реализация на клас за стек:

```
struct Node {
    int data;
    Node* next;
};

class Stack {
private:
    Node* topNode;
public:
    Stack() {
        topNode = nullptr;
    }

    ~Stack() {
        while (topNode != nullptr) {
            pop();
        }
    }

    void push(int value) {
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = topNode;
        topNode = newNode;
    }

    void pop() {
        if (topNode == nullptr) {
            cout << "Empty stack!";
            return;
        }

        Node* temp = topNode;
        topNode = topNode->next;
        delete temp;
    }
}
```

```

int top() const {
    if (topNode == nullptr) {
        cout << "Empty stack!";
        return;
    }

    return topNode->data;
}

void print() const {
    Node* current = topNode;

    while (current != nullptr) {
        cout << current->data << " ";
        current = current->next;
    }
}
};

```

13.4 Опашка. Логическо описание. Характеристики на статичната, динамичната и свързаната реализация. Сложност на операциите по добавяне и премахване на елемент. Дефиниране на клас за опашка, използващ една от реализациите.

Опашката е крайна редица от елементи от един и същи тип. Операцията включване е допустима за елементите от единия край на редицата, който се нарича край на опашката, а операцията изключване на елемент – само за елементите от другия край на редицата, който се нарича начало на опашката. Възможен е пряк достъп само до елемента, намиращ се в началото на опашката. При тази организация на логическите операции, пръв се изключва най-отдавна включеният елемент. Затова опашката се определя още като структура от данни – пръв влязъл – пръв излязъл (first in – first out, FIFO). Широко се използват два основни начина за физическо представяне на опашка: последователно и свързано.

При последователното представяне първоначално в паметта се запазва блок, вътре в който опашката да расте и да се съкращава. Включването на елемент в опашката се осъществява чрез поместването му в последователни адреси в неизползваната част веднага след края на опашката. Обикновено се счита, че блокът от памет е цикличен – когато краят на опашката достигне края на разпределения блок памет, но има освободена памет в неговото начало, там може да се извърши включване на елементи. При динамичната реализация размерът на опашката може да се променя по време на изпълнение на програмата чрез заделяне и освобождаване на памет. Тя позволява ефективно използване на паметта и избягва ограничението на фиксирания размер.

При свързаното представяне последователните елементи на опашката се съхраняват на различни места в оперативната памет, а не в последователно разположени полета. Връзката между отделните елементи се осъществява чрез указател към следващия елемент. Ако елементът е последен в опашката, стойността на този указател трябва да бъде някаква различима стойност (например NULL). За задаване на опашката са достатъчни указатели към началото и края на опашката. В общия случай елементите на опашката при свързаното представяне се състоят от две полета – data (данните, записани в елемента) и next (указател към следващия елемент).

Сложност за добавяне и премахване:

- Добавяне на елемент: $O(1)$
- Премахване на елемент: $O(1)$

Реализация на клас за опашка:

```

template <typename T>
class Queue {
private:
    struct Node {
        T data;
        Node* next;

        Node(const T& value, Node* nextNode = nullptr)
            : data(value), next(nextNode) {}
    };
};

```

```

};

Node* frontPtr;
Node* rearPtr;

public:
    Queue() {
        frontPtr = nullptr;
        rearPtr = nullptr;
    }

    ~Queue() {
        while (frontPtr != nullptr) {
            dequeue();
        }
    }

    void enqueue(const T& value) {
        Node* newNode = new Node(value);

        if (frontPtr == nullptr) {
            frontPtr = rearPtr = newNode;
        }
        else {
            rearPtr->next = newNode;
            rearPtr = newNode;
        }
    }

    void dequeue() {
        if (frontPtr == nullptr) {
            cout << "Empty Queue!";
            return;
        }

        Node* temp = frontPtr;
        frontPtr = frontPtr->next;

        delete temp;

        if (frontPtr == nullptr){
            rearPtr = nullptr;
        }
    }

    void print() const {
        Node* current = frontPtr;

        while (current != nullptr) {
            std::cout << current->data << " ";
            current = current->next;
        }
    }
};
}

```

13.5 Дървовидни структури от данни - кореново дърво и двоично кореново дърво. Логическо описание. Начини на представяне в паметта. Дефиниране на клас, реализиращ кореново дърво или двоично кореново дърво.

Кореново дърво от тип T е рекурсивна структура от данни, която е или празна или е образувана от:

- данна от тип T, наречена корен на дървото.
- двоично дърво от тип T наречено поддърво (може да са повече от 1)

Множеството на върховете (възлите) на едно дърво се определя рекурсивно: празното двоично дърво няма

върхове, а върховете на непразно дърво са неговият корен и върховете на поддърветата му.

Листа на кореново дърво са върховете, чиито поддървета са празни.

Двоично дърво: Двоичното дърво е кореново дърво, което има най-много две поддървета, които ще наричаме ляво и дясно поддърво.

Представяне в паметта: Двоичното дърво може да се представи по два начина:

При свързаното представяне елементите на дървото се съхраняват на различни места в оперативната памет, а не в последователно разположени полета. Връзката между отделните върхове се осъществява чрез указател към поддърветата му. Ако върхът е листо, стойността на този указател трябва да бъде някаква различима стойност (например NULL). За задаване на дърво е достатъчен само указател към корена. В общия случай елементите на двоичното дърво при свързаното представяне се състоят от три полета – data (данните, записани в елемента), left (указател към лявото поддърво) и right (указател към дясното поддърво).

В последователното представяне всички елементи на двоичното дърво се съхраняват в масив с фиксиран размер. Тук стойността се съдържа в клетките на самия масив, а структурата на дървото се определя от неговите индекси. За елемент на индекс i имаме, че лявото му поддърво се намира на индекс $2i + 1$, а дясното на $2i + 2$ (започвайки от индекс 0). За да определим дали един връх е листо проверяваме индексите на неговите деца са извън размера на масива или дали стойностите на тези индекси са някаква фиксирана от нас константа, например INT_MIN.

```
template <typename T>
class BinaryTree {
private:
    struct Node
    {
        T data;
        Node* left;
        Node* right;

        Node(const T& value) {
            data = value;
            left = nullptr;
            right = nullptr;
        }
    };

    Node* root;

    void clear(Node* current) {
        if (current == nullptr) {
            return;
        }

        clear(current->left);
        clear(current->right);

        delete current;
    }

public:
    BinaryTree() {
        root = nullptr;
    }

    ~BinaryTree() {
        clear(root);
    }

    void insert(const T& value){ // inserting element at the first free position
        Node* newNode = new Node(value);

        if (root == nullptr) {
            root = newNode;
            return;
        }
    }
};
```

```

    }

    std::queue<Node*> q;
    q.push(root);

    while (!q.empty())
    {
        Node* current = q.front();
        q.pop();

        if (current->left == nullptr) {
            current->left = newNode;
            return;
        } else {
            q.push(current->left);
        }

        if (current->right == nullptr) {
            current->right = newNode;
            return;
        } else {
            q.push(current->right);
        }
    }
}

void inorder(Node* current) const { // in-order traversal left-root-right
    if (current == nullptr) {
        return;
    }

    inorder(current->left);
    cout << current->data << " ";
    inorder(current->right);
}
};

```

13.6 Двоично кореново дърво за търсене. Логическо описание. Начини за представяне в паметта. Сложност на операциите по добавяне, премахване и търсене на елемент. Дефиниране на клас реализиращ двоично кореново дърво за търсене.

Двоично наредено дърво от тип T се дефинира рекурсивно по следния начин: празното двоично дърво от тип T е наредено и непразно двоично дърво от тип T е наредено тогава и само тогава, когато всички върхове на лявото му поддърво са по-големи от корена и всички върхове на дясното му поддърво са по-малки от корена и освен това, лявото и дясното му поддърво са двоични наредени дървета от тип T .

Нека t е двоично наредено дърво от тип T . Включването на елемента A от тип T в t се осъществява по следния начин:

- ако t е празното дърво, новото двоично наредено дърво е с корен елемента A и празни ляво и дясно поддървета;
- ако t е непразно и A е по-малко от корена му, A се включва в лявото поддърво на t ;
- ако t е непразно и A е по-голям от корена му, A се включва в дясното поддърво на t .

Ще използваме този начин за включване на елементи за да създаваме двоичните наредени дървета. В този случай, те притежават следното свойство: смесеното обхождане сортира във възходящ ред елементите от върховете на дървото. Изтриването на връх елемент A от двоичното наредено дърво t се извършва по следната схема:

- ако коренът на t е по-голям от A , изтриваме A от лявото поддърво на t ;
- ако коренът на t е по-малък от A , изтриваме A от дясното поддърво на t ;
- ако коренът на t съвпада с A и t има празно ляво поддърво, то t се заменя с дясното си поддърво;

- ако коренът на t съвпада с A и t има празно дясно поддърво, то t се заменя с лявото си поддърво;
- ако коренът на t съвпада с A и t има непразни ляво и дясно поддърво, то се намира най-големият елемент в лявото поддърво (най-дясното му листо), разменя се неговата стойност със стойността на корена (която е A) и въпросният връх се изтрива (той има празно дясно поддърво, така че влизаме в предния случай).

Сложност на операциите: Нека двоичното дърво съдържа n върха.

- Добавяне на елемент: $O(\log n)$ при почти-балансирано дърво, $O(n)$ иначе.
- Търсене на елемент: $O(\log n)$ при почти-балансирано дърво, $O(n)$ иначе.
- Изтриване на елемент: $O(\log n)$ при почти-балансирано дърво, $O(n)$ иначе.

```
template <typename T>
class BinarySearchTree
{
private:
    struct Node {
        T data;
        Node* left;
        Node* right;

        Node(const T& value) {
            data = value;
            left = nullptr;
            right = nullptr;
        }
    };

    Node* root;

public:
    Node* insert(Node* current, const T& value) {
        if (current == nullptr) {
            return new Node(value);
        }

        if (value < current->data) {
            current->left = insert(current->left, value);
        } else if (value > current->data) {
            current->right = insert(current->right, value);
        }

        return current;
    }

    void inorder(Node* current) const {
        if (current == nullptr) {
            return;
        }

        inorder(current->left);
        std::cout << current->data << " ";
        inorder(current->right);
    }

    Node* findMax(Node* current) {
        while (current->right != nullptr) {
            current = current->right;
        }

        return current;
    }

    Node* remove(Node* current, const T& value) {
        if (current == nullptr) {
            return nullptr;
        }
    }
};
```



```

    }

    if (value < current->data) {
        current->left = remove(current->left, value);
    } else if (value > current->data) {
        current->right = remove(current->right, value);
    }
    else {
        if (current->left == nullptr) {
            Node* temp = current->right;
            delete current;
            return temp;
        }

        if (current->right == nullptr) {
            Node* temp = current->left;
            delete current;
            return temp;
        }

        //if node has two children
        Node* maxLeft = findMax(current->left);
        current->data = maxLeft->data;
        current->left = remove(current->left, maxLeft->data);
    }

    return current;
}

void clear(Node* current)
{
    if (current == nullptr) {
        return;
    }

    clear(current->left);
    clear(current->right);

    delete current;
}

};

```

14 Функционално програмиране. Обща характеристика на функционалния стил на програмиране. Дефиниране и използване на функции. Модели на оценяване. Функции от по-висок ред.

14.1 Характерни особености на функционалния стил на програмиране.

Функция е програмна единица, която връща резултат. Те могат да бъдат от произволно висок ред - подавани като параметри, връщани и т.н. функциите:

- Могат да имат параметри.
- Могат да се прилагат над аргументи.
- Аргументите им могат да бъдат други функции.
- Могат да се дефинират чрез себе си (рекурсия).

Във функционалното програмиране основното действие е обръщение към функции. Основният начин за разделяне на програмата на части е дефинирането на функция, като основното правило за композиция е суперпозицията на функции.

Функционалният стил е представител на декларативния стил на програмиране.

- В чистото функционално програмиране обикновено отсъстват конструкции като променливи с изменяемо състояние, цикли и присвояване. Вместо тях се използват рекурсия и композиция на функции.
- В декларативните езици (ДЕ) програмата явно посочва какви свойства има желаният резултат. В ДЕ, в частност ФП програмите се изграждат по схемата:

Програма = списък от дефиниции на функции или списък от равенства.

14.2 Основни компоненти на функционалния стил на програмиране.

Основният компонент на функционалните програми са функциите, тяхното дефиниране, използване и оценяване.

Дефинирането на функции представлява свързване на имена с изрази, които се оценяват. Рекурсията е основен метод за дефиниране.

14.3 Примитивни изрази.

Това са изрази, които не могат да се декомпонират - атомарни изрази на функционалните езици.

В Scheme атомарни изрази са:

- Булеви константи: (`#f`, `#t`)
- Числови константи: (`15`, `2/3`, `-1.1`)
- Знакови константи: (`#a`, `# newline`)
- Низови константи: (`"Scheme"`)
- Вградена функция: (`+`, `-`, `*`, `/`, `sqrt`)

14.4 Средства за комбиниране и абстракция.

Изразът е или атом (атомарна константа или променлива) или обръщение към функция, което в Scheme е име на функция (оператор) и аргументите на функцията (операнди).

Пример:

```
(+ 2 3)
(* (+ 1 2) 4)
(sqrt 25)
```

В първия пример оператор е `+`, а операндите са `2` и `3`. Във втория пример имаме вложени комбинации.

В Scheme комбинация се нарича израз, конструиран като списък от изрази, заградени в скоби. Има следният синтаксис:

(оператор операнд₁ операнд₂ ... операнд_n)

- Първият елемент на списъка се нарича оператор и се очаква да се оцени до функция, която да се приложи върху оценките на останалите аргументи, наричани операнди.
- Стойността на комбинацията се получава чрез прилагане на процедурата, зададена чрез оператора, към аргументите, които са стойностите на операндите.

В Scheme функция се дефинира, използвайки следния синтаксис:

(define (име параметри) тяло)

Пример:

```
(define (square x)
  (* x x))
```

Приложение:

```
(square 5) ; -> 25
```

За дефиниране на съставна функция в Scheme се използва синтаксисът:

(define (име параметри)
 (функция1 (функция2 параметри)))

Пример:

```
(define (sum-of-squares x y)
  (+ (square x) (square y)))
```

Приложение:

```
(sum-of-squares 3 4) ; -> 25
```

14.5 Оценяване на изрази.

Общо правило за оценяване на комбинациите:

- Оценяват се подизразите на комбинацията
- Прилага се функцията, която е оценка на най-левия подизраз (операторът) към операндите, които са оценки на останалите подизрази.

Пример: $(*(+ 2 (* 4 6)) (+ 3 5 7))$ изисква прилагането на общото правило върху четири различни комбинации. Оценяването на комбинацията може да бъде представена по следния начин:

$$(*(+2(*4 6))(+3 5 7)) \rightarrow (*(+2 24)(15)) \rightarrow (*26 15) \rightarrow 390$$

Правила за оценяване на числа, низове, имена и вградени оператори:

- Оценката на число е самото число
- Оценката на низ е самият низ
- Оценката на вграден оператор е поредица от машинни инструкции, които реализират този оператор
- Оценките на останалите имена са стойностите, свързани с тези имена в текущата среда

Оператори, които са изключения от общото правило за оценяване на комбинации се наричат специални форми. Примери за такива оператори са define, if, cond.

14.6 Дефиниране на функция и оценяване на приложение на функция.

При оценяване на комбинация, чиито оператор е функция, се оценяват елементите на комбинацията и се прилага процедурата, която е стойност на оператора на комбинацията, към аргументите, които са стойностите на операндите на комбинацията.

Пример: `(define (square x)(* x x))`

При оценяване `define`, символът `square` се свързва със зададена дефиниция в текущата среда.

Модел на заместването при оценяване на обръщени към функции:

- При оценяване на комбинация най-напред се оценяват подизразите на тази комбинация, след което оценката на първия подизраз се прилага върху оценките на останалите подизрази.
- Прилагането на функция към получените аргументи става като формалните параметри се заместят със съответните аргументи. Оценка на последния израз от тялото става оценка на обръщението към функцията.

14.7 Модели на оценяване. Апликативно (`call-by-value`) и нормално (`call-by-name`) оценяване.

При посочения модел на оценяване чрез заместване са възможни два подхода за оценяване: апликативен и нормален.

Апликативния подход за оценяване: отначало се оценяват операндите и след това към получените оценки се прилага резултатът от оценяването на оператора, т.е. иска да се оцени всичко, което е дадено като аргумент преди да почне да прилага функцията.

Нормално оценяване: първо се разгъва тялото на функцията чрез заместване на аргументите без те да се оценяват предварително. Аргументите се оценяват едва когато стойностите им станат необходими.

Двата модела могат да дадат различни резултати при един и същи програми. Пример:

```
(define (p) (p))

(define (test x y)
  (if (= x 0) 0 y))
```

Изразът:

```
(test 0 (p))
```

- При апликативно оценяване програмата ще се зацikli, защото първо се оценява `(p)`.
- При нормално оценяване резултатът е 0, защото `y` никога не се използва.

В стандартните езици за програмиране също има нормално (лениво) оценяване в някаква форма. Например при конюнкция, ако едно е лъжа, то останалите не се пресмятат.

14.8 Функции от по-висок ред. Функциите като параметри и оценки на обръщени към функции.

Функция от по-висок ред е такава функция, която приема функция за параметър или връща функция като резултат.

Пример в Scheme:

```
(define (fixed_point f x) (= (f x) x))
```

Къринг: В `haskell` можем да разгледаме функция с $n + 1$ аргумента, като такава с 1 аргумент, която връща функция с n аргумента. Това представяне се нарича къринг.

```
add :: Int -> Int -> Int
add x y = x + y

inc = add 1

inc 5 -- -> 6
```

14.9 Анонимни (лямбда) функции.

Можем да конструираме параметрите на функции от по-висок ред "на място без да има задаваме имена. Синтаксис: (lambda (параметри) тяло). Оценява се до функционален обект със съответните параметри и тяло.

Пример:

```
(lambda (x) (+ x 3)) -> #<procedure>
```

Приложена:

```
((lambda (x) (+ x 3)) 5) -> 8
```

Анонимни функции от по-висок ред:

```
(define (twice f) (lambda (x) (f (f x))))
```

Приложена:

```
(twice square) -> #<procedure>  
((twice square) 3) -> 81
```

15 Функционално програмиране. Списъци. Потоци и отложено оценяване.

Разгледай по-внимателно, съгласувай с консултацията.

15.1 Списъци. Представяне.

Списъци в Scheme

Списъкът е рекурсивно дефинирана структура:

- Празният списък '() е списък.
- Ако h е произволна стойност и t е списък, то $(cons\ h\ t)$ е списък.

При това:

- h се нарича **глава** на списъка;
- t се нарича **опашка** на списъка.

Конструиране на списъци:

- $(list\ a_1\ \dots\ a_n)$ създава списък от елементите $a_1, \dots, a_n$.
- Еквивалентно:

```
(cons a1
      (cons a2
            ...
            (cons an '())))
```

Списъци в Haskell

Списъкът е крайна или безкрайна последователност от елементи от един и същ тип.

Основен конструктор е операторът `::`:

```
1 : (2 : (3 : (4 : [])))
```

Съкратен запис:

```
[1,2,3,4]
```

Операторът `:` е дясно-асоциативен:

```
[1,2,3,4] = 1 : 2 : 3 : 4 : []
```

15.2 Основни операции със списъци.

Операции в Scheme

- `car` — връща първия елемент на списък;
- `cdr` — връща опашката на списък;
- `null?` — проверява дали списъкът е празен.

Намиране на дължина:

```
(define (length l)
  (define (iter lst count)
    (if (null? lst)
        count
        (iter (cdr lst) (+ count 1))))
  (iter l 0))
```

Обединяване на списъци:

```
(append '(a b) '(c d))
;; -> (a b c d)
```

Обръщане на списък:

```
(reverse '((a b) (c d) e))  
;; -> (e (c d) (a b))
```

Проверка за равенство:

- `eq?` — проверява дали два аргумента са един и същ обект;
- `equal?` — проверява структурно равенство.

Проверка за принадлежност:

- `memq` използва `eq?`;
- `member` използва `equal?`.

Примери:

```
(member '(a b) '((a b) (c d)))  
;; -> ((a b) (c d))  
  
(member 'a '(b a c a d))  
;; -> (a c a d)
```

Основни функции в Haskell

- `head :: [a] -> a` — глава на списък;
- `tail :: [a] -> [a]` — опашка;
- `null :: [a] -> Bool` — проверка за празен списък;
- `length :: [a] -> Int` — дължина;
- `reverse :: [a] -> [a]` — обръщане;
- `elem :: Eq a => a -> [a] -> Bool` — принадлежност;
- `init :: [a] -> [a]` — всички елементи без последния;
- `last :: [a] -> a` — последен елемент;
- `take :: Int -> [a] -> [a]` — първите n елемента;
- `drop :: Int -> [a] -> [a]` — списък без първите n елемента.

List comprehensions в Haskell

Механизъм за конструиране на нови списъци чрез използване на съществуващи.

Общ вид:

```
[ expr | generator, condition ]
```

където:

- генераторът е от вида:

```
pattern <- list
```

- условието е булев израз.

Примери:

```
[2*x | x <- [1..5]]  
-- [2,4,6,8,10]  
  
[(x,y) | x <- [1,2,3],  
         y <- [5,6,7],  
         x + y <= 8]  
-- [(1,5),(1,6),(1,7),(2,5),(2,6),(3,5)]
```

15.3 Функции от по-висок ред за работа със списъци.

Функции от по-висок ред са функции, които приемат функции като аргументи или връщат функции като резултат.

Трансформиране — `map`

```
map :: (a -> b) -> [a] -> [b]

map _ [] = []
map f (x:xs) = f x : map f xs
```

Примери:

```
map (^2) [1,2,3]
-- [1,4,9]

map (\f -> f 2) [(^2),(+1),(*3)]
-- [4,3,6]
```

Филтриране — `filter`

```
filter :: (a -> Bool) -> [a] -> [a]

filter _ [] = []
filter p (x:xs)
  | p x      = x : filter p xs
  | otherwise = filter p xs
```

Примери:

```
filter odd [1..5]
-- [1,3,5]
```

Акумулиране — `fold`

Дясно свиване:

```
foldr :: (a -> b -> b) -> b -> [a] -> b

foldr f z []      = z
foldr f z (x:xs) = f x (foldr f z xs)
```

Ляво свиване:

```
foldl :: (b -> a -> b) -> b -> [a] -> b

foldl f z []      = z
foldl f z (x:xs) = foldl f (f z x) xs
```

Пример:

```
foldr (+) 0 [1,2,3]
-- 6
```

15.4 Отложено оценяване.

Отложеното оценяване (lazy evaluation) е механизъм, при който даден израз се оценява едва когато стойността му стане необходима.

Promise (обещание)

Promise е отложено изчисление, което може да бъде форсирано по-късно.

В Scheme:

- `delay` — създава promise;
- `force` — оценява promise.

Пример:

```
(define delayed
  (delay (+ a 3)))

(define a 5)

(force delayed)
;; -> 8
```

След първото форсиране резултатът се запомня и не се преизчислява.

15.5 Безкрайни потоци и безкрайни списъци.

Поток (stream) е последователност, чиито елементи се изчисляват при нужда.

Потоците позволяват работа с:

- много големи структури;
- безкрайни последователности.

Основни операции:

- `cons-stream` — конструктор;
- `stream-car` — първи елемент;
- `stream-cdr` — опашка;
- `empty-stream?` — проверка за празен поток.

15.6 Основни операции и функции от по-висок ред.

Аналогично на списъците могат да се дефинират:

- `stream-map`;
- `stream-filter`;
- `stream-ref`;
- `stream-take`.

Тези операции работят лениво и изчисляват само необходимите елементи.

15.7 Работа с безкрайни потоци.

Неявно конструиране

Използват се генератори.

Пример:

```
(define (integers-from n)
  (cons-stream n
    (integers-from (+ n 1))))

(define integers
  (integers-from 1))
```

Явно конструиране

Използва се рекурсия върху самия поток.

Пример:

```
(define ones
  (cons-stream 1 ones))

(define ints
  (cons-stream 1
    (add-streams ones ints)))
```

Тук всеки следващ елемент на потока се получава чрез използване на вече конструираната част.

16 Синтаксис и семантика на предикатното смятане от първи ред.

Език на предикатното смятане от първи ред:

1. Логическа част, съдържаща:

- Изброимо множество от букви: $Var = x_1, \dots, x_n, \dots, n, \dots \in \mathbb{N}$, които наричаме индивидуални променливи.
- Съжителни връзки: едноместни ($\neg$) и двуместни ($\&, \vee, \rightarrow, \leftarrow$)
- Квантори: $\forall, \exists$
- Помощни символи: $(,)$

2. Нелогическа част на предикатен език α

- Азбука на индивидуалните константи: $Const_\alpha$
- Азбука на функционалните символи: $Func_\alpha$, където всеки функционален символ f има фиксирана ариност $\#f > 0$.
- Азбука на предикатните символи: $Pred_\alpha$
- Някоя азбука може да е празна.

Терм: Индуктивна дефиниция:

- Индивидуалните променливи са термове.
- Индивидуалните константи от $Const_\alpha$ са термове от α
- Ако $\tau_1, \dots, \tau_n$ са термове, а $f \in Func_\alpha, \#f = n$, то $f(\tau_1, \dots, \tau_n)$ е терм.

Ще означаваме множеството от термове на езика α с $\mathcal{T}_\alpha$

Формула: Индуктивна дефиниция:

- Атомарните формули от α са формули от α
- Ако φ, ψ са формули от α , то $\neg\varphi, (\varphi \& \psi), (\varphi \vee \psi), (\varphi \rightarrow \psi), (\varphi \leftrightarrow \psi)$ са формули от α
- Ако φ е формула, а x е индивидуална променлива, то $\forall x\varphi$ и $\exists x\varphi$ са също формули от α

Структура за предикатен език от първи ред: Нека α е предикатен език от първи ред. Структура за езика α е наредена двойка $\langle A, \mathbb{I} \rangle$, където A е непразно множество от обекти (универсум), а $\mathbb{I}$ е интерпретация на α в A :

- $\mathbb{I}(c) \in A$
- $\mathbb{I}(f) : A^{\#f} \rightarrow A$
- $\mathbb{I}(p) \subseteq \underbrace{A \times \dots \times A}_{\#p}$

Оценка на индивидуалните променливи: Оценка на $v \in A$ е: $v : Var \rightarrow A$

Стойност на формула: Стойността на формулата φ в структура $\mathcal{A}$ при оценка v е $\|\varphi\|^{\mathcal{A}}[v] \in \{\text{И}, \text{Л}\}$

Разглеждаме възможностите за φ

- $\varphi \in At_\alpha$, т.е. $\varphi = p(\tau_1, \dots, \tau_n)$, където τ_i са термове от α , p е предикатен символ.

$$\|\varphi\|^{\mathcal{A}}[v] = \text{И} \stackrel{def}{\iff} \langle \|\tau_1\|^{\mathcal{A}}[v], \dots, \|\tau_n\|^{\mathcal{A}}[v] \rangle \in p^{\mathcal{A}}$$

- $\varphi = (\tau_1 \doteq \tau_2)$

$$\|\varphi\|^{\mathcal{A}}[v] = \text{И} \stackrel{def}{\iff} \|\tau_1\|^{\mathcal{A}}[v] = \|\tau_2\|^{\mathcal{A}}[v]$$

•

$$\|\neg\varphi\|^{\mathcal{A}}[v] = \text{И} \stackrel{def}{\iff} H_{\neg}(\|\varphi\|^{\mathcal{A}}[v])$$

- $\|\varphi \sigma \psi\|^{\mathcal{A}} = \text{И} \stackrel{def}{\iff} H_{\sigma}(\|\varphi\|^{\mathcal{A}}[v], \|\psi\|^{\mathcal{A}}[v])$

- $\|\forall a\varphi\|^{\mathcal{A}} = \text{И} \stackrel{def}{\iff} \|\varphi\|^{\mathcal{A}}[v_a^x] = \text{И}$ и всеки път, когато $a \in A$, където:

$$v_a^x(y) = \begin{cases} v(y), & y \neq x \\ v(a), & y = x \end{cases}$$

- $\|\exists a\varphi\|^{\mathcal{A}} = \text{И} \stackrel{def}{\iff}$ и има елемент $a \in A$, т.че $\|\varphi\|^{\mathcal{A}}[v_a^x] = \text{И}$, където:

$$v_a^x(y) = \begin{cases} v(y), & y \neq x \\ v(a), & y = x \end{cases}$$

Вярност при оценка: Ако $\|\varphi\|^{\mathcal{A}}[v] = \text{И}$ пишем $\mathcal{A} \models_v \varphi$ и казваме, че формулата φ е вярна в структурата $\mathcal{A}$ при оценка v .

В противен случай пишем $\mathcal{A} \not\models_v \varphi$

Изпълнима формула: Формулата φ се нарича изпълнима, ако има структура $\mathcal{A}$ и оценка v , такива че $\mathcal{A} \models_v \varphi$

Тъждествена вярност на формула: В структурата $\mathcal{A}$ ако при произволна оценка v формулата φ е вярна, то пишем $\mathcal{A} \models \varphi$ и казваме, че формулата е вярна.

Предикатна тавтология: Ако за всяка структура $\mathcal{A}$ е в сила, че $\mathcal{A} \models \varphi$, то казваме, че φ е предикатна тавтология.

Област на действие: Нека $\varphi = \alpha Q\beta$, $Q \in \{\exists, \forall\}$. Съгласно ЕСА $\beta = x\beta_1$, където x е индивидуна променлива, откъдето $\varphi = \alpha Qx\beta_1$. Това участие се казва, че е квантор по x

Има единствена формула $\psi : \beta_1 = \psi\beta_2$, $\varphi = \alpha Qx\psi\beta_2$. Това участие на думата $x\psi$ във φ се нарича област на действие на отбелязаното участие на Q .

Свободно и свързано участие на променлива във формула: Едно участие на индивидуалната променлива x във формулата φ се нарича свободно участие на x във φ , ако то не е в област на действие на квантор по x . В противен случай, то се нарича свързано участие на x във φ .

Свързани променливи за формула: Нека φ е предикатна формула. Свързани променливи на φ са всички индивидуални променливи, които имат свързано участие във φ . Бележим ги с $Var^{bd}[\varphi]$.

Свободни променливи: Свободни променливи на φ са всички индивидуални променливи, които имат свободно участие във φ . Бележим ги с $Var^{free}[\varphi]$.

16.1 Доказва се, че верността на формула в структура при оценка не зависи от стойността на променливите, които не се срещат като свободни във формулата.

Твърдение: Нека α е език на предикатното смятане и $\mathcal{A}$ е структура за α . Тогава всеки път, когато φ е формула от α , а v_1, v_2 са оценки в $\mathcal{A}$, удовлетворяващи условието (*):

$$\text{за всяка променлива } x \in Var^{free}[\varphi] \Rightarrow v_1(x) = v_2(x)$$

То тогава е в сила равенството: $\|\varphi\|^{\mathcal{A}}[v_1] = \|\varphi\|^{\mathcal{A}}[v_2]$

Доказателство: Индукция относно построяването на φ :

1. φ е атомарна: $\varphi = p(\tau_1, \dots, \tau_n)$, τ_i са термове. Нека v_1, v_2 са оценки, удовлетворяващи условието (*). $Var^{free}[\varphi] = Var[\tau_1] \cup \dots \cup Var[\tau_n]$

Нека $1 \leq j \leq n$. Тогава за всяко $x \in Var[\tau_j]$ има $v_1(x) = v_2(x)$. Следователно $\|\tau_j\|^{\mathcal{A}}[v_1] = \|\tau_j\|^{\mathcal{A}}[v_2]$

$$\begin{aligned} \|\varphi\|^{\mathcal{A}}[v_1] = \text{И} &\leftrightarrow \langle \|\tau_1\|^{\mathcal{A}}[v_1], \dots, \|\tau_n\|^{\mathcal{A}}[v_1] \rangle \in p^{\mathcal{A}} \\ &\leftrightarrow \langle \|\tau_1\|^{\mathcal{A}}[v_2], \dots, \|\tau_n\|^{\mathcal{A}}[v_2] \rangle \in p^{\mathcal{A}} \\ &\leftrightarrow \|\varphi\|^{\mathcal{A}}[v_2] = \end{aligned}$$

2. $\varphi = (\tau_1 \stackrel{\circ}{=} \tau_2)$:

$$\begin{aligned}\|\varphi\|^{\mathcal{A}}[v_1] = \text{И} &\leftrightarrow \|\tau_1\|^{\mathcal{A}}[v_1] = \|\tau_2\|^{\mathcal{A}}[v_1] \\ &\leftrightarrow \|\tau_1\|^{\mathcal{A}}[v_2] = \|\tau_2\|^{\mathcal{A}}[v_2] \leftrightarrow \|\varphi\|^{\mathcal{A}}[v_2]\end{aligned}$$

3. $\varphi = \neg\varphi_1$ и за φ твърдението е вярно.

Нека v_1, v_2 удовлетворяват (*). Имаме, че $Var^{free}[\varphi] = Var^{free}[\varphi_1]$, откъдето $v_1(x) = v_2(x)$ за всяко $x \in Var^{free}[\varphi_1]$

От И.Х. $\|\varphi_1\|^{\mathcal{A}}[v_1] = \|\varphi_1\|^{\mathcal{A}}[v_2]$

$$\|\varphi\|^{\mathcal{A}}[v_1] = H_{\neg}(\|\varphi_1\|^{\mathcal{A}}[v_1]) = H_{\neg}(\|\varphi_1\|^{\mathcal{A}}[v_2]) = \|\varphi\|^{\mathcal{A}}[v_2]$$

4. $\varphi = (\varphi_1 \sigma \varphi_2)$ и за φ_1, φ_2 твърдението е вярно.

Нека v_1, v_2 удовлетворяват (*). Имаме, че $Var^{free}[\varphi] = Var^{free}[\varphi_1] \cup Var^{free}[\varphi_2]$, откъдето $v_1(x) = v_2(x)$ за всяко $x \in Var^{free}[\varphi_j], j \in \{1, 2\}$

От И.Х. $\|\varphi_j\|^{\mathcal{A}}[v_1] = \|\varphi_j\|^{\mathcal{A}}[v_2]$

$$\begin{aligned}\|\varphi\|^{\mathcal{A}}[v_1] &= H_{\sigma}(\|\varphi_1\|^{\mathcal{A}}[v_1], \|\varphi_2\|^{\mathcal{A}}[v_1]) \\ &= H_{\sigma}(\|\varphi_1\|^{\mathcal{A}}[v_2], \|\varphi_2\|^{\mathcal{A}}[v_2]) = \|\varphi\|^{\mathcal{A}}[v_2]\end{aligned}$$

5. $\varphi = Qy\psi$ и за ψ твърдението е вярно, $Q \in \{\exists, \forall\}$, y е индивидуна променлива.

Нека v_1, v_2 удовлетворяват (*). Имаме, че $Var^{free}[\psi] \subseteq Var^{free}[\varphi] \cup \{y\}$

Нека $a \in A$. Да разгледаме $v_{1_a}^y, v_{2_a}^y$ модифицираните оценки v_1, v_2 .

Тогав за всяко $x \in Var^{free}[\psi]$ има $v_{1_a}^y(x) = v_{2_a}^y(x)$.

От И.Х. за ψ , приложено към $v_{1_a}^y, v_{2_a}^y$ имаме, че $\|\psi\|^{\mathcal{A}}[v_{1_a}^y] = \|\psi\|^{\mathcal{A}}[v_{2_a}^y]$

- $Q = \forall$

Нека $\|\varphi\|^{\mathcal{A}}[v_1] = \text{И}$

Нека $a \in A$ и да разгледаме, $\psi, v_{1_a}^y, v_{2_a}^y : \|\psi\|^{\mathcal{A}}[v_{1_a}^y] = \|\psi\|^{\mathcal{A}}[v_{2_a}^y]$

Т.к $\|\varphi\|^{\mathcal{A}}[v_1] = \text{И}$ за това a има $\|\psi\|^{\mathcal{A}}[v_{1_a}^y] = \text{И}$ Следователно: $\|\psi\|^{\mathcal{A}}[v_{2_a}^y] = \text{И}$. Тъй като a бе произволно от A , то $\|\psi\|^{\mathcal{A}}[v_2] = \text{И}$. По аналогичен начин получаваме, че $\|\varphi\|^{\mathcal{A}}[v_1] = \text{И}$

Така $\|\varphi\|^{\mathcal{A}}[v_1] = \|\varphi\|^{\mathcal{A}}[v_2]$

- $Q = \exists$

Нека $\|\varphi\|^{\mathcal{A}}[v_1] = \text{И}$

Следователно има $a \in A$ и нека a_0 е свидетел за това, т.е. $\|\psi\|^{\mathcal{A}}[v_{1_{a_0}}^y] = \text{И}$

Имаме, че за произволно $a \in A$ е в сила $\psi, v_{1_a}^y, v_{2_a}^y : \|\psi\|^{\mathcal{A}}[v_{1_a}^y] = \|\psi\|^{\mathcal{A}}[v_{2_a}^y]$, в частност при $a = a_0$.

Следователно: $\|\psi\|^{\mathcal{A}}[v_{2_{a_0}}^y] = \text{И}$. Откъдето $\|\psi\|^{\mathcal{A}}[v_2] = \text{И}$. По аналогичен начин получаваме, че $\|\varphi\|^{\mathcal{A}}[v_1] = \text{И}$

Така $\|\varphi\|^{\mathcal{A}}[v_1] = \|\varphi\|^{\mathcal{A}}[v_2]$

□

16.2 Формулира се твърдение за стойността при замяна на променливи с термове (лема за субституциите) и се доказва в частния случай на безкванторна формула.

Теорема: Нека $\mathcal{A}$ е структура за α , φ е формула от α , x е индивидуна променлива, а τ е терм, такъв че заместването на всяко свободно участие на x във φ с τ е допустимо. Тогав всеки път, когато v_1, v_2 са оценки в $\mathcal{A}$, изпълняващи (*):

$$\begin{cases} v_1(x) = \tau^{\mathcal{A}}[v_2] \\ v_1(y) = v_2(y), \text{ за всяко } y \in Var^{free}[\varphi] \setminus \{x\} \end{cases}$$

е в сила равенството: $\|\varphi\|^{\mathcal{A}}[v_1] = \|\varphi[x/\tau]\|^{\mathcal{A}}[v_2]$

Доказателство: Индукция по построението на φ - безкванторна

1. φ е атомарна: $\varphi = p(\kappa_1, \dots, \kappa_k)$ и нека $\varphi[x/\tau]$ е допустимо.

Нека v_1, v_2 удовлетворяват (*) за φ, x, τ . Ако $z \in Var[\kappa_i]$, то $z \in Var^{free}[\varphi]$.

Така имаме $z \in Var[\kappa_i]$, откъдето

$$\begin{cases} v_1(x) = \tau^{\mathcal{A}}[v_2], z = x \\ v_1(z) = v_2(z), z \neq x \end{cases}$$

Следователно $\kappa_i^{\mathcal{A}}[v_1] = \kappa_i[x/\tau]^{\mathcal{A}}[v_2]$

$$\begin{aligned} \|\varphi\|^{\mathcal{A}}[v_1] = \text{И} &\leftrightarrow \langle \kappa_1^{\mathcal{A}}[v_1], \dots, \kappa_k^{\mathcal{A}}[v_1] \rangle \in p^{\mathcal{A}} \\ &\leftrightarrow \langle \kappa_1[x/\tau]^{\mathcal{A}}[v_2], \dots, \kappa_k[x/\tau]^{\mathcal{A}}[v_2] \rangle \in p^{\mathcal{A}} \\ &\leftrightarrow \|p(\kappa_1[x/\tau], \dots, \kappa_k[x/\tau])\|^{\mathcal{A}}[v_2] = \text{И} \\ &\leftrightarrow \|p(\kappa_1, \dots, \kappa_k)[x/\tau]\|^{\mathcal{A}}[v_2] = \|\varphi[x/\tau]\|^{\mathcal{A}}[v_2] = \text{И} \end{aligned}$$

2. $\varphi = \kappa_1 \stackrel{\circ}{=} \kappa_2$ - аналогично

3. $\varphi = \neg\varphi_1$ и за φ_1 твърдението е вярно.

$\varphi[x/\tau]$ е допустима, следователно $\varphi_1[x/\tau]$ е допустима. $\varphi[x/\tau] = \neg[\varphi_1[x/\tau]]$

Нека v_1, v_2 удовлетворяват (*) за φ, x, τ . Тъй като $Var^{free}[\varphi] = Var^{free}[\varphi_1]$, то v_1, v_2 изпълняват (*) и за φ_1 . Получаваме:

$$\begin{aligned} \|\varphi_1\|^{\mathcal{A}}[v_1] &= \|\varphi_1[x/\tau]\|^{\mathcal{A}}[v_2] \\ H_{\neg}(\|\varphi_1\|^{\mathcal{A}}[v_1]) &= H_{\neg}(\|\varphi_1[x/\tau]\|^{\mathcal{A}}[v_2]) \\ \|\varphi\|^{\mathcal{A}}[v_1] &= \|\varphi[x/\tau]\|^{\mathcal{A}}[v_2] \end{aligned}$$

4. $\varphi = (\varphi_1 \sigma \varphi_2), \sigma \in \{\&, \vee, \Rightarrow, \Longleftrightarrow\}$ и теоремата е изпълнена за φ_1, φ_2 .

Нека $\varphi[x/\tau]$ е допустима и $v_1, v_2, \varphi, x, \tau$ удовлетворяват (*).

Щом $\varphi[x/\tau]$ е допустима, то $\varphi_j[x/\tau]$ също е допустима.

$$Var^{free}[\varphi] = Var^{free}[\varphi_1] \cup Var^{free}[\varphi_2].$$

$$Var^{free}[\varphi] \setminus \{x\} = Var^{free}[\varphi_1] \setminus \{x\} \cup Var^{free}[\varphi_2] \setminus \{x\}$$

Следователно $\|\varphi_j\|^{\mathcal{A}}[v_1] = \|\varphi_j[x/\tau]\|^{\mathcal{A}}[v_2]$

$$\begin{aligned} \varphi[v_1] &= H_{\sigma}(\|\varphi_1\|^{\mathcal{A}}[v_1], \|\varphi_2\|^{\mathcal{A}}[v_1]) \\ &= H_{\sigma}(\|\varphi_1[x/\tau]\|^{\mathcal{A}}[v_2], \|\varphi_2[x/\tau]\|^{\mathcal{A}}[v_2]) \\ &= \|(\varphi_1[x/\tau] \sigma \varphi_2[x/\tau])\|^{\mathcal{A}}[v_2] = \\ &= \|(\varphi_1 \sigma \varphi_2)[x/\tau]\|^{\mathcal{A}}[v_2] = \|\varphi\|^{\mathcal{A}}[v_2] \end{aligned}$$

□

17 Изводимост и компютърно генериране на доказателства.

17.1 Литерали и дизюнкти.

Литерал: Литерал е формула, която е или съждителна променлива, или отрицание на съждителна променлива.

$$L = \begin{cases} p \\ \neg p \end{cases}$$

Дуален литерал отбелязваме с L^δ и дефинираме:

$$L^\delta \leq \begin{cases} \neg p, & L = p \\ p, & L = \neg p \end{cases}$$

Дизюнкт: наричаме всяко множество от литерали, включително и празното множество.

Верен в структура дизюнкт: Нека D е дизюнкт, а I е булева оценка за някоя структура. Казваме, че I е модел за D и пишем $I \models D$, ако има $L \in D : I \models L$. Едно множество от дизюнкти S е изпълнимо, ако има поне един модел за него, т.е. съществува булева оценка I , такава че всеки път, когато $D \in S, I \models D$

Празния дизюнкт: Празният дизюнкт отбелязваме с $\blacksquare$. Само той е неизпълним.

Доказателство: За всяка булева оценка $I, I \models \blacksquare \leftrightarrow$ има литерал $L \in \blacksquare, I \models L$. Но празният дизюнкт не съдържа литерали, откъдето $I \not\models \blacksquare$

17.2 Резолвента

Правилото за резолюция: Нека D_1, D_2 са дизюнкти и L е литерал. Казваме, че правилото за резолюция е приложимо към D_1, D_2 и пишем $!R_L(D_1, D_2)$, ако $L \in D_1$ и $L^\delta \in D_2$.

Резултат от прилагането на правилото за резолюция е нов дизюнкт $R_L(D_1, D_2) = (D_1 \setminus \{L\}), D_2 \setminus \{L^\delta\}$

Резолвента: Резолвента на два дизюнкта D_1, D_2 наричаме всеки дизюнкт D , който се получава от D_1, D_2 при прилагане на правилото за резолюцията.

Резолютивен извод: Нека S е множество от дизюнкти. Резолютивен извод от S се нарича крайна редица $D_1, \dots, D_n$, т.че всеки нейн член е елемент на S или е резолвента на два предходни члена.

Дизюнктът D е **резолютивно изводим** от S , ако има резолютивен извод от S , чийто последен член е D .
Пишем $S \vdash^r D$

Лема: Нека D е резолвента на D_1 и D_2 . Всеки път, когато I е модел за $\{D_1, D_2\}$ е в сила $I \models D$

Доказателство: Нека $D = R_L(D_1, D_2)$. Нека I е произволен модел за $\{D_1, D_2\}$.

$$D = (D_1 \setminus \{L\}) \cup (D_2 \setminus \{L^\delta\})$$

1. сл. $I \models L$, следователно $I \not\models L^\delta$. Но $I \models D_2$, откъдето има литерал $L_1 \in D_2 : I \models L_1, I \not\models L^\delta$, следователно $L_1 \neq L^\delta$ и $L_1 \in D_2 \setminus \{L^\delta\} \subseteq D$

Следователно $L_1 \in D$ и $I \models L_1$ и значи $I \models D$

2. $I \not\models L$. Имаме, че $I \models D_1$, следователно има литерал $L_1 \in D_1 : I \models L_1$ и $L_1 \neq L$ и ето защо $L_1 \in D_1 \setminus \{L\} \subseteq D$, така $L_1 \in D$ и $I \models L_1$, откъдето $I \models D$

И в двата случая получихме $I \models D$. □

17.3 Теорема за резолютивната изводимост

:

Теорема за коректност: Нека S е множество от съждителни дизюнкти. Ако $S \vdash^r \blacksquare$, то S е неизпълнимо.

Доказателство: Нека $S \vdash^r \blacksquare$. Нека $D_1, \dots, D_n$ е резолютивен извод от S и $D_n = \blacksquare$. Да допуснем, че S е изпълнимо. Нека I е булева оценка, която е модел за S , т.е. за всеки дизюнкт $D \in S$ имаме $I \models D$

С индукция по $1 \leq k \leq n$ ще докажем, че $I \models D_k$

- $k = 1$. Тогава D_1 е със сигурност елемент на S (не е резолвента на предишни два) и $I \models D_1$

- Нека $1 \leq k < n$ и за всички $1 \leq m \leq k$ имаме, че $I \models D_m$
- Разглеждаме D_{k+1} :
 - Ако $D_{k+1} \in S$, то $I \models D_{k+1}$
 - Ако D_{k+1} е резолвента на $D_{j_1}, D_{j_2}, 1 \leq j_1 \leq j_2 \leq k$, то съгласно И.Х. $I \models D_{j_1}, I \models D_{j_2}$ и от лемата $I \models D_{k+1}$

Така за всяко $k : 1 \leq k \leq n$ имаме $I \models D_k$, но $D_n = \blacksquare$ и значи $I \models \blacksquare \not\models$

□

Теорема за пълнота: Ако S е неизпълнимо множество от съждителни дизюнкти, то $S \vdash^r \blacksquare$

18 Бази от данни. Релационен модел на данните.

18.1 Релационен модел на данните

Релационният модел се основава на математическото понятие релация. Всяка релация е множество от елементи, които се състоят от n компонента и се наричат n -торки. Чрез една релация се моделира даден клас от обекти, а всяка n -торка от релацията представлява конкретен обект от този клас.

Атрибути: Служат за имена на колони в таблицата.

Схема на релация: Образува се от името на релацията и множеството от нейните атрибути. Означава се с името на релацията, последвано от списък с атрибути, ограден в скоби. Пример:

`Movies(title, year, length)`

Схема на релационна база от данни: Съвкупността от схемите на всички релации в дизайна на една релационна база данни.

Кортежи: Наредена n -торка от релацията (конкретен ред с данни в таблицата). Пример:

`(Star Wars, 1977, 124)`

Домейн: Всеки атрибут от схемата на релацията има свой домейн, който дефинира множеството от допустимите му атомарни стойности. Елементите на всеки кортеж трябва да имат стойности, които принадлежат на домейна на съответния атрибут. Пример:

`title - string, year - integer, length - integer`

Релация: Двумерна таблица, представляваща множество от уникални кортежи. В контекста на БД релацията е таблицата, в която се съхраняват данните.

База от данни: Колекция от взаимосвързани релации.

Реализация на релационната база от данни: Процесът преминава през концептуално, логическо и физическо проектиране, при което:

- Определят се данните и обектите, които ще се съхраняват.
- Определят се връзките между обектите.
- Определят се първичните/външните ключове и свързващите колони.
- Налагат се ограничения за цялостност (integrity constraints) върху обектите и връзките.
- Премахва се излишната информация чрез процеса на нормализация.
- Базата от данни се реализира физически чрез конкретна Система за управление на бази данни (СУБД).

Видове операции върху релационната база от данни

За извършване на операции върху схемата и данните се използва езикът SQL, който се разделя на две основни групи команди:

1. **DDL (Data Definition Language)** - синтаксис за описание, създаване или промяна на схемата на базата от данни. Основните операции са: 'CREATE', 'ALTER', 'DROP'.

- С CREATE създаваме нови таблици:

```
CREATE TABLE Students (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

- С ALTER модифицираме съществуваща структура:

```
ALTER TABLE Students ADD Email VARCHAR(100);
```

- С DROP изтриваме изцяло таблица от базата:

```
DROP TABLE Students;
```

2. **DML (Data Manipulation Language)** - синтаксис за работа с данните (записите) в таблиците.

- **SELECT** – използва се за извличане и заявки към данни.

```
SELECT * FROM Students WHERE Age > 18;
```

- **INSERT** – използва се за добавяне на нов запис.

```
INSERT INTO Students (ID, Name, Age) VALUES (1, 'Ivan', 20);
```

- **UPDATE** – използва се за промяна на съществуващи данни.

```
UPDATE Students SET Age = 21 WHERE ID = 1;
```

- **DELETE** – използва се за изтриване на записи.

```
DELETE FROM Students WHERE ID = 1;
```

Заявки към релационната база от данни: Теоретично заявките биват *процедурни* (потребителят указва какво иска и как да се вземе - напр. Релационна алгебра) и *декларативни* (указва се само какво се търси, а СУБД решава как - напр. SQL).

18.2 Релационна алгебра: основни операции

Релационната алгебра е формална система, чиито операнди са релации, а операторите извършват действия за извличане на данни.

Обединение: Бинарна, комутативна и асоциативна операция: $R \cup S$

- Прилага се върху релации със съвместими схеми (еднакъв брой и тип на съответните атрибути).
- Резултатът съдържа всички кортежи, които се съдържат в R , в S или и в двете, като дубликатите се премахват.

Разлика: Бинарна операция (не е комутативна): $R - S$

- Прилага се върху релации със съвместими схеми.
- Резултатът съдържа кортежите, които са в R , но не представени в S .

Декартово произведение: Бинарна, комутативна и асоциативна операция: $R \times S$

- Резултатът е нова релация, съдържаща всички възможни конкатенации (комбинации) на всеки кортеж от R с всеки кортеж от S .
- Ако R и S имат еднакви имена на атрибут A , в резултата те се преименуват на $R.A$ и $S.A$.

Проекция: Унарна операция: $\pi_{\langle attr_list \rangle}(R)$

- Извършва "вертикално" изрязване на таблицата.
- Резултатът съдържа само колоните (атрибутите), посочени в списъка $\langle attr_list \rangle$, като повтарящите се кортежи се премахват автоматично.

Селекция: Унарна операция: $\sigma_{\langle predicate \rangle}(R)$

- Извършва "хоризонтално" филтриране на таблицата.
- Резултатът съдържа само кортежите от R , които удовлетворяват логическото условие (предиката). Две последователни селекции могат да си сменят местата (комутират): $\sigma_{C1}(\sigma_{C2}(R)) = \sigma_{C2}(\sigma_{C1}(R))$.

18.3 Релационна алгебра: допълнителни операции

Допълнителните операции могат да се изразят чрез комбинация от основните операции.

Сечение: Бинарна, комутативна и асоциативна операция: $R \cap S$

- Прилага се над съвместими релации. Връща само общите за двете релации кортежи.
- Може да се изрази чрез разлика: $R \cap S = R - (R - S)$.

Частно: Бинарна операция: $R \div S$ (или $R : S$)

- **НЕ** е комутативна и **НЕ** е асоциативна операция.

- Използва се за заявки, включващи квантор за всеобщност (напр. "всички"). Резултатът съдържа кортежите от R (по атрибутите, липсващи в S), които са комбинирани с **всеки един** кортеж от релацията S .

Съединение: Бинарна операция: $R \bowtie_C S$

- Представлява декартово произведение, последвано от селекция по дадено условие C .
- Еквивалентно на: $\sigma_C(R \times S)$.

Естествено съединение: Бинарна, комутативна и асоциативна операция: $R \Join S$

- Свързва кортежите от двете релации въз основа на съвпадение на стойностите във всички атрибути с еднакви имена.
- За разлика от декартовото произведение и обикновеното съединение, автоматично премахва дублиращите се колони (оставя само едно копие от общите атрибути).

19 Бази от данни. Нормални форми.

19.1 Нормални форми. Проектиране схемите на релационните бази от данни.

Проектирането на схемите цели правилно дефиниране на същностите и техните свойства. За формален анализ и преобразуване на релационните схеми се използва процесът на нормализация.

Нормализация: Процес на преобразуване на релационни схеми чрез налагане на ограничения върху тях, с цел елиминиране на излишък на данни и деструктивни аномалии.

19.2 Аномалии ограничения, ключове.

Аномалия от излишество: Повтаряне на едни и същи данни (напр. адрес на библиотека) за всеки свързан с тях запис (напр. книга).

(Биб1, София, БазиДанни, 5)

(Биб1, София, Програмиране, 3) -- 'София' се повтаря излишно

Аномалия при обновяване: Необходимост от промяна на множество редове при актуализация на едно-единствено свойство. Ако адресът на 'Биб1' се промени, той трябва да се редактира на всички съответни редове в таблицата, което крие риск от противоречивост.

Аномалия при добавяне: Невъзможност за въвеждане на информация за даден обект, преди за него да са дефинирани стойности на други ключови атрибути.

-- Не можем да запишем нова библиотека и нейния адрес,
-- ако тя все още няма заведени книги (ако Книга е част от ключа).

Аномалия при изтриване: Загуба на важна и съществуваща независима информация при изтриване на записи. Ако изтрием единствената книга в дадена библиотека, ще загубим и информацията за самия номер и адрес на тази библиотека.

Ключ: Минимално множество от атрибути K , което определя еднозначно всеки кортеж в релацията. Удовлетворява условията:

1. За всеки два кортежа $r_1, r_2 \in R \Rightarrow r_1[K] \neq r_2[K]$.
2. Минималност – никое негово строго подмножество не притежава първото свойство.

Първични и потенциални ключове: Една релация може да има няколко потенциални ключа. Избраният от разработчика единствен ключ се нарича **първичен ключ**.

Потенциални ключове: {FacultyNumber}, {EGN}

Първичен ключ: FacultyNumber

Външен ключ (K^*): Атрибут или списък от атрибути в релация R , който не е ключ за нея, но се явява първичен ключ в друга релация Q . Служи за реализиране на връзки.

Students(FN, Name, GroupID*) -> Groups(GroupID, Description)

Суперключ: Множество от атрибути, което съдържа в себе си ключ и определя еднозначно кортежите, но без изискване за минималност.

{FacultyNumber, Name, Age} -- Суперключ, съдържащ ключа FacultyNumber

Ограничения за цялостност: Правила за стойностите в колоните, служещи за налагане на семантична коректност:

- PRIMARY KEY / UNIQUE: Забранява дублирането на стойности.
- NOT NULL: Не допуска липса на стойност (стойност null). Компонентите на първичния ключ задължително са NOT NULL.
- CHECK: Изисква стойностите да отговарят на определено логическо условие. Пример: CHECK (Age >= 18).
- FOREIGN KEY: Изисква всяка стойност да съществува като ключ в реферираната релация.

19.3 Функционални зависимости, аксиоми на армстронг.

Функционална зависимост (ФЗ): Означава се като $X \rightarrow Y$. За всеки два кортежа t_1, t_2 в релацията, ако съвпадат по атрибутите X , задължително съвпадат и по атрибутите Y ($t_1.X = t_2.X \Rightarrow t_1.Y = t_2.Y$).

FacultyNumber -> StudentName, EGN -> DateOfBirth

Тривиална ФЗ: Зависимост $X \rightarrow B$, при която дясната страна е подмножество на лявата страна ($B \subseteq X$). При нетривиалните ФЗ поне един атрибут от десния състав не е в левия.

Тривиална: {FacultyNumber, Name} -> Name

Нетривиална: FacultyNumber -> Name

Аксиоми на Армстронг: Правила за извеждане на нови зависимости от дадено множество F :

- **A1. Рефлексивност:** Ако $Y \subseteq X$, то $X \rightarrow Y$. (напр. FN, Name -> FN)
- **A2. Разширение (Попълнение):** Ако $X \rightarrow Y$, то $XW \rightarrow YW$. (напр. от FN -> Name следва FN, Age -> Name, Age)
- **A3. Тразитивност:** Ако $X \rightarrow Y$ и $Y \rightarrow Z$, то $X \rightarrow Z$. (напр. от FN -> GroupID и GroupID -> SpecName следва FN -> SpecName)

Основни следствия:

- **Обединение:** Ако $X \rightarrow Y$ и $X \rightarrow Z$, то $X \rightarrow YZ$.
- **Декомпозиция:** Ако $X \rightarrow Y$ и $Z \subseteq Y$, то $X \rightarrow Z$.
- **Псевдотранзитивност:** Ако $X \rightarrow Y$ и $WY \rightarrow Z$, то $XW \rightarrow Z$.

19.4 Първа, втора, трета нормална форма. Нормална форма на Бойс-Код.

Първични и непървични атрибути: Първичен атрибут е всеки, който влиза в състава на поне един потенциален ключ. Всички останали са непървични.

Първа нормална форма (1НФ): Схемата е в 1НФ, ако домейните на всички нейни атрибути съдържат само атомарни (неделими) стойности и няма повтарящи се кортежи.

Не е в 1НФ: Phones = '0888123456, 0899111222' (неатомарна стойност)

В 1НФ: Стойностите се разделят в отделни кортежи (редове).

Втора нормална форма (2НФ): Схемата е в 2НФ, ако е в 1НФ и всеки непървичен атрибут е в пълна функционална зависимост от ключа (не зависи от негово подмножество).

Ключ: {StudentID, CourseID}. Зависимост: CourseID -> CourseName.

Нарушава 2НФ: CourseName зависи частично само от част от ключа (CourseID).

Решение: Декомпозиция на две релации: (StudentID, CourseID) и (CourseID, CourseName).

Трета нормална форма (3НФ): Схемата е в 3НФ, ако е в 1НФ и никой непървичен атрибут не зависи транзитивно от ключ. *Алтернативна дефиниция:* За всяка нетривиална ФЗ $X \rightarrow Y$ е изпълнено: X е суперключ или Y е първичен атрибут.

Ключ: StudentID. Зависимости: StudentID -> GroupID, GroupID -> Faculty.

Нарушава 3НФ: StudentID -> Faculty е транзитивна зависимост.

Решение: Декомпозиция на (StudentID, GroupID) и (GroupID, Faculty).

Нормална форма на Бойс-Код (BCNF): Релацията е в BCNF тогава и само тогава, когато за всяка нетривиална функционална зависимост $X \rightarrow Y$, лявата страна X е суперключ. BCNF е по-строга от 3НФ (не допуска изключението " Y е първичен атрибут").

Ключ: {Student, Subject}. Зависимости: {Student, Subject} -> Teacher, Teacher -> Subject.

Нарушава BCNF: Teacher -> Subject, но Teacher не е суперключ.

19.5 Многозначни зависимости; аксиоми на функционалните и многозначните зависимости; съединение без загуба.

Многозначна зависимост (МЗ / MVD): Означава се с $X \twoheadrightarrow Y$. Утвърждава, че за дадени фиксирани стойности на X , съответните стойности на Y са напълно независими от стойностите на останалите атрибути C в релацията.

Схема: (Teacher, Subject, Textbook)

МЗ: Teacher ->> Subject | Teacher ->> Textbook

Ако учител преподава математика и физика и ползва учебник А и учебник Б, в таблицата трябва да присъстват всички възможни комбинации (общо 4 реда).

Тривиална МЗ: Зависимост $X \rightarrow\rightarrow Y$, при която $Y \subseteq X$ или $X \cup Y$ съдържа всички атрибути на релационната схема.

Аксиоми за функционални и многозначните зависимости:

- **Преобразуване на ФЗ в МЗ:** Ако $X \rightarrow Y$, то $X \rightarrow\rightarrow Y$.
- **Допълнение:** Ако $X \rightarrow\rightarrow Y$ е в сила за R , то $X \rightarrow\rightarrow (R \setminus (X \cup Y))$.

Съединение без загуба (Lossless Join): Свойство при декомпозиция на релация R на подрелации R_1 и R_2 , гарантиращо, че естественото им съединение дава точно изходната релация ($R = R_1 \bowtie R_2$). Изпълнено е, ако общите атрибути са суперключ за поне една от декомпозираните схеми ($R_1 \cap R_2 \rightarrow R_1$ или $R_1 \cap R_2 \rightarrow R_2$).

19.6 Четвърта нормална форма.

Четвърта нормална форма (4НФ): Релационната схема е в 4НФ тогава и само тогава, когато за всяка нетривиална многозначна зависимост $X \rightarrow\rightarrow Y$, лявата ѝ страна X е суперключ за релацията.

Правило за декомпозиция до 4НФ: Ако нетривиална МЗ $X \rightarrow\rightarrow Y$ нарушава 4НФ, схемата се заменя с две нови схеми: $R_1(X \cup Y)$ и $R_2(X \cup (R \setminus (X \cup Y)))$.

Схема: (Teacher, Subject, Textbook) с МЗ Teacher $\rightarrow\rightarrow$ Subject.

Декомпозиция до 4НФ: R1(Teacher, Subject) и R2(Teacher, Textbook).

20 Изкуствен интелект: Пространство на състоянията - определение, характеристики на състоянията, търсене в пространство на състояния.

Състояние: Представяне на задачата в процеса на нейното решаване. Те могат да бъдат начални, междинни или крайни (цели).

Действие: Възможните действия, които един агент може да извърши, когато е в дадено състояние.

Пространство на състоянията: Множеството от всички състояния, които са достижими от началното чрез произволна поредица от действия.

Пространството от състоянията може да се представи чрез граф, където възлите са състоянията, а ребрата действията.

Търсене в пространствата на състоянията: Когато се намираме в състояние s намираме всички непосетени, достижими състояния (фронт) и избираме в кое състояние да преминем (зависи от алгоритъма, който използваме).

20.1 Локално търсещи алгоритми.

Локално търсещи алгоритми: Търсещ алгоритъм, който използва само един връх (вместо да разглежда множество от върхове) и на всяко състояние разглежда единствено своите съседи. Обикновено не запазват пътя от началното до целевото състояние.

Имат две преимущества над глобално търсещите алгоритми:

- Използват константна памет
- Могат да често да стигнат до близки до оптималното решения в големи или безкрайни пространства на състоянията.

Hill-climbing (Изкачване на хълмове): Алгоритъмът изкачване на хълмове е алчен локално търсещ алгоритъм, следи текущо състояние и на всяка итерация преминава към съседното такова с най-висока стойност на целевата функция. (Насочва се в посоката с най-стръмното изкачване).

Алгоритъмът спира, когато няма съсед с по-висока стойност. Hill-climbing може да "заседне" ако е в състояние, което е:

- Локален максимум
- Плато: област, където всички съседи имат една и съща стойност. Платото може да, както да е плосък локален максимум, така и "рамо плоска област, непосредствено след, която да има възможност за изкачване.

Проблемът с платото донякъде може да се реши като позволим "движение на страни": движим се в една посока с надеждата, че се намираме в плато, което е рамо. Ако платото е локален максимум алгоритъмът ще зацikli, освен ако няма лимит за движенията на страни.

Проблемът с локалния максимум може да се реши чрез "случайно рестартиране": пускаме алгоритъма "Изкачване на хълмове" няколко пъти от случайно избрано начално състояние.

Simulated annealing (Симулирано втвърдяване): Идеята на симулираното втвърдяване подобна на изкачването на хълмове, но вместо най-добрия съсед се избира случаен такъв.

Ако избраният съсед ни приближава до целта, то той се избира. В противен случай, алгоритъмът избира ход с някаква вероятност, която намалява експоненциално спрямо колко е "лош" този ход.

Името на алгоритъма идва от металургията, защото използва променливата температура - в началото започваме с висока температура T , която позволява да се разглеждат "лоши" ходове (равновероятно да се избере кой да е ход). В продължение на алгоритъма температурата T намалява, което понижава вероятността да се разглеждат "лоши" ходове. Температурата се регулира от scheduler и ако се намали достатъчно бавно, алгоритъмът ще намери глобалния оптимум с вероятност клоняща към 1.

Локално търсене в лъч: Идеята на локалното търсене в лъч е да пазим k състояния, вместо само едно.

Започваме с k случайно генерирани начални състояния. На всяка стъпка генерираме всички наследници на всички k състояния. Ако някое от тях е целево, алгоритъмът спира. В противен случай избираме k -те най-добри наследници измежду всички и повтаряме предишните стъпки.

Важно е да се отбележи, че локалното търсене в лъч не е същото като да направим k случайни рестартирания на изкачване на хълмове. Причината за това е, че всички наследници на текущите състояния (лъчите) се групират, след което се избират k -те най-добри (например с приоритетна опашка). Това означава, че някой начален лъч може да е изцяло пренебрегнат от алгоритъма, което ни води до проблема на локалното търсене в лъч.

Алгоритъмът може да страда от ниско ниво на разнообразия на k -те състояния, т.е. те могат да са от един малък регион от пространството на състоянията. В този случай разширяването не носи много информация. Решения предлага стохастичното локално търсене в лъч, при което наследниците се избират с вероятност пропорционална на стойностите им.

Генетичен алгоритъм: Вариант на стохастичното търсене в лъч, в което състоянията наследници се създават чрез комбиниране на две текущи състояния, вместо да се модифицира текущо.

Като търсенето в лъч, генетичните алгоритми започват с k случайно генерирани състояния, наречени **популация**. Всяко състояние или **индивид** се представя като низ над крайна азбука (най-често $\{0, 1\}$). Тези низове наричаме хромозоми.

Всяко състояние се оценява от оценяваща (fitness) функция, която ни дава колко близо до целта е съответното състояние. Следващата стъпка е селекция, където се избира два индивида за кръстосване. Селекцията избира два индивида, като вероятностното разпределение на популацията се определя от резултатите дадени от оценяващата функция - по-"добрите" индивида са по-вероятно да бъдат избрани.

Кръстосването на два индивида става чрез избиране на индекс i от хромозома. Едното дете взема хромозомите от родител A до индекс i включително и хромозомите от родител B от индекс $i + 1$ нататък. Другото дете прави обратното. Има различни видове кръстосвания, които са модификация на написаното като се избират повече индекси или се разглеждат цикли в родителите.

Накрая се прилага **мутация**. Идеята е, че както в природата искаме да поддържаме генетично разнообразие от едно поколение до следващото. Мутацията разглежда всеки индивид и с малка вероятност променя някой ген в хромозома му.

20.2 Задачи за удовлетворяване на ограничения: определение, търсене с възврат, намаляване на конфликтите.

Задача за удовлетворяване на ограниченията: Състои се от три компонента:

- Множество от променливи $\{X_1, \dots, X_n\}$
- Множество от дефиниционни области $\{D_1, \dots, D_n\}$, по една за всяка променлива.
- Множество от ограничения, които задават допустими комбинации от стойности за променливите.

Задачата за удовлетворяване на ограниченията (ЗУО) е решена, когато всяка променлива има стойност от своята дефиниционна област, която удовлетворява всяко ограничение за тази променлива.

Всяко ограничение C_j се състои от двойка $\langle \text{обхват}, \text{релация} \rangle$, където *обхват* е наредена k -орка от променливите, които участват в ограничението, *релация* е релацията, която определя стойностите, които тези променливи могат да приемат.

Пример:

$$\langle (X_1, X_2), \{(3, 1), (3, 2), (2, 1)\} \rangle$$

Състояния: Множества от променливи, на които са присвоени конкретни стойности.

Присвояване: Всяко състояние в ЗУО се дефинира от присвояване на стойност от променлива на някоя или всички променливи, $\{X_i = v_i, \dots\}$. От едно състояние отиваме в друго, като присвоим нова стойност на променлива от текущото или добавим нова променлива.

Присвояване, което не нарушава нито едно ограничение ще наричаме **консистентно**.

Пълно присвояване ще наричаме присвояване, след което всяка променлива има стойност.

Оттук решение на ЗУО е пълно, консистентно присвояване. Оттук, задачата се свежда до търсене на целево състояние в пространство от състояния. Ще разгледаме два възможни начина: търсене с възврат (backtracking) и намаляване на конфликтите.

Решаване на ЗУО чрез търсене с възврат: Прилагаме търсене в дълбочина, като на всяка стъпка присвояваме стойност на една променлива. Целта ни е през цялото време да имаме консистентно присвояване.

Ако при добавянето на нова стойност нарушим консистентността, то връщаме назад в дървото на търсенето и разглеждаме друга стойност за родителя (търсенето спира за този клон).

Тъй като присвояванията са комутативни (редът на извършването им не е от значение), то дървото на търсене ще се разширява всяка стъпка с най-много $d := \max(D_i)$ върха и ще имаме най-много d^n листа.

Алгоритъмът може да се подобри като се разгледат следните идеи:

- Ред на разглеждане на променливите и техните стойности: Първо се разглеждат променливите чиито ограничения са с най-малко възможни стойности и с най-много недобавени съседи в k -орката.
- Преразглеждане на ограниченията: Когато се присвои нова променлива с определена стойност $X_i = v_i$, да се вземат само ограниченията, за които $X_i = v_i$.
- Интелигентен възврат: вместо да се върнем една стъпка назад до родителя и да пробваме с нова стойност, алгоритъма да върне назад до променлива, която би оправила конфликта.

Алгоритъм min-conflicts: Търсенето с възврат обхваща цялото пространство от състояния. Локално-търсещите алгоритми са ефикасен начин за решаване на ЗУО.

Целевата функция е *брой конфликти* и тъй като локално търсещите алгоритми работят с пълни състояния инициализираме с едно пълно присвояване и операторът за получаване на ново състояние е промяна на стойност на променлива.

На всяка стъпка локално-търсещията алгоритъм цели да минимизира целевата функция, т.е. да минимизира броя конфликти.

20.3 Избор на следващ ход при игра за двама играчи: мини-макс процедура, алфа-бета отсичане.

Да разгледаме игра с нулева сума, в която участват двама играчи MIN и MAX, разполагащи с перфектна информация - всеки играч знае предните стъпки и само един може да спечели.

Нека първи да започне MAX, след което двамата играчи ще се редуват, като играят оптимално. minimax процедурата дава полезността MAX да бъде в дадена позиция.

В крайно състояние стойността на minimax е стойността, която ще се получи като резултат. На всяка стъпка MAX иска да се премести в състояние с максимална стойност, а MIN в такова с минимална.

$$MINIMAX(s) = \begin{cases} UTILITY(s, MAX), & \text{ако } s \text{ е крайно} \\ \max_{s' \text{ се получава от } s} MINIMAX(s'), & \text{ако MAX е на ход и } s \text{ не е крайно} \\ \min_{s' \text{ се получава от } s} MINIMAX(s'), & \text{ако MIN е на ход и } s \text{ не е крайно} \end{cases}$$

Процедурата MINIMAX е пълен и оптимален (при игра срещу оптимален опонент), чиято времева сложност е $O(b^m)$ и сложност по памет $O(bm)$. Времовата сложност я прави неприложима за игри като шах.

Алфа-бета отсичане: Възможно е да пресметнем minimax стойността на една игра без да разглеждаме всяка една стойност по дървото на ходовете. Такъв алгоритъм е алфа-бета отсичането:

```
function ALPHA-BETA-SEARCH(state) returns an action
  v ← MAX-VALUE(state, -∞, +∞)
  return the action in ACTIONS(state) with value v
```

```
function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
```

```
  v ← -∞
  for each a in ACTIONS(state) do
    v ← max(v, MIN-VALUE(RESULT(s, a), α, β))
    if v ≥ β then return v
    α ← max(α, v)
  return v
```

```

function MIN-VALUE( $state, \alpha, \beta$ ) returns a utility value
  if TERMINAL-TEST( $state$ ) then return UTILITY( $state$ )

```

```

   $v \leftarrow +\infty$ 

```

```

  for each  $a$  in ACTIONS( $state$ ) do

```

```

     $v \leftarrow \min(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 

```

```

    if  $v \leq \alpha$  then return  $v$ 

```

```

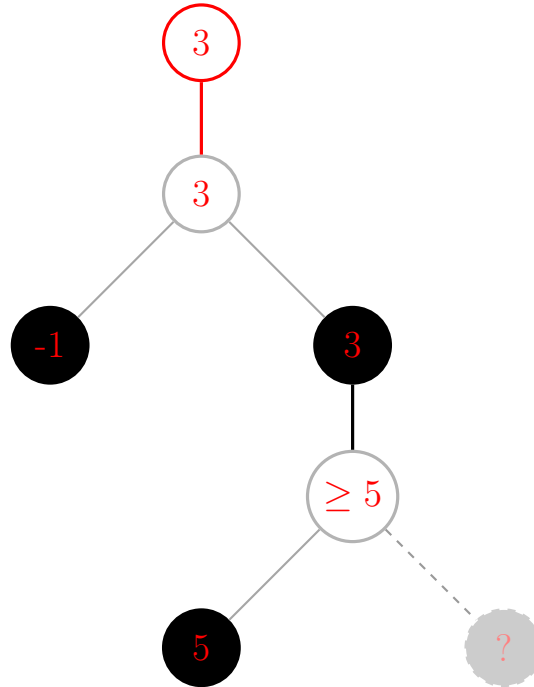
     $\beta \leftarrow \min(\beta, v)$ 

```

```

  return  $v$ 

```



21 Съвременни софтуерни технологии.

21.1 Софтуерен продукт

Софтуерният продукт представлява не просто самият изпълним код, а цялостно решение, което включва също така изчерпателна документация, конфигурационни файлове, данни за настройка на средата и ръководства за потребителя. Той е резултат от инженерна дейност, която трансформира изискванията на клиента в работеща система. За разлика от физическите продукти, софтуерът не се износва, но бързо остарява поради промени в средата на експлоатация и нуждите на потребителите. Неговото качество се определя не само от коректното изпълнение на функциите, но и от редица атрибути като надеждност, сигурност, поддръжка и лесна за употреба.

21.2 Софтуерен процес

Софтуерният процес представлява структуриран и организиран набор от дейности, които са необходими за разработването на софтуерна система при спазване на зададени срокове и постигане на високо качество на резултата. Този процес включва не само техническите дейности по създаване на кода, но и управленски практики, които координират ресурсите, хората и времето. Добре дефинираният софтуерен процес осигурява повтораемост, предвидимост и е основа за непрекъснато усъвършенстване на организацията.

21.3 Основни фази на софтуерните процеси

Независимо от конкретната методология, всеки софтуерен процес включва няколко основни фази, които често се изпълняват циклично или итеративно. Първата фаза е комуникация, която цели да установи разбирателство между клиента и разработчика относно поставените цели и изисквания. Следва фазата на планиране, където се определят ресурсите, сроковете и се оценяват рисковете. Моделирането включва създаване на абстрактни представи на системата – както от гледна точка на проблемната област (анализ), така и от гледна точка на софтуерното решение (проектиране). Конструирването обхваща писането на код и изпълнението на модулни тестове. Накрая, внедряването е процесът на предаване на крайния продукт на клиента, включващ инсталация, обучение и поддръжка.

21.4 Модел на софтуерен процес

Моделът на софтуерен процес представлява опростено, абстрактно описание на един реален софтуерен процес, което го представя от определена перспектива. Той служи като шаблон, който улавя същността на процеса, без да навлиза в подробности, които са специфични за всеки отделен проект. Основните цели на използването на модели на процеси са няколко. Първо, те формират общо разбиране сред всички участници в разработката относно дейностите, нужните ресурси и съществуващите ограничения. Второ, те помагат за намиране на несъответствия, излишества и пропуски в планирания процес още преди той да бъде изпълнен. Трето, те позволяват на екипа да намери и оцени подходящите дейности, които водят до постигане на целите. И накрая, моделът служи като основа за адаптиране на един общ корпоративен процес към специфичните изисквания на конкретна ситуация или проект.

21.5 Модел на водопада

Моделът на водопада се характеризира с ясно разграничени и линейно подредени фази, което го прави лесен за разбиране и прилагане. Основният му принцип е, че всяка дейност трябва да бъде напълно завършена и документирана, преди екипът да премине към следващата. Този модел дефинира строго входовете и изходите на всяка дейност, както и интерфейсите между отделните стъпки, а ролите на разработчиците са ясно разпределени и фиксирани. Прилагането на модела е най-подходящо в случаите, когато изискванията са добре осъзнати и могат да бъдат ясно формулирани още в началото на проекта. Той се използва успешно при проекти, които са ясно организирани, както и при повторяеми или много големи проекти, за които времето и бюджетът не са критични ограничения. Основните проблеми на водопада произтичат от неговата липса на гъвкавост. На практика е изключително трудно за потребителя да формулира всичките си изисквания в началото, а строгото разделение на етапи затруднява реакцията при промени. Освен това този модел е произлязъл от областта на хардуерното инженерство и не отчита напълно същността на софтуера като творчески процес, изискващ чести итерации и връщане назад за преработване на решения.

21.6 Модел на бързата разработка

Моделът на бързата разработка (RAD) представлява гъвкав и итеративен подход, който поставя акцент върху бързото създаване на прототипи и честата обратна връзка от клиента. За разлика от линейния водопад, RAD позволява на разработчиците да се адаптират към промените по време на изграждането на

системата. Основните предимства на този модел са подобрената гъвкавост и възможността за създаване на кратки итерации, което съкращава цялостното време за разработка. Тъй като интеграцията може да започне в началните етапи и кодът да се използва повторно във всеки момент, времето за тестване значително намалява. От гледна точка на потребителите, качеството е по-високо, тъй като те взаимодействат с развиващите се прототипи и могат да насочват развитието на бизнес функционалността. Въпреки това, RAD има и съществени недостатъци. Той не е подходящ за много големи приложения, които изискват разделяне на множество модули и значителни човешки ресурси. Често в този модел липсва акцент върху важни качествени атрибути на софтуера като дългосрочна поддръжка и архитектурна цялост. Фокусът върху прототипите може да доведе до непрекъснати незначителни промени в отделни компоненти, като същевременно бъдат игнорирани проблеми на системната архитектура. Освен това RAD изисква интензивно участие на клиента на много етапи, което прави процеса по-сложен и ресурсоемък от други методологии.

21.7 Еволюционни модели

Еволюционните модели за разработка на софтуер се основават на принципа на постепенното изграждане на системата чрез последователни версии, които се усъвършенстват с времето. Основната разлика между еволюционния модел и модела RAD е, че RAD се фокусира върху изключително бързото създаване на прототипи с активно потребителско участие, докато еволюционният модел поставя акцент върху дългосрочното развитие и усъвършенстване на зрящата система. Сред предимствата на еволюционния подход е възможността клиентът да започне да използва системата много преди целият продукт да бъде завършен. Функционалностите с най-висок приоритет са тествани най-много, което гарантира, че критичните части на системата са стабилни. Тъй като в разработването на ранните версии участват по-малко хора, екипът може да бъде разширяван постепенно, в зависимост от получената обратна връзка. Освен това итерациите, които изискват използването на нова или непозната технология, могат да бъдат планирани за по-късен етап, когато екипът вече е натрупал опит. Проблемите при еволюционните модели са свързани основно с комуникацията и управлението на обхвата. Необходимостта от активно участие на клиентите през целия проект може да доведе до закъснения, ако те не са винаги на разположение. Уменията за комуникация и координация придобиват особено голямо значение за успеха на проекта. Често неформалните заявки за подобрения след завършването на всяка стъпка могат да доведат до объркване в екипа. В най-лошия случай този модел може да доведе до така наречения „score-creep“ – бавно, постепенно и неконтролируемо разширяване на първоначалния обхват на приложението.

21.8 Прототипен модел

Прототипният модел е подход, при който основната цел е бързото създаване на работещ, но съкратен вариант на системата, който да изясни изискванията на клиента. Това може да включва създаване на основните потребителски интерфейси без значително кодиране на бизнес логиката, разработване на съкратена версия, която изпълнява ограничено подмножество от функции, или използване на съществуващи компоненти, които демонстрират бъдещите възможности. Прототипният модел се прилага най-често в проекти, където изискванията на потребителите не са достатъчно ясни или дизайнът на системата е твърде сложен, за да бъде предвиден предварително. Основните проблеми на този модел са свързани с управлението на очакванията и риска от лош дизайн. Прототипирането може да използва значителни ресурси, а крайният резултат да не успее да удовлетвори очакванията на клиента. Съществува сериозна опасност самият прототип да стане част от крайния продукт, което често води до лошо проектирана, трудна за поддръжка система, защото прототипът обикновено не е изграден с оглед на архитектурната цялост. Поради тези причини, прототипният модел не е подходящ за използване при разработване на софтуерни системи, където проблемът вече е добре разбран и интерфейсът е ясен и прост.

21.9 Спираловиден модел

Спираловидният модел е еволюционен модел на софтуерен процес, който съчетава принципите на прототипния модел и модела на водопада, но добавя като основен движещ фактор анализа на риска. За разлика от линейните модели, спираловидният модел следва итеративен (цикличен) подход, при който всяка итерация (завъртане на спиралата) преминава през четири квадранта: определяне на целите и алтернативите, оценка на рисковете, разработка и валидация, както и планиране на следващата итерация. Характерна особеност на модела е наличието на множество точки на прогреса, наречени „anchor point milestones“, които се преглеждат в края на всяка итерация. Тези точки служат за оценка дали проектът трябва да продължи, да бъде прекратен или да бъдат преразгледани неговите цели. Спираловидният модел е най-подходящ за големи, скъпи и сложни проекти, където рискът от неуспех е висок и където изискванията са слабо разбрани в началото.

21.10 Сравнителна характеристика на моделите на софтуерни процеси

Всеки от разгледаните модели на софтуерни процеси има своите силни и слаби страни, което ги прави подходящи за различни типове проекти. Водопадният модел е най-подходящ за проекти със стабилни, добре разбрани изисквания и ниска степен на несигурност, но се проваля при чести промени. Прототипният модел и моделът RAD са идеални за проекти с висока степен на неяснота в изискванията, където е необходима бърза обратна връзка от клиента, но крият риск от архитектурна неефективност. Еволюционните модели предлагат баланс между гъвкавост и структура, като са подходящи за дългосрочни проекти, които могат да бъдат пуснати на пазара на етапи. Спираловидният модел предоставя най-добрата защита срещу рискове, но е най-сложен за управление и изисква висока експертиза. Изборът на модел не е еднократно решение – често съвременните организации комбинират елементи от различни модели (хибридни подходи) и ги адаптират към специфичните нужди на всеки проект, като вземат предвид фактори като размер на екипа, критичност на системата, бюджет и време.

21.11 Гъвкави методи - Extreme Programming

Extreme Programming (XP) е методология за разработка на софтуер, която се базира на дванадесет основни практики, целящи постигането на бързо, поетапно разработване и лесно адаптиране към променящите се изисквания. Една от ключовите практики е поетапното планиране чрез потребителски истории (user stories), като вместо подробен общ план, изискванията за всяка версия се определят заедно с клиента и се описват кратко. Методологията насърчава пускането на малки версии, като първо се разработи минималната полезна функционалност, а след това системата се пуска често с постепенно добавяне на нови функции. XP въвежда и практиката на разработка, водена от тестове (Test-Driven Development), при която тестовете се пишат преди самия код, което помага да се уточни поведението на програмата и гарантира непрекъснато тестване. Непрекъснатата интеграция изисква след завършването на всяка задача кодът да бъде интегриран в общата система, като всички автоматични тестове трябва да преминат успешно преди приемането на новата версия. Принципът на опростен дизайн повелява да се създава само необходимият дизайн за текущите изисквания, без да се добавя излишна сложност за бъдещи, все още неизвестни нужди. Две от най-характерните практики са програмирането по двойки (pair programming), където двама програмисти работят заедно върху един и същи код, като взаимно се проверяват и подпомагат, както и колективната собственост на кода, която позволява на всеки разработчик да променя всяка част от кода, като целият екип носи отговорност за цялостната система. Накрая, XP изисква клиентът да бъде физически на място в екипа, за да участва активно в уточняването на изискванията и приоритетите.

21.12 Гъвкави методи - SCRUM

Scrum е гъвкава методология, която предоставя рамка за организация и планиране на проекта, като за разлика от XP, тя не налага конкретни технически практики, а се фокусира върху процесите на управление. Основната мотивация за възникването на Scrum е нуждата на мениджърите от информация за разходите, времето за разработка и момента, в който продуктът ще бъде готов за пазара. Докато при планово-ориентираната разработка тази информация се осигурява чрез дългосрочни планове, които често се променят, Scrum разчита на краткосрочни цикли и непрекъсната обратна връзка. Основните понятия в Scrum включват: продукт (софтуерът, който се разработва); собственик на продукта (член на екипа, който определя функционалностите и изискванията и участва в тестването); продуктов беклог (подреден списък със задачи, функции и грешки, които предстоят да бъдат изпълнени); и самоорганизиращ се екип за разработка от 5 до 8 души. Работата се организира в кратки периоди, наречени спринти (обикновено 2–4 седмици), през които екипът създава нова версия или подобрение. В рамките на спринта, всеки ден се провежда кратка среща (Daily Scrum) за преглед на напредъка и планиране на работата за деня. Scrum Master е човекът, който подпомага екипа и следи за правилното прилагане на Scrum процеса. Резултатът от всеки спринт е потенциално готов за доставка инкремент (Potentially Shippable Product Increment) – версия с достатъчно високо качество за използване от клиента. За да измерва своята производителност, екипът използва метриката скорост (velocity), която представлява оценка на количеството работа, което може да завърши в рамките на един спринт.

21.13 Изисквания към софтуерните системи

Изискванията представляват спецификации на това, което трябва да бъде разработено в софтуерния проект. Те описват начина на поведение на системата при взаимодействие с потребителя, нейните системни характеристики или атрибути, както и ограниченията върху процеса на разработване и самия продукт. Чрез изискванията се дефинира разбирането за функционалностите на системата, обосновава се необходимостта от системата и стойността на нейната разработка. Важен принцип е, че изискванията определят какво трябва да прави дадена система, но не и как да го прави, оставяйки техническата реализация на дизайна. Изискванията служат като база за планиране на проекта, управление на рисковете и тестването. Те също така определят какви отстъпки могат да бъдат направени при договаряне и разработка, както и как да се

управляват промените. Тъй като изискванията са зависими от гледните точки на различните участници в процеса на разработване (клиенти, потребители, разработчици, тестващи), управлението на техните често противоречащи си интереси е една от най-сложните задачи в софтуерното инженерство.

21.14 Функционални и нефункционални изисквания

Функционалните изисквания описват конкретните функции, услуги или поведения, които системата трябва да предоставя. Когато се представят като потребителски изисквания, те обикновено са описания на високо ниво на детайлност за това какво трябва да прави системата. Обаче, когато станат системни изисквания, те трябва да бъдат детайлизирани, като се специфицират нужната входна информация и получаваната изходна информация за всяка функция. Конкретното съдържание на функционалните изисквания зависи от типа на софтуера, потенциалните потребители и вида на системата, в която ще бъде използван. От друга страна, нефункционалните изисквания определят системни характеристики и ограничения, които не са свързани с конкретна функция, а с цялостното качество на софтуера – например надеждност, време за отговор, сигурност, поддръжка и преносимост. Нефункционалните изисквания могат да се отнасят и до процеса на разработка, като например използване на конкретен софтуерен инструмент, програмен език, методология или параметри на разработване. Често нефункционалните изисквания се оказват по-критични от функционалните, защото от тях зависи пригодността на цялата система за реална експлоатация. Например една система, която изпълнява всички функции коректно, но реагира твърде бавно или често спира да работи, е практически неизползваема.

21.15 Анализ и проектиране на софтуерните изисквания

Анализът и проектирането на софтуерните изисквания представляват ключови дейности в ранните фази на софтуерния проект. Те включват проверка на извлечените изисквания дали правилно, точно, пълно и атомарно описват нуждите на клиента. Атомарността означава, че всяко изискване трябва да бъде формулирано така, че да не може да бъде разделено на по-малки, смислени части. Следващата стъпка е оценка на взаимодействието на изискванията, тъй като често едно изискване може да противоречи на друго или да дублира неговата функционалност. Най-важната част от този процес е приоритизирането на изискванията и разрешаването на конфликтите между тях. Приоритизирането обикновено се извършва съвместно с клиента, като се вземат предвид бизнес стойността, техническият риск и зависимостите между отделните изисквания. Разрешаването на конфликти може да включва преговори, компромиси или търсене на алтернативни технически решения, които да задоволят противоречащите си изисквания едновременно.

21.16 Езици за описание. UML

Обектно-ориентираният анализ описва проблемната област чрез обекти, които включват както реални обекти от околния свят, с които системата взаимодейства, така и кандидат-обекти, които се въвеждат специално за търсене на софтуерно решение. Всеки обект в този контекст се описва чрез своя клас (като шаблон за създаване на обекти), атрибути (данни, които съхранява) и операции или методи (действия, които може да изпълнява). Обектно-ориентираният дизайн, за разлика от анализа, определя софтуерното решение чрез по-конкретни елементи като компоненти, интерфейси, обекти, класове, атрибути и операции. Процесът на проектиране обикновено започва с обектите, определени при анализа, след което те се допълват, прецизират или променят до получаване на окончателен, реалистичен дизайн. UML (Unified Modeling Language) е стандартен език за обектно-ориентирано моделиране, който се използва за специфициране, визуализиране, конструиране и документиране на софтуерни системи. Неговите основни елементи включват моделни елементи (които дефинират основните концепции и семантика), нотация (графично представяне на моделите) и насоки (правила и добри практики за използване). Основните цели на UML са да предостави лесен за използване визуален език, който позволява обмен и разбиране на модели между разработчиците. Той осигурява механизми за разширяване и специализация, поддържа добавяне на нови концепции и нотации при необходимост и може да бъде адаптиран към различни приложни области. UML е независим от конкретен програмен език и процес на разработка, има формална основа за разбиране на моделите и подпомага развитието на инструменти за обектно моделиране. Той поддържа високо ниво на абстракция като компоненти, колаборации, рамки (frameworks) и шаблони (patterns), като обединява добрите практики от методологиите Booch, OMT и OOSE.

21.17 Верификация и валидация на софтуера

Софтуерното тестване е основният механизъм за верификация и валидация, който представлява процес на изпълнение на програмата с данни, симулиращи потребителски вход. По време на тестването се наблюдава поведението на програмата, за да се провери дали то съответства на очакваното. Един тест е успешен (pass), когато действителният резултат съвпада с очаквания, и неуспешен (fail), когато се наблюдава различие.

Важен принцип е, че ако дадена програма работи правилно за определен набор от входни данни, можем да предположим, че ще работи коректно и за подобни входни данни. Функционалното тестване проверява функционалността на цялата система с основни цели: откриване на възможно най-много грешки и доказване, че системата изпълнява своето предназначение. Потребителското тестване има за цел да провери дали софтуерът е полезен и лесен за използване от крайните потребители, като се фокусира върху това дали функциите действително подпомагат потребителите в работата им и дали те могат лесно да откриват и използват тези функции. Тестването на производителност и натоварване проверява скоростта и поведението на системата при различни нива на натоварване, като целите са постигане на приемливо време за реакция и обработка, нормална работа при очакваното натоварване и плавно мащабиране при увеличаване на броя потребители или заявки. Накрая, тестването на сигурността проверява защитата и надеждността на системата по отношение на защита на потребителските данни, предотвратяване на кражба или повреда на информация и запазване на цялостта на системата.

21.18 Управление на качеството

Управлението на качеството има за цел да гарантира, че софтуерният продукт отговаря на изискваното ниво на качество, като тази дейност се осъществява на две основни нива. На организационно ниво, управлението на качеството включва създаването на стандартизирани процеси и стандарти, които водят до предвидимо разработване на качествен софтуер. На проектно ниво, то се изразява в прилагането на конкретни процеси за качество и в проверка дали тези процеси се спазват стриктно. За всеки проект се изготвя план за качество, който определя конкретните цели за качество на проекта, както и процесите и стандартите, които ще бъдат използвани за тяхното постигане. Основните дейности по управление на качеството включват осигуряване на независима проверка на процеса по разработка. Екипът по качеството проверява резултатите от проекта (deliverables), за да гарантира тяхното съответствие с организационните стандарти и цели. За да бъде тази проверка ефективна, екипът по качеството трябва да бъде независим от екипа по разработка. Тази независимост позволява обективна оценка на качеството, тъй като разработчиците често са склонни да пренебрегват или минимизират проблемите в собствената си работа, докато независимият екип може да докладва безпристрастно за установените несъответствия.

22 Архитектури на софтуерни системи.

22.1 Софтуерна архитектура.

Софтуерна архитектура: Структурата или структурите на една система, включващи нейните софтуерни елементи, техните външно видими свойства и взаимоотношенията между тях.

Архитектурата е също така съвкупността от ключови проектни решения, които трудно могат да бъдат променени на по-късен етап, заедно с обосновката (мотивите) за тяхното вземане.

Структури и изгледи на архитектурата: Изглед е представяне на системата от конкретна гледна точка или перспектива.

Гледната точка (Viewpoint) определя правилата, нотациите и конвенциите за създаване на даден изглед.

Заинтересованите страни (Stakeholders) имат различни интереси и изисквания към системата.

Архитектурата представлява съвкупност от изгледи, а не една единствена голяма диаграма.

Пример за изглед е 4+1 моделът, който съдържа:

- Логически изглед, описващ функционалността на системата и ключовите абстракции.
- Процесен изглед, описващ паралелното изпълнение на процеси и нишки.
- Физически изглед, описващ разполагането (deployment) на системата върху хардуерната инфраструктура.
- Изглед на разработката, описващ организацията на компонентите и модулите в системата.
- Сценарии (+1): Представят конкретни случаи на употреба и взаимодействия в системата. Използват се за проверка и валидиране на съгласуваността между останалите архитектурни изгледи.

Необходимост от софтуерни архитектури: Софтуерната архитектура определя дългосрочните качества на системата, като мащабируемост, адаптивност, устойчивост и други. Тя свързва бизнес целите с техническите решения. Лошата архитектура води до негъвкавост, крехкост на системата и натрупване на технически дълг. Архитектурата се развива непрекъснато — тя не е еднократна фаза от проектирането.

Влияние на архитектурата върху проекта и организацията: Така нареченият закон на Конуей:

„Организациите проектират системи, които отразяват техните комуникационни структури.“

Структурата на екипите и начинът, по който те комуникират, оказват влияние върху модулността и архитектурата на системата. Пример за това е как Spotify въвеждат моделът с автономни екипи (squads), след което архитектурният им стил става микрослуги.

Изводът е, че структурата на екипите и начинът, по който те комуникират, оказват влияние върху модулността и архитектурата на системата.

22.2 Изисквания към качеството.

При проектиране на софтуерна архитектура как можем да разберем дали архитектурата е достатъчно добра?

Това е ролята на качествените атрибути.

Качествени атрибути: нефункционалните свойства, които определят колко добре се представя една система

Изисквания: Разделят се на две групи:

- Функционални изисквания: казват *какво* трябва да прави софтуерната система.
- Нефункционални изисквания: казват *как* софтуерната система трябва да работи.

Функционалните изисквания са ориентирани към крайния потребител. Нефункционалните са ориентирани към техническата част от екипа - хората, които разработват, тестват и поддържат системата. Познати са също като *качествени изисквания*.

Качествените атрибути могат да бъдат групирани в три основни категории:

- Технологични качества: надеждност, модифицируемост, производителност, сигурност, тестваемост и използваемост.

Те оказват пряко влияние върху поведението на системата и потребителското изживяване.

- Бизнес качества: време за излизане на пазара, разходи за разработка и възвръщаемост на инвестициите. Те показват доколко архитектурата подпомага организационните и стратегическите цели на бизнеса.
- Архитектурни качества: концептуална цялост, последователност и модулност. Тези качества са присъщи на самата архитектура и косвено влияят върху всички останали качества на системата.

22.3 Архитектурни стилове.

Архитектурният стил определя семейство от системи, които споделят обща структурна организация.

Той конкретизира:

- Типовете компоненти в системата;
- Отговорностите на тези компоненти;
- Моделите на взаимодействие и комуникация между тях.

С други думи, архитектурният стил предоставя набор от принципи и правила за организиране на софтуерната система, като определя как отделните части са структурирани и как си взаимодействат.

Многослоен стил: Системата е организирана в слоеве, като всеки слой предоставя услуги на слоя над него.

- Всеки слой зависи единствено от непосредствено по-долния слой.
- Използва се в операционни системи, корпоративен софтуер и многослойни приложения.

Предимства

- **Модулност и изолация** – отделните слоеве могат да се развиват независимо един от друг.
- **Преизползваемост** – общи услуги (например достъп до данни) могат да се използват от множество приложения.
- **Лесна поддръжка** – промените обикновено са ограничени в рамките на конкретен слой.
- **Ясна структура** – подпомага абстракцията и разделянето на отговорностите.

Недостатъци

- **Допълнителни разходи за производителност** – данните може да преминават през ненужни слоеве.
- **Негъвкавост** – стриктното разделяне на слоеве понякога е изкуствено наложено.
- **Трудно управление на напречни функционалности** – аспекти като логване, сигурност и мониторинг са по-трудни за интегриране.
- **Изтичане на зависимости** – по-високите слоеве могат да заобикалят предвидените абстракции и да достъпват по-ниски слоеве директно.

Модел-изглед-контролер (MVC)

MVC е архитектурен модел, който разделя приложението на три основни компонента:

- **Model (Модел)** – съдържа данните на приложението и бизнес състоянието.
- **View (Изглед)** – визуалното представяне на данните и потребителския интерфейс.
- **Controller (Контролер)** – обработва входните действия на потребителя и актуализира модела.

Предимства

- **Разделяне на отговорностите** – позволява паралелна работа на различни екипи.
- **Гъвкав потребителски интерфейс** – интерфейсът може да бъде променян без засягане на бизнес логиката.
- **Множество изгледи върху един и същ модел** – например табличен и графичен изглед на едни и същи данни.
- **Широко разпространен модел** – поддържа се от множество рамки за разработка (frameworks).
- **Подобрена тестваемост** – логиката в модела е отделена от потребителския интерфейс.

Недостатъци

- **Контролерът често се превръща в „God Object“** – натрупва твърде много отговорности.
- **Все още съществува свързаност между View и Model** – не е напълно елиминирана зависимостта между тях.
- **Не е идеален за интерфейси със сложно управление на състоянието** – например настолни и мобилни приложения.
- **Различните рамки интерпретират MVC по различен начин** – няма единна и строго дефинирана реализация.

Поточни (Pipe-and-Filter)

Pipe-and-Filter е архитектурен стил, при който обработката на данни се извършва чрез последователност от независими компоненти, наречени филтри.

- Компонентите, наречени **филтри (filters)**, обработват потоци от данни.
- Данните преминават през **тръби (pipes)**, които свързват филтрите последователно.
- Всеки филтър е независим и не споделя състояние с останалите.
- Използва се при компилаторни конвейери, обработка на данни и мултимедийно стрийминг съдържание.

Предимства

- **Висока преизползваемост** – филтрите могат лесно да бъдат комбинирани по различни начини.
- **Лесна модификация** – филтрите могат да бъдат заменяни или пренареждани без промени в останалите компоненти.
- **Паралелизъм** – различните филтри могат да обработват данни едновременно.
- **Простота** – ясният и линеен поток от данни улеснява разбирането и анализа на системата.

Недостатъци

- **Ограничена интерактивност** – подходящ е предимно за пакетна или потокова обработка на данни.
- **Сложна обработка на грешки** – управлението на грешки между отделните тръби може да бъде трудно.
- **Допълнителни разходи при трансформация на данни** – данните често се преобразуват между отделните филтри.
- **Неподходящ за системи със състояние** – не е добър избор за процеси, управлявани от състояние или сложна контролна логика.

Service-Oriented Architecture (SOA)

SOA е архитектурен стил, базиран на независими услуги, които комуникират през мрежа.

- Услугите предоставят функционалност чрез стандартизирани договори (contracts).
- Фокусът е върху интеграцията на хетерогенни корпоративни системи.
- Проектиран е за преизползваемост, интероперативност и корпоративни работни процеси.

Предимства

- **Слаба свързаност (loose coupling)** между корпоративните системи.
- **Стандартизирана комуникация** между компонентите.
- **Насърчава преизползването** на функционалности чрез услуги.
- **Подходящ за големи организации** със съществуващи (legacy) системи.
- **Поддържа оркестрация** на сложни бизнес процеси.

Недостатъци

- **ESB (Enterprise Service Bus)** лесно се превръща в тясно място или единична точка на отказ.
- **Тежка администрация и стандартизация** – често включва XML и сложни протоколи.
- **Разгръщането може да остане монолитно** – споделени бази данни и инфраструктура между услугите.

- Не е оптимизирана за cloud-native мащабиране.

22.4 Проектиране на софтуерна архитектура.

Архитектура и процесът ADD (Attribute-Driven Design)

Архитектурата не може да разчита единствено на интуиция.

- Необходими са систематични, повторяеми и обясними решения.
- Изискванията за атрибути на качество силно влияят върху архитектурата.
- Повечето системни провали са архитектурни провали, а не грешки в кода.

Кога се използва ADD?

- При наличие на множество заинтересовани страни с противоречиви цели.
- В ранните етапи на проекта (фаза на дефиниране на архитектурата).
- Когато атрибутите на качество (QAs) са с висок приоритет.
- При проектиране на нова система.
- При значително рефакториране или преработка на съществуваща система.

Стъпка 1: Потвърждаване и приоритизиране на драйверите

- Идентифициране на:
 - заинтересовани страни;
 - функционални изисквания;
 - атрибути на качество;
 - ограничения.
- Приоритизиране на архитектурно значимите изисквания (ASRs).

Резултат: приоритизиран списък от ASRs.

Стъпка 2: Избор на високоннивов архитектурен модел

Решението се базира на:

- атрибути на качество;
- ограничения;
- предметната област.

Типични избори:

- слоеве (Layers);
- тръби и филтри (Pipes-and-Filters);
- клиент-сървър (Client-Server);
- микросървиси (Microservices);
- събитийно-ориентирани системи (Event-driven);
- cloud-native архитектури.

Стъпка 3: Прецизиране на модулната декомпозиция

- Разделяне на системата на модули.
- Разпределяне на отговорности.
- Съобразяване с качествените изисквания.
- Идентифициране на напречни (cross-cutting) аспекти като логване, автентикация и мониторинг.

Стъпка 4: Избор на архитектурни тактики

Тактиките са решения, които задават как системата трябва да се справя със стимул от някой качествен атрибут.

- **Надеждност (availability)** – повторни опити, отказоустойчивост, репликация.
- **Производителност (performance)** – кеширане, предварителни изчисления.
- **Модифицируемост (modifiability)** – плъгини, абстракционни слоеве.
- **Сигурност (security)** – автентикация, валидация, криптиране.

Стъпка 5: Дефиниране на интерфейси и взаимодействия

- Дефиниране на интерфейсите между модулите.
- Описание на обмена на данни.
- Определяне на комуникационните модели.
- Документиране чрез диаграми (например C4 модел).

Стъпка 6: Проверка и усъвършенстване

- Идентифициране на рискове.
- Документиране на компромиси (trade-offs).
- Проверка на удовлетворяване на сценарии.
- Записване на решенията и тяхната обосновка.

Стъпка 7: Итеративно прилагане

- ADD е итеративен процес.
- Всеки подсистема преминава през същите стъпки.
- Архитектурата се „изгражда“ постепенно, на слоеве.

22.5 Документиране на софтуерна архитектура.

Документирането на софтуерна архитектура е ключава дейност в жизнения цикъл на всеки проект. То има за цел да опише архитектурата по такъв начин, че различните участници в проекта могат да я разберат, използват, изграждат и поддържат.

Основните предназначения на документацията са:

- Описание на системата.
- Средства за обучение.
- Средство за комуникация между заинтересовани страни.
- Основа за анализ и изграждане на системата.
- Основа за анализ на инциденти.

Основни елементи в документацията на архитектурата:

1. Въведение
2. Архитектурни драйвери (функционални и качествени изисквания, бизнес цели и контекст)
3. Обхват на системата
4. Обща стратегия на решението
5. Основни градивни блокове
6. Поведение по време на изпълнение
7. Разполагане
8. Напречни (cross-cutting) концепции
9. Архитектурни решения
10. Качествени сценарии
11. Рискове
12. Терминологичен речник

23 Симетрични оператори в крайномерни евклидови пространства. Основни свойства. Теорема за диагонализация.

23.1 Симетричен оператор

Нека V е крайномерно евклидово пространство.

Симетричен оператор: Линейният оператор $\varphi : V \rightarrow V$ е симетричен, ако $\langle \varphi(u), v \rangle = \langle u, \varphi(v) \rangle$ за всеки $u, v \in V$

23.2 Матрица на симетричен оператор

Матрицата на симетричен оператор е симетрична: Нека $e_1, \dots, e_n$ е ортонормиран базис на V , т.е.

$$\langle e_i, e_j \rangle = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

Доказателство: Нека φ е симетричен оператор.

$$\begin{aligned} \varphi(e_i) &= a_{1i}e_1 + a_{2i}e_2 + \dots + a_{ni}e_n \\ a_{ij} &= \langle e_i, \varphi(e_j) \rangle = \langle \varphi(e_i), e_j \rangle = a_{ji} \Rightarrow A = A^T \end{aligned}$$

□

Лема: Нека $A \in M_{n \times n}(\mathbb{R})$, $u, v \in \mathbb{R}$ (вектор стълбове). Изпълнено е равенството:

$$\langle Au, v \rangle = \langle u, A^T v \rangle$$

Доказателство: $A = (a_{ij})_{n \times n}, u = \begin{pmatrix} u_1 \\ \vdots \\ u_n \end{pmatrix}, v = \begin{pmatrix} v_1 \\ \vdots \\ v_n \end{pmatrix}$

$$Au = \begin{pmatrix} a_{11}u_1 + a_{12}u_2 + \dots + a_{1n}u_n \\ \dots \\ a_{n1}u_1 + a_{n2}u_2 + \dots + a_{nn}u_n \end{pmatrix}$$

Имаме, че:

$$\langle Au, v \rangle = \sum_{i=1}^n \sum_{j=1}^n a_{ij} u_j v_i$$

От друга страна:

$$A^T v = \begin{pmatrix} a_{11}v_1 + a_{21}v_2 + \dots + a_{n1}v_n \\ \dots \\ a_{1n}v_1 + a_{2n}v_2 + \dots + a_{nn}v_n \end{pmatrix}$$
$$\langle u, A^T v \rangle = \sum_{i=1}^n \sum_{j=1}^n a_{ij} u_j v_i$$

23.3 Характеристични корени на симетричен оператор

Нека φ е симетричен оператор. Тогава φ има само реални характеристични корени.

Доказателство: Нека $e_1, \dots, e_n$ е ортонормиран базис на V . Матрицата на φ спрямо него е симетрична. Искаме да докажем, че характеристичният полином $f_A(x) = \det(A - xE)$ е с реални коефициенти.

Да допуснем, че $\exists z \in \mathbb{C} : z \neq 0, \quad f_A(z) = 0$

Тогава $\exists v \in \mathbb{C}^n$ вектор-стълб : $v \neq 0$ и $(A - zE)v = 0$, т.е. $Av = zv$

От $Av = zv$ имаме, че техните комплексно-спрегнати са равни: $\bar{A}\bar{v} = \bar{z}\bar{v}$

От предната лема имаме:

$$\langle Av, \bar{v} \rangle = \langle v, A^T \bar{v} \rangle \stackrel{\text{симетрична}}{=} \langle v, A \bar{v} \rangle$$

Имаме, че:

$$\begin{aligned}\langle Av, \bar{v} \rangle &= \langle zv, \bar{v} \rangle = z \langle v, \bar{v} \rangle = z \|v\|^2 \\ \langle v, A\bar{v} \rangle &= \langle v, \bar{z}\bar{v} \rangle = \bar{z} \langle v, \bar{v} \rangle = \bar{z} \|v\|^2\end{aligned}$$

Но $v \neq 0 \Rightarrow \|v\|^2 > 0 \Rightarrow z = \bar{z} \Rightarrow z \in \mathbb{R}$

□

23.4 Собствени вектори

Тв: Всеки два собствени вектора, съответстващи на различни стойности, са ортогонални помежду си.

Доказателство: Нека φ е симетричен оператор и нека λ, μ са два негови собствени вектора, съответстващи на различни стойности, т.е. $\varphi(v) = \lambda v, \varphi(u) = \mu u, \lambda \neq \mu$

$$\lambda \langle u, v \rangle = \langle u, \lambda v \rangle = \langle u, \varphi(v) \rangle = \langle \varphi(u), v \rangle = \langle \mu u, v \rangle = \mu \langle u, v \rangle$$

Имаме, че $\lambda \neq \mu$ и че $\lambda \langle u, v \rangle = \mu \langle u, v \rangle$. Оттук:

$$\underbrace{(\lambda - \mu)}_{\neq 0} \langle u, v \rangle = 0 \Rightarrow \langle u, v \rangle = 0$$

□

Деф: Подпространството $U < V$ наричаме φ -инвариантно, ако $\forall u \in U \rightarrow \varphi(u) \in U$

Деф: Ортогонално допълнение на $U < V$ наричаме:

$$U^\perp = \{v \in V \mid \langle u, v \rangle = 0 \quad \forall u \in U\}$$

Лема: Нека $\varphi : V \rightarrow V$ е симетричен оператор, $U < V$ е φ -инвариантно. Тогава $U^\perp$ също е φ -инвариантно.

Доказателство: Нека $v \in U^\perp$, т.е. $\forall u \in U \quad \langle v, u \rangle = 0$

Искаме $\varphi(v) \in U^\perp$. За целта трябва $\langle \varphi(v), u \rangle = 0$ за всяко $u \in U$.

$$\langle \varphi(v), u \rangle = \underbrace{\langle v, \varphi(u) \rangle}_{\substack{\in U^\perp \\ \in U}} = 0$$

Теорема: Съществува ортонормиран базис $g_1, \dots, g_n$ на пространството, в който матрицата на симетричен оператор е диагонална.

Доказателство: Нека $\dim V = n$. Индукция по $\dim V$

База: $V = l(g)$ ако $|g| = 1 \Rightarrow \varphi(g) \in l(g) \Rightarrow \varphi(g) = \lambda g$

И.С: Нека твърдението е вярно за всички пространства с размерност $< n$. Спрямо някакъв ортономриран базис матрицата на φ е симетрична. Знаем, че $\exists \lambda_1 \in \mathbb{R}$ - реален корен на φ . Оттук $\exists g_1$ собствен вектор: $\varphi(g_1) = \lambda_1 g_1$ и нека $|g_1| = 1$

Твърдим, че $l(g_1)$ е φ -инвариантна. Наистина:

$$\varphi(\alpha g_1) = \alpha \varphi(g_1) = \alpha \lambda g_1 \in l(g_1)$$

Тогава $(l(g_1))^\perp$ също е φ -инвариантна. $\dim((l(g_1))^\perp) = \dim V - 1$

Имаме, че $\varphi_{U^\perp}$ е симетричен оператор и от индукционното допускане съществува ортонормиран базис на $U^\perp$. Нека този базис е $g_2, \dots, g_n$. Имаме, че $g_1 \perp g_i, i = 2, \dots, n$ и $|g_1| = 1$. Следователно $g_1, g_2, \dots, g_n$ ортономиран базис за $U \oplus U^\perp = V$.

Накрая, понеже всеки g_i е собствен вектор $\varphi(g_i) = \lambda_i g_i$, матрицата на φ в базиса $(g_1, \dots, g_n)$ е диагонална със собствените стойности по диагонала и нули навсякъде другаде.

24 Симетрична и алтернативна група. Теорема на Кейли. Теорема за хомоморфизмите на групи.

24.1 Симетрична група S_n - представяне на елементите като произведение на независими цикли.

Симетрична група: Нека $n \in \mathbb{N}$. Симетрична група на n елемента наричаме:

$$S_n = \{f : \{1, \dots, n\} \rightarrow \{1, \dots, n\} \mid f \text{ е биекция} \}$$

Цикъл: $\sigma \in S_n$ наричаме цикъл ако:

$$\sigma(i_1) = i_2, \sigma(i_2) = i_3, \dots, \sigma(i_{k-1}) = i_k, \sigma(i_k) = i_1$$

и $\sigma(i_j) = i_j, \quad j \notin \{1, \dots, k\}$

Бележим с: $\sigma = (i_1, \dots, i_k)$

Твърдение: Всяка пермутация е произведение на цикли.

Доказателство: Нека i_1 е произволно число от $\{1, \dots, n\}$. Разглеждаме числата: $i_1, \sigma(i_1) = i_2, \sigma(i_2) = i_3, \dots, \sigma(i_{k+1}) = i_k$, където k е най-голямото естествено число, за което тези числа са различни. Тогава $\sigma(i_k)$ е някое от тях.

Твърдим, че $\sigma(i_k) = i_1$. При $k = 1$ е очевидно.

Да разгледаме $k > 1$. Да допуснем, че $\sigma(i_k) = i_2$. Имаме, че $i_k \neq i_1$, но $\sigma(i_k) = \sigma(i_1) \neq$

Следователно $\sigma(i_k) = i_1$ и нека означим $\sigma_1 = (i_1 \dots i_k)$.

Нека разгледаме j_1 (ако съществува такова), което не участва в σ_1 . По аналогичен начин построяваме цикъл $\sigma_2 = (j_1, \dots, j_s)$.

Циклите σ_1, σ_2 са независими (в противен случай бихме получили две числа, които имат равни образи под действието на σ). Продължаваме по този начин, докато изчерпим всички числа от $\{1, \dots, n\}$. Очевидно σ е произведение на получените независими цикли.

24.2 Спрягане на елементите на S_n

Твърдение: Нека $\sigma = (i_1 \dots i_k), \rho \in S_n$. Тогава:

$$\rho \sigma \rho^{-1} = (\rho(i_1), \rho(i_2), \dots, \rho(i_k))$$

Доказателство: $\rho \sigma \rho^{-1}(\rho(i_s)) = \rho(\sigma(i_s)) = \rho(i_{s+1}), s = 1, \dots, k-1$

Нека $j \notin \{\rho(i_1), \dots, \rho(i_k)\}$. Тогава $j \neq \rho(i_s) \Rightarrow \rho^{-1}(j) \neq i_s, s \in \{1, \dots, k-1\}$

За j имаме:

$$\rho \sigma \underbrace{\rho^{-1}(j)}_{\notin i_1, \dots, i_s} = \rho(\rho^{-1}(j)) = j$$

□

Твърдение: Нека $\pi = \sigma_1 \dots \sigma_l$ - произведение на независими цикли. Тогава

$$\rho \pi \rho^{-1} = (\rho \sigma_1 \rho^{-1}) (\rho \sigma_2 \rho^{-1}) \dots (\rho \sigma_l \rho^{-1})$$

Произведение на независими цикли със същите дължини като $\sigma_1, \dots, \sigma_l$

Доказателство:

$$\begin{aligned} \rho \pi \rho^{-1} &= \rho \sigma_1 \cdot \sigma_2 \dots \sigma_l \\ &= \rho \sigma_1 \rho^{-1} \dots \rho \sigma_2 \rho^{-1} \dots \rho \sigma_l \rho^{-1} \\ &= (\rho \sigma_1 \rho^{-1}) (\rho \sigma_2 \rho^{-1}) \dots (\rho \sigma_l \rho^{-1}) \\ &\stackrel{\text{предното тв}}{=} (\rho(i_{1_1}) \dots \rho(i_{1_{k_1}})) \dots (\rho(i_{l_1}) \dots \rho(i_{l_{k_l}})) \end{aligned}$$

Където $\sigma_j = (i_{j_1}, \dots, i_{j_{k_j}})$

24.3 Транспозиции и представяне на елементите като транспозиции

Транспозиция: наричаме цикъл с дължина 2. (Такъв, който разменя два елемента).

Твърдение: Всяка пермутация може да се представи като произведение на транспозиции, т.е. S_n се поражда от всички транспозиции.

Доказателство: Знаем, че всяка пермутация $\sigma \in S_n$ може да се представи като произведение на независими цикли. За един цикъл $\sigma_1 = (i_1 \dots i_k)$ можем лесно да проверим, че:

$$(i_1 \dots i_k) = (i_1 i_k)(i_1 i_{k-1}) \dots (i_1 i_3)(i_1 i_2)$$

24.4 Алтернативна група

Алтернативна група:

$$A_n = \{\pi \in S_n \mid \pi \text{ е произведение на четен брой транспозиции}\}$$

Забележка: $|A_n| = \frac{n!}{2}$

Хомоморфизъм, ядро, образ: Нека G, K са групи.

$\varphi : G \rightarrow K$ е хомоморфизъм, ако $\varphi(a \cdot_G b) = \varphi(a) \cdot_K \varphi(b)$

- Ядро на φ е $\text{Ker} \varphi = \{g \in G \mid \varphi(g) = e_K\} \subseteq G$
- Образ на φ е $\text{Im} \varphi = \{\varphi(g) \mid g \in G\} \subseteq K$
- φ е изоморфизъм, ако е биекция.

24.5 Теорема на Кейли: всяка крайна група е изоморфна на подгрупа на S_n

Теорема: Нека G е група от ред n .

Тогава $\exists K < S_n : |K| = n$ и $G \cong K$ (G и K са изоморфни)

Доказателство: Нека $a \in G, L_a : G \rightarrow G, L_a(g) = ag$

Твърдим, че L_a е биекция. Проверка:

- Инекция:

$$L_a(g) = L_a(h) \Rightarrow ag = ah \Rightarrow g = h \Rightarrow L_a \text{ е инекция.}$$

- Сюрекция: Нека $h \in G$:

$$L_a(a^{-1}h) = aa^{-1}h = h \Rightarrow L_a \text{ е сюрекция.}$$

За всяко $a \in G, L_a$ е биекция от G в G , следователно $L_a \in S_n$. Нека $K = \{L_a \mid a \in G\}$ Проверяваме дали $K < S_n$. K е подгрупа на S_n ако е затворена относно нейната операция и $a \in K \iff a^{-1} \in K$

- Затвореност:

$$(L_a \cdot L_b)(g) = L_a(L_b(g)) = L_a(bg) = abg = L_{ab}(g) \Rightarrow L_a \cdot L_b \in K$$

- Обратен елемент:

$$L_a \cdot L_{a^{-1}} = L_{aa^{-1}} = L^e = L_{a^{-1}} \cdot L_a \Rightarrow (L_a)^{-1} = L_{a^{-1}} \in K$$

Имаме, че $K < S_n$

Искаме за $a \neq b \Rightarrow L_a \neq L_b$

Наистина $L_a(e) = a \neq b = L_b(e)$, откъдето получаваме, че $|K| = n$

Имаме, че $\varphi : G \rightarrow K, \varphi(a) = L_a$ е биекция.

$\varphi(a \cdot b) = L_{ab} = L_a \cdot L_b = \varphi(a) \cdot \varphi(b)$ - хомоморфизъм.

Следователно φ е изоморфизъм, откъдето $G \cong K$

□

24.6 Теорема за хомоморфизмите при групи

Нормална подгрупа: Нека G е група, $H < G, g \in G$. Дефинираме:

- $gH = \{gh \mid h \in H\}$ - ляв съседен клас на G
- $Hg = \{hg \mid h \in H\}$ - десен съседен клас на G

H е нормална подгрупа на G — $H \triangleleft G$, ако $\forall g \in G \Rightarrow gH = Hg$

Факторгрупа: Нека $H \triangleleft G$. Факторгрупа наричаме множеството:

$$G/H = \{aH \mid a \in G\}$$

Имаме, че $aH \cdot bH = abHH = abH$

Теорема: Нека $\varphi : G \rightarrow K$ е хомоморфизъм. Тогава $\text{Ker}\varphi \triangleleft G$ и $G/\text{Ker}\varphi \cong \text{Im}\varphi$

Доказателство: $\text{Ker}\varphi \stackrel{?}{<} G$. Нека $a, b \in \text{Ker}\varphi$, т.е. $\varphi(a) = \varphi(b) = e_K$.

- $\varphi(a \cdot b) = \varphi(a) \cdot \varphi(b) = e_K \cdot e_K = e_K \Rightarrow ab \in \text{Ker}\varphi$
- $\varphi(a^{-1}) = (\varphi(a))^{-1} = e_K^{-1} = e_K \Rightarrow a^{-1} \in \text{Ker}\varphi$

Следователно $\text{Ker}\varphi < G$. За да видим дали $\text{Ker}\varphi$ е нормална подгрупа на G , проверяваме дали $ghg^{-1} \in \text{Ker}\varphi$, за $h \in \text{Ker}\varphi, g \in G$

$$\varphi(ghg^{-1}) = \varphi(g) \underbrace{\varphi(h)}_{e_K} \varphi(g^{-1}) = \varphi(g)\varphi(g^{-1}) = \varphi(gg^{-1}) = \varphi(e_G) = e_K$$

$$G/\text{Ker}\varphi = \{g\text{Ker}\varphi \mid g \in G\}$$

$$g\text{Ker}\varphi = \{gh \mid h \in \text{Ker}\varphi\}$$

$$\text{Нека } \Theta : G/\text{Ker}\varphi \rightarrow \text{Im}\varphi, \Theta(g\text{Ker}\varphi) = \varphi(g)$$

Коректност на Θ : Ако $g_1\text{Ker}\varphi = g_2\text{Ker}\varphi$ имаме, че:

$$\{g_1h \mid h \in \text{Ker}\varphi\} = \{g_2h \mid h \in \text{Ker}\varphi\}$$

$$\text{Следователно ако } g_1 = g_2h, h \in \text{Ker}\varphi \Rightarrow \varphi(g_1) = \varphi(g_2)\varphi(h) = \varphi(g_2)$$

Оттук Θ е коректно дефинирана функция. Θ хомоморфизъм ли е?

$$\Theta(g_1g_2\text{Ker}\varphi) = \varphi(g_1g_2) = \varphi(g_1)\varphi(g_2) = \Theta(g_1\text{Ker}\varphi)\Theta(g_2\text{Ker}\varphi)$$

Θ е хомоморфизъм. Ясно е, че Θ е сюрекция.

Инективност: Разглеждаме $\Theta(g_1\text{Ker}\varphi) = \Theta(g_2\text{Ker}\varphi)$, откъдето $\varphi(g_1) = \varphi(g_2)$

$$\varphi(g_1g_2^{-1}) = \varphi(g_1)\varphi(g_2^{-1}) = \varphi(g_2)\varphi(g_2^{-1}) = e_K \Rightarrow g_1g_2^{-1} \in \text{Ker}\varphi$$

Оттук $g_1 = hg_2 \in \text{Ker}\varphi g_2 = g_2\text{Ker}\varphi$. Аналогично $g_2 \in g_1\text{Ker}\varphi$

Получаваме, че $g_1\text{Ker}\varphi = g_2\text{Ker}\varphi \Rightarrow \varphi$ е инекция, с което теоремата е доказана. □

25 Теорема на Ферма. Теорема за средните стойности (Рол, Лагранж, Коши). Формула на Тейлър.

25.1 Локален екстремум на функция на една променлива

Нека $f : D \rightarrow \mathbb{R}, D \subseteq \mathbb{R}$

Казваме, че x_0 е **точка на локален минимум** за $f(x)$, ако:

$$\exists \delta > 0 : (x_0 - \delta, x_0 + \delta) \subset D \text{ и } f(x_0) \leq f(x), x \in (x_0 - \delta, x_0 + \delta)$$

Казваме, че x_0 е **точка на строг локален минимум** за $f(x)$, ако

$$f(x_0) < f(x), x \in (x_0 - \delta, x_0 + \delta), x \neq x_0$$

25.2 Необходимо условие за локален екстремум за диференцируеми функции (теорема на Ферма)

Теорема: Ако x_0 е точка на локален екстремум за функцията $f(x)$ и $f(x)$ е диференцируема в x_0 , то $f'(x_0) = 0$

Бележка: Това означава, че допирателната към графиката на функция в нейна точка на локален екстремум е хоризантална.

Доказателство: Понеже $f(x)$ е диференцируема в т. x_0 , то съществува границата:

$$f'(x_0) := \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0}$$

Нека x_0 е точка на локален минимум за $f(x)$. Тогава $\exists \delta > 0$ такова, че интервалът $x_0 - \delta, x_0 + \delta$ се съдържа в дефиниционната област на функцията и

$$f(x_0) \leq f(x) \forall x \in (x_0 - \delta, x_0 + \delta)$$

Тогава за всяко $x \in (x_0 - \delta, x_0 + \delta), x \neq x_0$, имаме:

$$\frac{f(x) - f(x_0)}{x - x_0} \begin{cases} \leq 0, x \in (x_0 - \delta, x_0) \\ \geq 0, x \in (x_0, x_0 + \delta) \end{cases}$$

От тези неравенства следва съответно, че (*):

$$\lim_{x \rightarrow x_0 - 0} \frac{f(x) - f(x_0)}{x - x_0} \leq \text{ и } \lim_{x \rightarrow x_0 + 0} \frac{f(x) - f(x_0)}{x - x_0} \geq 0$$

Тъй като:

$$f'(x_0) = \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0} = \lim_{x \rightarrow x_0 - 0} \frac{f(x) - f(x_0)}{x - x_0} = \lim_{x \rightarrow x_0 + 0} \frac{f(x) - f(x_0)}{x - x_0} \geq 0$$

От (*) имаме, че $f'(x_0)$ е както ≤ 0 , така и ≥ 0 , следователно $f'(x_0) = 0$

Нека x_0 е точка на локален максимум за $f(x)$. Тогава x_0 е точка на локален минимум за $-f(x)$ и, според вече доказаното, $(-f)'(x_0) = 0$, то $-f'(x_0) = 0 \Rightarrow f'(x_0) = 0$. $\square$

25.3 Теорема на Рол за средните стойности

Нека $f(x)$ е непрекъсната в $[a, b]$ и диференцируема в (a, b) . Ако $f(a) = f(b)$, то $\exists c \in (a, b) : f'(c) = 0$

Доказателство: Щом $f(x)$ е непрекъсната в $[a, b]$, то от теоремата на Вайерщрас $f(x)$ има НГС и НМС.

Ако поне едната от тях се достига в т. $c \in (a, b)$, то тя непременно е точка на локален екстремум.

От теоремата на Ферма: $f'(c) = 0$.

В противен случай ако нито НГС, нито НМС на $f(x)$ не се достигат в точка от (a, b) , то това става в т. a или в b . Но $f(a) = f(b)$. Следователно НМС = НГС, следователно за $x \in (a, b)$

$$f(x) \equiv \text{const} \Rightarrow f'(x) = 0 \forall x \in (a, b)$$

25.4 Теорема на Лагранж за средните стойности

Нека $f(x)$ е непрекъсната в $[a, b]$ и диференцируема в (a, b) . Тогава $\exists c \in (a, b) : f(b) - f(a) = f'(c)(b - a)$

Доказателство: Ще сведем към теоремата на Рол. Въвеждаме функцията $h(x) := f(x) - kx, x \in [a, b]$, като k е такава, че $h(a) = h(b)$. Имаме:

$$h(a) = h(b) \text{ т.е. } f(a) - ka = f(b) - kb \iff k = \frac{f(b) - f(a)}{b - a}$$

Освен това очевидно, че щом $f(x)$ е непрекъсната в $[a, b]$ и диференцируема в (a, b) , то и $h(x)$ е такава.

Така $h(x)$ удовлетворява предположенията на теоремата на Рол. Прилагаме тази теорема към $h(x)$. Така получаваме, че $\exists c \in (a, b) : h'(c) = 0$

Пресмятаме, че $h'(x) = f'(x) - k$. Следователно $f'(c) - k = 0$, т.е. $f'(c) = k$. Имаме, че $f(a) - ka = f(b) - kb$, откъдето:

$$f(b) - f(a) = kb - ka = k(b - a) = f'(c)(b - a)$$

25.5 Теорема на Коши за средните стойности

Нека $f(x)$ и $g(x)$ са непрекъснати в $[a, b]$ и диференцируеми в (a, b) . Нека $g'(x) \neq 0, x \in (a, b)$. Тогава:

$$\exists c \in (a, b) : \frac{f(b) - f(a)}{g(b) - g(a)} = \frac{f'(c)}{g'(c)}$$

Доказателство: Забележка: Имаме, че $g(a) \neq g(b)$, защото в противен случай от теоремата на Рол, приложена към $g(x)$, следва, че $\exists c \in (a, b) : g'(c) = 0$, което е в противоречие с изискването $g'(x) \neq 0, x \in (a, b)$

Ще сведем към теоремата на Рол.

Въвеждаме функцията $h(x) := f(x) - kg(x), x \in [a, b]$, като k е такава, че $h(a) = h(b)$. Имаме:

$$\begin{aligned} h(a) &= h(b) \iff \\ f(a) - kg(a) &= f(b) - kg(b) \iff \\ \iff k[g(b) - g(a)] &= f(b) - f(a) \iff \\ \xLeftrightarrow{g(b)-g(a) \neq 0} k &= \frac{f(b) - f(a)}{g(b) - g(a)} \end{aligned}$$

Освен това, тъй като $f(x)$ и $g(x)$ са непрекъснати в $[a, b]$, то и $h(x)$ е такава.

Оттук $h(x)$ удовлетворява теоремата на Рол и имаме, че $\exists c \in (a, b) : h'(c) = 0$

Пресмятаме, че $h'(x) = f'(x) - kg'(x)$. Следователно: $f'(c) - kg'(c) = 0 \Rightarrow \frac{f'(c)}{g'(c)} = k$

Но ние знаем, че $k = \frac{f(b) - f(a)}{g(b) - g(a)}$, откъдето:

$$\frac{f'(c)}{g'(c)} = \frac{f(b) - f(a)}{g(b) - g(a)}$$

25.6 Формула на Тейлър

Нека $f(x)$ притежава производни до ред $n + 1$ включително в $(x_0 - \delta, x_0 + \delta), \delta > 0$. Тогава за всяко $x \in (x_0 - \delta, x_0 + \delta)$ съществува ξ между x_0 и x :

$$f(x) = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k + \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)^{n+1}$$

Доказателство: Ще докажем, използвайки следните лема:

Лема 1: Нека $f(x)$ притежава производна от ред n в т. x_0 . Тогава

$$T_n(x) := \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k$$

притежава свойството $(T_n)^{(i)}(x_0) = f^{(i)}(x_0), i = 0, \dots, n$

Доказателство: Имаме:

$$\begin{aligned}
 (T_n)^{(i)}(x) &= \left(\sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x-x_0)^k \right)^{(i)} \\
 &= \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} ((x-x_0)^k)^{(i)} \\
 &= \sum_{k=0}^{i-1} \frac{f^{(k)}(x_0)}{k!} ((x-x_0)^k)^{(i)} + \sum_{k=i}^n \frac{f^{(k)}(x_0)}{k!} ((x-x_0)^k)^{(i)} \quad (\text{В лявата сума } i\text{-тата производна е } 0) \\
 &= \sum_{k=i}^n \frac{f^{(k)}(x_0)}{k!} ((x-x_0)^k)^{(i)} \\
 &= \sum_{k=i}^n \frac{f^{(k)}(x_0)}{k!} k(k-1)\dots(k-(i-1))(x-x_0)^{k-i}
 \end{aligned}$$

За $k = i$ в сумата имаме $f^{(i)}(x_0)$. При $x = x_0$ за $k > i$ множителят $(x_0 - x_0)^{k-i}$ е със степен > 0 и следователно се нулира. Оттук при $x = x_0$ сумата е равна на $f^{(i)}(x_0)$. Следователно $(T_n)^{(i)}(x_0) = f^{(i)}(x_0)$ $\square$

Лема 2: Нека $\varphi(x)$ и $\psi(x)$ притежават производни от ред $n+1$ включително в $(x-\delta, x+\delta, \delta > 0)$. Нека още:

$$\varphi^{(i)}(x_0) = \psi^{(i)}(x_0) = 0, i = 0, \dots, n$$

и $\psi^{(i)}(x) \neq 0, x \in (x_0 - \delta, x_0 + \delta), x \neq x_0$ и $i = 0, \dots, n+1$.

Тогав за всяко $x \in (x_0 - \delta, x_0 + \delta), x \neq x_0$, съществува c между x_0 и x такава, че $\frac{\varphi(x)}{\psi(x)} = \frac{\varphi^{(n+1)}(c)}{\psi^{(n+1)}(c)}$

Доказателство: Нека $x \in (x_0 - \delta, x_0 + \delta), x \neq x_0$. Ще разгледаме интервала между x_0 и x . Б.О.О нека $x_0 < x$. Имаме, че φ, ψ са непрекъснати в $[x_0, x]$ и диференцируеми в (x_0, x) , както и че $\psi'(x_0) \neq 0, x \in (x_0 - \delta, x_0 + \delta), x \neq x_0$. Изпълнени са условията на теоремата на Коши, следователно $\exists c_1 \in (x_0, x)$:

$$\frac{\varphi(x) - \varphi(x_0)}{\psi(x) - \psi(x_0)} = \frac{\varphi'(c_1)}{\psi'(c_1)}$$

Но имаме, че $\varphi^{(i)}(x_0) = \psi^{(i)}(x_0) = 0, i = 0, \dots, n$, следователно:

$$\frac{\varphi(x)}{\psi(x)} = \frac{\varphi'(c_1)}{\psi'(c_1)}$$

Отново имаме, че φ', ψ' са непрекъснати в $[x_0, c_1]$ и диференцируеми в (x_0, c_1) , както и че $\psi''(x) \neq 0, x \in (x_0 - \delta, x_0 + \delta), x \neq x_0$. Изпълнени са условията на теоремата на Коши, следователно $\exists c_2 \in (x_0, c_1)$:

$$\frac{\varphi'(c_1) - \varphi'(x_0)}{\psi'(c_1) - \psi'(x_0)} = \frac{\varphi''(c_2)}{\psi''(c_2)}$$

Но имаме, че $\varphi^{(i)}(x_0) = \psi^{(i)}(x_0) = 0, i = 0, \dots, n$, следователно:

$$\frac{\varphi'(c_1)}{\psi'(c_1)} = \frac{\varphi''(c_2)}{\psi''(c_2)}$$

Комбинирайки с предното имаме:

$$\frac{\varphi(x)}{\psi(x)} = \frac{\varphi''(c_2)}{\psi''(c_2)}$$

Нека сме приложили теоремата $k < n$ пъти и да разгледаме за $k+1$. Имаме, че $\varphi^{(k)}, \psi^{(k)}$ са непрекъснати в (x_0, c_k) и диференцируеми в $[x_0, c_k]$, както и че $\psi^{(k)}(x) \neq 0, x \in (x_0 - \delta, x_0 + \delta), x \neq x_0$. Оттук имаме, че:

$$\frac{\varphi^{(k)}(c_1) - \varphi^{(k)}(x_0)}{\psi^{(k)}(c_1) - \psi^{(k)}(x_0)} = \frac{\varphi^{(k+1)}(c_{k+1})}{\psi^{(k+1)}(c_{k+1})}$$

Знаем, че

$$\frac{\varphi(x)}{\psi(x)} = \frac{\varphi^{(k)}(c_k)}{\psi^{(k)}(c_k)}$$

Също имаме, че $\varphi^{(i)}(x_0) = \psi^{(i)}(x_0) = 0, i = 0, \dots, n$, следователно:

$$\frac{\varphi^{(k)}(c_1)}{\psi^{(k)}(c_1)} = \frac{\varphi(x)}{\psi(x)} = \frac{\varphi^{(k+1)}(c_{k+1})}{\psi^{(k+1)}(c_{k+1})}$$

Оттук за $n+1$ и $c := c_{n+1} \in (x_0, x)$:

$$\frac{\varphi(x)}{\psi(x)} = \frac{\varphi^{(n+1)}(c)}{\psi^{(n+1)}(c)}$$

Доказахме Лема 2.

Формулата на Тейлър е тривиална за $x = x_0$. Нека $x \neq x_0$. Прилагаме Лема 2 с $\varphi(x) := f(x) - T_n(x)$, $\psi(x) = (x - x_0)^{n+1}$.

Благодарение на Лема 1, $\varphi(x)$ удовлетворява условията на Лема 2. Имам, че $\psi^{(i)}(x) = (n+1)n\dots(n-i+2)(x-x_0)^{n-i+1}, i = 0, \dots, n+1$. Лема 2 влече, че съществува c между x_0 и x , такова че:

$$\frac{\varphi(x)}{\psi(x)} = \frac{\varphi^{(n+1)}(x)}{\psi^{(n+1)}(x)}$$

Следователно:

$$\varphi(x) = \frac{\varphi^{(n+1)}(x)}{\psi^{(n+1)}(x)} \psi(x)$$

Откъдето:

$$f(x) - T_n(x) = \frac{f^{(n+1)}(c)}{(n+1)!} (x - x_0)^{n+1}$$

С което получаваме формулата на Тейлър. □

26 Определен интеграл. Дефиниция и свойства. Интегруемост на непрекъснатите функции. Теорема на Нютон-Лайбниц.

26.1 Разбиване на интервал, големи и малки суми на Дарбу

Разбиване на интервал: Разбиване на интервала $[a, b]$ наричаме всяко множество от точки $\tau = \{x_0, x_1, \dots, x_n\}$ такива, че:

$$a = x_0 < x_1 < \dots < x_n = b$$

Точките $x_0, \dots, x_n$ се наричат дялящи. Диаметър на разбиването τ наричаме числото $d(\tau) := \max_{i=1, \dots, n} (x_i - x_{i-1})$

Точките $c_1, \dots, c_n$ наричаме междинни за разбиването τ , ако $c_i \in [x_{i-1}, x_i]$, $i = 1, \dots, n$

Суми на Дарбу: Нека $f : [a, b] \rightarrow \mathbb{R}$ е ограничена. За разбиване $\tau : a = x_0 < \dots < x_n = b$ на $[a, b]$ дефинираме съответно малката и голяма сума на Дарбу на $f(x)$ чрез:

$$s_\tau := s_\tau(f) := \sum_{i=1}^n m_i(x_i - x_{i-1}), \quad m_i := \inf_{x \in [x_{i-1}, x_i]} f(x)$$

$$S_\tau := S_\tau(f) := \sum_{i=1}^n M_i(x_i - x_{i-1}), \quad M_i := \sup_{x \in [x_{i-1}, x_i]} f(x)$$

Твърдение: При добавяне на нови дялящи точки, малките суми на Дарбу не намаляват, а големите - не нарастват.

Доказателство: Ще покажем, че твърдението е вярно при добавянето на една дяляща точка. Ще разгледаме малките суми на Дарбу - доказателството е аналогично за големите.

Нека $\tau_1 : a = x_0 < \dots < x_n = b$ е разбиване на $[a, b]$ и τ_2 е разбиването на $[a, b]$, което се получава от τ_1 с добавяне на дялящата точка x' . Нека $x' \in (x_{j-1}, x_j)$. Тогава:

$$s_{\tau_2} - s_{\tau_1} = m'_j(x' - x_{j-1}) + m''_j(x_j - x') - m_j(x_j - x_{j-1})$$

Тук:

- $m_j := \inf_{x \in [x_{j-1}, x_j]} f(x)$
- $m'_j := \inf_{x \in [x_{j-1}, x']} f(x)$
- $m''_j := \inf_{x \in [x', x_j]} f(x)$

Имаме, че $m'_j, m''_j \geq m_j$

$$s_{\tau_2} - s_{\tau_1} = m'_j(x' - x_{j-1}) + m''_j(x_j - x') - m_j(x_j - x_{j-1}) \geq m_j(x' - x_{j-1} + x_j - x' - x_j + x_{j-1}) = 0$$

Определен интеграл - дефиниция на Дарбу: Нека $f : [a, b] \rightarrow \mathbb{R}$ е ограничена. Долен, съответно горен, определен интеграл на $f(x)$ върху $[a, b]$ (в смисъл на Дарбу) наричаме числата:

$$\underline{I} := \inf_{\tau} s_{\tau} \qquad \bar{I} := \sup_{\tau} S_{\tau}$$

Ако $\underline{I} = \bar{I}$ казваме, че $f(x)$ е интегруема върху $[a, b]$ и стойността им наричаме определен интеграл на $f(x)$ върху $[a, b]$.

26.2 Дефиниране на Риманов интеграл чрез подхода на Дарбу

Нека $f : [a, b] \rightarrow \mathbb{R}$ е интегруема по смисъла на дефиницията на Дарбу. Това означава, че $\underline{I} = \bar{I} =: I$. Ще докажем, че $\lim_{d(\tau) \rightarrow 0} R_{\tau} = I$. Това означава, че $f(x)$ е интегруема в смисъла на дефиницията на Риман и определеният и интеграл по смисъла на Риман съвпада точно с този по дефиниция на Дарбу.

$$\begin{array}{ccc} s_{\tau} \leq R_{\tau} \leq S_{\tau} & \forall \tau & \\ \downarrow & \downarrow & \text{при } d(\tau) \rightarrow 0 \\ \underline{I} = I & \bar{I} = I & (\text{Лема на Дарбу}). \end{array}$$

26.3 Критерий за интегрируемост, Дарбу

Ограничената функция $f : [a, b] \rightarrow \mathbb{R}$ е интегрируема върху $[a, b]$, тогава и само тогава, когато:

$$\forall \varepsilon > 0 \exists \text{ разбиване на } \tau \text{ на } [a, b] : S_\tau - s_\tau < \varepsilon$$

Доказателство: Нека $f(x)$ е интегрируема. Тогава $\underline{I} = \bar{I} =: I$, т.е.

$$I := \sup_\tau s_\tau = \inf_\tau S_\tau$$

Нека $\varepsilon > 0$ е произволно. Числото $I - \frac{\varepsilon}{2}$ не е горна граница на множеството от малките суми на Дарбу (I е горната граница). Следователно съществува s_{τ_1} :

$$s_{\tau_1} > I - \frac{\varepsilon}{2}$$

Аналогично, $I + \frac{\varepsilon}{2}$ не е долна граница на множеството от големите суми на Дарбу. Следователно съществува S_{τ_2} такава, че

$$S_{\tau_2} < I + \frac{\varepsilon}{2}$$

Двете неравенства ни дават, че:

$$S_{\tau_2} - s_{\tau_1} < I + \frac{\varepsilon}{2} - \left(I - \frac{\varepsilon}{2}\right) = \varepsilon$$

Образува разбиването $\tau = \tau_1 \cup \tau_2$. Знаем, че при добавяне на нови точки малките суми на Дарбу не намаляват и големите не нарастват. Оттук:

$$S_\tau - s_\tau \leq S_{\tau_2} - s_{\tau_1} < \varepsilon$$

Обратно, нека $\varepsilon > 0$ е произволно. Тогава съществува разбиване τ такова, че $S_\tau - s_\tau < \varepsilon$. Следователно:

$$\bar{I} - \underline{I} \leq S_\tau - s_\tau < \varepsilon$$

Така установихме, че:

$$0 \leq \bar{I} - \underline{I} < \varepsilon \quad \forall \varepsilon > 0$$

Следователно $\underline{I} = \bar{I}$ и $f(x)$ е интегрируема върху $[a, b]$ в смисъл на Дарбу, следователно е интегрируема по Риман. $\square$

26.4 Всяка непрекъснатата функция в краен и затворен интервал е интегрируема по Риман

Доказателство: Нека $f : [a, b] \rightarrow \mathbb{R}$ е непрекъснатата функция.

От теоремата на Кантор имаме, че f е равномерно непрекъснатата. Следователно:

$$\forall \varepsilon > 0 \exists \delta > 0 : |x - y| < \delta \implies |f(x) - f(y)| < \frac{\varepsilon}{b - a}$$

Нека τ е такова разбиване на $[a, b]$, че дължината на всеки подинтервал удовлетворява $x_i - x_{i-1} < \delta$

За всеки подинтервал $[x_{i-1}, x_i]$ означаваме:

$$M_i = \sup_{[x_{i-1}, x_i]} f, \quad m_i = \inf_{[x_{i-1}, x_i]} f$$

Тъй като дължината на интервала е по-малка от δ , то от равномерната непрекъснатост следва, че:

$$M_i - m_i < \frac{\varepsilon}{b - a}$$

Оттук:

$$S_\tau - s_\tau = \sum_{i=1}^n (M_i - m_i)(x_i - x_{i-1})$$

$$S_\tau - s_\tau = \sum_{i=1}^n \frac{\varepsilon}{b-a} (x_i - x_{i-1})$$

$$S_\tau - s_\tau = \frac{\varepsilon}{b-a} \sum_{i=1}^n (x_i - x_{i-1})$$

$$\text{Но } \sum_{i=1}^n (x_i - x_{i-1}) = b - a$$

$$\text{Откъдето } S_\tau - s_\tau = \frac{\varepsilon}{b-a} (b-a) = \varepsilon$$

Получихме, че

$$\forall \varepsilon \exists \tau : S_\tau - s_\tau < \varepsilon$$

Откъдето по критерия на Дарбу f е интегрируема по Риман върху $[a, b]$

□

26.5 Основни свойства на Римановия интеграл

1. Линеиност: Нека $f, g : [a, b] \rightarrow \mathbb{R}$ са интегрируеми и $k \in \mathbb{R}$. Тогава:

- Функцията $f(x) + g(x)$ е интегрируема върху $[a, b]$ и:

$$\int_a^b [f(x) + g(x)] dx = \int_a^b f(x) dx + \int_a^b g(x) dx$$

- Функцията $kf(x)$ е интегрируема върху $[a, b]$ и:

$$\int_a^b kf(x) dx = k \int_a^b f(x) dx$$

2. Монотонност: Нека $f, g : [a, b] \rightarrow \mathbb{R}$ са интегрируеми.

- Ако $f(x) \geq 0$ в $[a, b]$, то:

$$\int_a^b f(x) dx \geq 0$$

- Ако $f(x) \leq g(x)$ в $[a, b]$, то:

$$\int_a^b f(x) dx \leq \int_a^b g(x) dx$$

- Функцията $|f(x)|$ също е интегрируема, при това:

$$\left| \int_a^b f(x) dx \right| \leq \int_a^b |f(x)| dx$$

3. Адитивност: Нека $f : [a, b] \rightarrow \mathbb{R}$ и $c \in (a, b)$.

- Ако $f(x)$ е интегрируема върху $[a, b]$, то:

$$\int_a^b f(x) dx = \int_a^c f(x) dx + \int_c^b f(x) dx$$

26.6 Теорема за средните стойности

Нека $f : [a, b] \rightarrow \mathbb{R}$ е непрекъсната. Тогава съществува $c \in [a, b]$:

$$\int_a^b f(x)dx = f(c)(b-a)$$

Доказателство: Тъй като $f : [a, b] \rightarrow \mathbb{R}$ е непрекъсната, то тя има НМС и НГС. Да ги означим с m и M , за които е изпълнено:

$$m \leq f(x) \leq M, \quad x \in [a, b]$$

Интегрирайки тези неравенства в $[a, b]$ (M, m са константни функции), получаваме:

$$m(b-a) \leq \int_a^b f(x)dx \leq M(b-a) \implies m \leq \frac{1}{b-a} \int_a^b f(x)dx \leq M$$

Знаем, че всяка непрекъсната функция в $[a, b]$ приема всички стойности между минимума и максимума си, т.е. между m и M . Оттук следва, че $\exists c \in [a, b]$:

$$m \leq c \leq M \implies f(c) = \frac{1}{b-a} \int_a^b f(x)dx$$

С което доказахме теоремата. □

26.7 Теорема на Нютон-Лайбниц

Ако f е непрекъсната в $[a, b]$, то:

$$\frac{d}{dx} \int_a^x f(t)dt = f(x)$$

Доказателство: Нека $F(x) = \int_a^x f(t)dt$. Също нека $x_0 \in [a, b]$ е произволно фиксирано и $h \neq 0 : x_0 + h \in [a, b]$. За диференчното частно на $F(x)$ в точката x_0 имаме:

$$\begin{aligned} \frac{F(x_0 + h) - F(x_0)}{h} &= \frac{1}{h} \left(\int_a^{x_0+h} f(t)dt - \int_a^{x_0} f(t)dt \right) = \\ &= \frac{1}{h} \left(\int_{x_0}^{x_0+h} f(t)dt \right) = \\ &= \frac{1}{h} f(c_h) [(x_0 + h) - x_0] = f(c_h) \end{aligned}$$

Където $c_h \in (x_0, x_0 + h)$. Оттук при $c_h \rightarrow x_0$ при $h \rightarrow 0$ функцията $f(x)$ е непрекъсната в т. x_0 . Следователно $f(c_h) = f(x_0)$. Получаваме:

$$\lim_{h \rightarrow 0} \frac{F(x_0 + h) - F(x_0)}{h} = f(x_0)$$

Следователно $F(x)$ е диференцируема в т. x_0 и $F'(x_0) = f(x_0)$. x_0 бе произволно фиксирана в интервала, откъдето теоремата е доказана. □

Формула на Нютон-Лайбниц: Нека $f : [a, b] \rightarrow \mathbb{R}$ е непрекъсната. Ако $G(x)$ е коя да е примитивна на $f(x)$ върху $[a, b]$, то:

$$\int_a^b f(x)dx = G(b) - G(a)$$

Доказателство: От горната теорема имаме, че $F(x)$ е примитивна на $f(x)$ в $[a, b]$. Знаем, че съществува $C \in \mathbb{R}$ такова, че $G(x) = F(x) + C, x \in [a, b]$. В частност да разгледаме $x = a$. Имаме, че $F(a) = 0$. Следователно $C = G(a)$, откъдето получаваме:

$$G(x) = F(x) + G(a), \quad x \in [a, b]$$

Следователно

$$F(b) = G(b) - G(a)$$

Имаме, че $F(b) = \int_a^b f(x)dx$, откъдето:

$$\int_a^b f(x)dx = G(b) - G(a)$$

□

27 Уравнения на права в равнината

Нека $K = O_{XY}$ е АКС в равнината.

27.1 Векторно-параметрично уравнение на права. Координатни (скалярни) параметрични уравнения на права в равнината.

Параметрични уравнения: Нека $\overrightarrow{OM_0} = \vec{r_0}, \overrightarrow{OM} = \vec{r}$ са радиус вектори. Нека $\vec{p} \neq \vec{0}$ е даден вектор.

$$\exists! \text{ права } g \begin{cases} M_0 \in g \\ g \parallel \vec{p} \end{cases}$$

За произволна точкака M от g е в сила:

$$\overrightarrow{M_0M} \parallel \vec{p} \iff \exists! s \in \mathbb{R} : \overrightarrow{M_0M} = s \cdot \vec{p}$$

Оттук имаме взаимно еднозначно съответствие между $s \in \mathbb{R}$ и точка $M \in g$.

$$1). \text{ Векторно параметрично уравнение: } \overrightarrow{M_0M} = \overrightarrow{OM} - \overrightarrow{OM_0} = \vec{r} - \vec{r_0} \iff \vec{r} - \vec{r_0} = s \cdot \vec{p}$$

$$g : \vec{r} = \vec{r_0} + s \cdot \vec{p}, s \in \mathbb{R}$$

2: **Координатни параметрични уравнения:** Нека $M_0(x_0, y_0), M(x, y), \vec{p}(p_1, p_2)$:

$$g : \begin{cases} x = x_0 + s \cdot p_1 \\ y = y_0 + s \cdot p_2, s \in \mathbb{R} \end{cases}$$

27.2 Теорема за общо уравнение на права в равнината. Взаимни положения между две прави в равнината.

Теорема: $\Rightarrow$ Всяка права в равнината има спрямо K уравнение от вида $Ax + By + C = 0, (A, B) \neq (0, 0)$.

Обратно, всяко уравнение от вида $Ax + By + C = 0, (A, B) \neq (0, 0)$ определя права в равнината.

$$\text{Доказателство: } \Rightarrow: \text{ Нека } g \begin{cases} M_0 \in g \\ g \parallel \vec{p} \end{cases}$$

Тогава:

$$\begin{aligned} \overrightarrow{M_0M}(x - x_0, y - y_0) \parallel \vec{p}(p_1, p_2) &\iff \\ &\iff \begin{vmatrix} x - x_0 & p_1 \\ y - y_0 & p_2 \end{vmatrix} = 0 \iff \\ p_2(x - x_0) - p_1(y - y_0) &= 0 \end{aligned}$$

$$g : p_2 \cdot x - p_1 \cdot y - p_2 \cdot x_0 + p_1 \cdot y_0 = 0$$

Полагаме: $A = p_2, B = -p_1, C = -p_2 \cdot x_0 + p_1 \cdot y_0$. Получаваме, че: $g : Ax + By + C = 0$, от $\vec{p}(p_1, p_2) \neq \vec{0} \Rightarrow (A, B) \neq (0, 0)$

$\Leftarrow$: Разглеждаме уравнението: $Ax + By + C = 0, (A, B) \neq (0, 0)$

Нека x_0, y_0 е едно решение. Определяме вектор $\vec{p}(-B, A) \Rightarrow \vec{p} \neq \vec{0}$

$$\text{Тогава } \exists! \text{ права } g \begin{cases} M_0 \in g \\ g \parallel \vec{p}(-B, A) \end{cases}$$

Координатните параметрични уравнения на тази права g са:

$$\begin{aligned} g &= \begin{cases} x = x_0 - B\lambda \\ y = y_0 - A\lambda, \lambda \in \mathbb{R} \end{cases} \\ &\iff \frac{x - x_0}{-B} = \frac{y - y_0}{-A} \iff \\ &\iff A(x - x_0) + B(y - y_0) = 0 \\ &\iff Ax + By - Ax_0 - By_0 = 0 \end{aligned}$$

Но $Ax_0 + By_0 + C = 0$, откъдето $C = -Ax_0 - By_0$ следователно уравнението $Ax + By + C = 0$ описва точно правата g . $\square$

Условие за колинеарност на права и вектор: Нека $g : Ax + By + C, (A, B) \neq (0, 0), g \parallel \vec{p}(-B, A)$ е права, $\vec{q}(q_1, q_2)$ е вектор.

$$\vec{q}(q_1, q_2) \parallel g \parallel \vec{p}(-B, A) \iff \begin{vmatrix} q_1 & -B \\ q_2 & A \end{vmatrix} = 0 \iff Aq_1 + Bq_2 = 0$$

Взаимни положения на две прави в равнината:

Нека:

$$\begin{aligned} g_1 : A_1x + B_1y + C_1 &= 0 \\ g_2 : A_2x + B_2y + C_2 &= 0 \end{aligned}$$

Имаме 3 възможности за взаимните положения на g_1 и g_2 .

1. сл.

$$r \begin{pmatrix} A_1 & B_1 \\ A_2 & B_2 \end{pmatrix} = 2 \Rightarrow g_1 \cap g_2 = \text{т. } P - \text{единствена обща точка}$$

2. сл.

$$r \begin{pmatrix} A_1 & B_1 \\ A_2 & B_2 \end{pmatrix} = 1 \text{ и } r \begin{pmatrix} A_1 & B_1 & C_1 \\ A_2 & B_2 & C_2 \end{pmatrix} = 2 \Rightarrow g_1 \parallel g_2 \text{ и нямат обща точка}$$

3. сл.

$$r \begin{pmatrix} A_1 & B_1 & C_1 \\ A_2 & B_2 & C_2 \end{pmatrix} = 1 \Rightarrow g_1 \equiv g_2$$

27.3 Декартово уравнение на права в равнината. Нормално уравнение на права в равнината.

Работим с ОКС: O_{XY}

Декартово уравнение на права: Разглеждаме: $g : Ax + By + C, B \neq 0$, т.е. $g \nparallel O_Y$, откъдето

$$g : Y = -\frac{A}{B}x - \frac{C}{B}$$

Полагаме: $k := -\frac{A}{B}, n := -\frac{C}{B}$

Получаваме, че $y = kx + n$. $k = \tan \alpha, \alpha = \angle(O_X^+, g)$, където (O, n) е пресечната точка на g и O_Y

Нормално уравнение на права: Нека $g : Ax + By + C = 0$

$$g \parallel \vec{p}(-B, A)$$

$g \perp \vec{n}_g(A, B)$ - нормален вектор.

$$|\vec{n}_g| = \sqrt{A^2 + B^2} \Rightarrow \vec{n}_1 \left(\frac{A}{\sqrt{A^2 + B^2}}, \frac{B}{\sqrt{A^2 + B^2}} \right)$$

$\vec{n}_1$ е единичен нормален вектор на g

Всички общи уравнения на g имат вида:

$$(\lambda A)X + (\lambda B)y + \lambda C = 0$$

Търсим $\lambda = ?$, така че $\vec{n}_1(\lambda A, \lambda B)$ да е единичен. Искаме:

$$\vec{n}_1^2 = (\lambda A)^2 + (\lambda B)^2 = 1 \Rightarrow \lambda^2 = \frac{1}{A^2 + B^2} \Rightarrow \lambda = \frac{\pm 1}{\sqrt{A^2 + B^2}}$$

Оттук имаме, че всяка права има точно две нормални уравнения:

$$g : \pm \frac{Ax + By + C}{\sqrt{A^2 + B^2}} = 0$$

Ако означим:

$$A_1 = \frac{A}{\sqrt{A^2 + B^2}}, \quad B_1 = \frac{B}{\sqrt{A^2 + B^2}}, \quad C_1 = \frac{C}{\sqrt{A^2 + B^2}}$$

То получаваме:

$$\begin{aligned} A_1 &= \cos \angle(\vec{e}_1, \vec{n}_1) \\ B_1 &= \cos \angle(\vec{e}_2, \vec{n}_1) \\ C_1 &= \delta(\text{т.} O, g) \end{aligned}$$

27.4 Разстояние от точка до права. Полуравнини.

Нека $g : A_1x + B_1y + C_1 = 0$ е нормално уравнение, $A_1^2 + B_1^2 = 1$

Нека т. $M_0(x_0, y_0)$ е точка от равнината.

Нека H е ортогоналната проекция на правата g до точката M_0 ($H \overset{\sigma g}{\mapsto} M_0$)

Имаме, че $\overrightarrow{HM_0} \parallel \vec{n}_1 \Rightarrow \exists! \delta : \overrightarrow{HM_0} = \delta \vec{n}_1$

$$H(x_H, y_H) \in g \Rightarrow \begin{cases} x_0 - x_H = \delta A_1 \\ y_0 - y_H = \delta B_1 \end{cases} \Rightarrow \begin{cases} x_H = x_0 - \delta A_1 \\ y_H = y_0 - \delta B_1 \end{cases}$$

Заместваме точките x_H, y_H в уравнението на g :

$$\begin{aligned} A_1(x_0 - \delta A_1) + B_1(y_0 - \delta B_1) + C_1 &= 0 \\ A_1x_0 + B_1y_0 + C_1 - \delta(A_1^2 + B_1^2) &= 0 \\ \delta = A_1x_0 + B_1y_0 + C_1 &= \frac{Ax_0 + By_0 + C}{\sqrt{A^2 + B^2}} \end{aligned}$$

δ ни дава ориентираното разстояние от точка M_0 до права g .

28 Уравнения на права и равнина в пространството.

Работим с ОКС $K = O_{XYZ}$

28.1 Векторно-параметрично уравнение на равнина. Координатни (скалярни) параметрични уравнения на равнина в пространството.

Нека $M_0(x_0, y_0, z_0)$ е фиксирана точка.

Нека $\vec{a}(a_1, a_2, a_3), \vec{b}(b_1, b_2, b_3)$ са ЛНЗ.

Имаме, че $\exists!$ равнина α
$$\begin{cases} M_0 \in \alpha \\ \alpha \parallel \vec{a} \\ \alpha \parallel \vec{b} \end{cases}$$

Нека т. $M(x, y, z)$ е произволна точка от α . Тогава $\overrightarrow{M_0M}$ е компланарен с $\vec{a}, \vec{b}$, т.е. $\exists! \lambda, \mu$:

$$\alpha : \begin{cases} x = x_0 + \lambda a_1 + \mu b_1 \\ y = y_0 + \lambda a_2 + \mu b_2 \\ z = z_0 + \lambda a_3 + \mu b_3 \end{cases}$$

Тези три равенства се наричат **скалярни параметрични уравнения** на равнината α , определена точката $M_0(x_0, y_0, z_0)$ и векторите $\vec{a}, \vec{b}$

Компланарността на $\overrightarrow{M_0M}$ с $\vec{a}, \vec{b}$ ни дава, че $\overrightarrow{M_0M} = \lambda \vec{a} + \mu \vec{b}$

Това равенство можем да запишем като $\overrightarrow{OM} = \overrightarrow{OM_0} + \lambda \vec{a} + \mu \vec{b}$, където O е началото на координатната система. Нека с r, r_0 означим радиус-векторите $\overrightarrow{OM}, \overrightarrow{OM_0}$ на точките M, M_0 .

Равенството: $r = r_0 + \lambda \vec{a} + \mu \vec{b}$ наричаме векторно параметрично уравнение на равнината α

28.2 Теорема за общо уравнение на равнина.

Теорема: Точка $M(x, y, z)$ от пространството лежи на равнината α точно тогава, когато координатите ѝ удовлетворяват:

$$Ax + By + Cz + D = 0, (A, B, C) \neq (0, 0, 0)$$

И обратно, всяко уравнение от вида $Ax + By + Cz + D = 0, (A, B, C) \neq (0, 0, 0)$ е уравнение на точно една равнина.

Доказателство: $\Rightarrow$ Нека в равнината α вземем точка $M_0(x_0, y_0, z_0)$ и два неколинеарни вектора $\vec{p}_1(a_1, b_1, c_1), \vec{p}_2(a_2, b_2, c_2)$ компланарни с M_0 . Точката $M(x, y, z)$ от пространството ще лежи в α точно тогава, когато $\overrightarrow{M_0M}, \vec{p}_1, \vec{p}_2$ са компланарни. Тези вектори са компланарни, когато смесеното произведение $\overrightarrow{M_0M} \vec{p}_1 \vec{p}_2 = 0$, т.е. когато детерминантата от координатите им е 0:

$$\begin{vmatrix} x - x_0 & y - y_0 & z - z_0 \\ a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \end{vmatrix} = 0$$

Което е изпълнено, когато:

$$(x - x_0) \begin{vmatrix} b_1 & c_1 \\ b_2 & c_2 \end{vmatrix} - (y - y_0) \begin{vmatrix} a_1 & c_1 \\ a_2 & c_2 \end{vmatrix} + (z - z_0) \begin{vmatrix} a_1 & b_1 \\ a_2 & b_2 \end{vmatrix} = 0$$

Полагаме: $A = b_1c_2 - b_2c_1, B = a_2c_1 - a_1c_2, C = a_1b_2 - a_2b_1$.

Дали $(A, B, C) \neq (0, 0, 0)$. Ако допуснем, че $A = B = C = 0 \Rightarrow \frac{a_1}{a_2} = \frac{b_1}{b_2} = \frac{c_1}{c_2} \Rightarrow \vec{p}_1 \parallel \vec{p}_2$ - противоречие с допускането, че не са колинеарни.

Заместваме и получаваме:

$$Ax + By + Cz - Ax_0 - By_0 - Cy_0 = 0$$

Нека $D = -(Ax_0 + By_0 + Cy_0)$ и получаваме за α :

$$\alpha : Ax + By + Cz + D = 0$$

$\Leftarrow$: Нека за x_0, y_0, z_0 е изпълнено: $Ax_0 + By_0 + Cz_0 + D = 0$.

Уравнението има решение, защото поне едно от A, B, C е различно от нула. Нека M_0 е точката с координати x_0, y_0, z_0 . Б.О.О можем да считаме, че $B \neq 0$ и да вземем векторите: $\vec{p}_1(-B, A, 0), \vec{p}_2(0, \frac{-C}{B}, 1)$ - ненулеви и

неколинеарни. Единствената равнина α : $\begin{cases} M_0(x_0, y_0, z_0) \in \alpha \\ \alpha \parallel \vec{p}_1 \\ \alpha \parallel \vec{p}_2 \end{cases}$ има уравнението:

$$\begin{vmatrix} x - x_0 & y - y_0 & z - z_0 \\ -B & A & 0 \\ 0 & \frac{-C}{B} & 1 \end{vmatrix} = 0$$

Получаваме: $(x - x_0)A + (y - y_0)(-B) + (z - z_0)C = 0 \iff Ax + By + Cz - (Ax_0 + By_0 + Cz_0) = 0$

Имаме, че $Ax_0 + By_0 + Cz_0 + D = 0$ и да положим: $D = -(Ax_0 + By_0 + Cz_0)$, откъдето получаваме уравнение за равнина β

$$\beta : Ax + By + Cz + D = 0$$

Но M_0 бе произволна точка α следователно $\beta \equiv \alpha$

□

28.3 Взаимни положения между две равнини.

Нека равнините α_1, α_2 са зададени с общите си уравнения:

$$\alpha_1 : A_1x + B_1y + C_1z + D_1 = 0$$

$$\alpha_2 : A_2x + B_2y + C_2z + D_2 = 0$$

Имаме 3 възможности за взаимните положения на α_1 и α_2 .

1. сл.

$$r \begin{pmatrix} A_1 & B_1 & C_1 \\ A_2 & B_2 & C_2 \end{pmatrix} = 2 \Rightarrow \alpha_1 \text{ пресича } \alpha_2$$

2. сл.

$$r \begin{pmatrix} A_1 & B_1 & C_1 \\ A_2 & B_2 & C_2 \end{pmatrix} = 1 \text{ и } r \begin{pmatrix} A_1 & B_1 & C_1 & D_1 \\ A_2 & B_2 & C_2 & D_2 \end{pmatrix} = 2 \Rightarrow \alpha_1 \parallel \alpha_2 \text{ и нямат обща точка}$$

3. сл.

$$r \begin{pmatrix} A_1 & B_1 & C_1 & D_1 \\ A_2 & B_2 & C_2 & D_2 \end{pmatrix} = 1 \Rightarrow \alpha_1 \equiv \alpha_2$$

28.4 Нормално уравнение на равнина.

Общо уравнение на равнината α : $Ax + By + Cz + D = 0$, в което коефициентите на пред текущите координати са координати на единичен нормален вектор на α се нарича нормално уравнение на α .

Уравнението $Ax + By + Cz + D = 0$ ще бъде нормално точно тогава, когато:

$$A^2 + B^2 + C^2 = 1$$

За да намерим всички нормални уравнения на α , от всички общи уравнения на α трябва да проверим за кои λ е изпълнено условието за нормалност

$$(\lambda A)^2 + (\lambda B)^2 + (\lambda C)^2 = 1$$

То е изпълнено за:

$$\lambda = \frac{\pm 1}{\sqrt{A^2 + B^2 + C^2}}$$

И така равнината α има точно две нормални уравнения:

$$\frac{Ax + By + Cz + D}{\pm \sqrt{A^2 + B^2 + C^2}}$$

28.5 Разстояние от точка до равнина.

Нека $\alpha : \frac{Ax+By+Cz+D}{\sqrt{A^2+B^2+C^2}}$

Нека

$$A_1 = \frac{A}{\sqrt{A^2+B^2+C^2}}, \quad B_1 = \frac{B}{\sqrt{A^2+B^2+C^2}}, \quad C_1 = \frac{C}{\sqrt{A^2+B^2+C^2}}, \quad D_1 = \frac{D}{\sqrt{A^2+B^2+C^2}}$$

Разглеждаме точка $M_0(x_0, y_0, z_0)$

Нека с H означим ортогоналната проекция на M_0 върху α .

Имаме, че: $\overrightarrow{HM_0} \parallel \vec{n}_1 \Rightarrow \exists! \delta :$

$$\overrightarrow{HM_0} = \delta \vec{n}_1 \iff \begin{cases} x_0 - x_h = \delta A_1 \\ y_0 - y_h = \delta B_1 \\ z_0 - z_h = \delta C_1 \end{cases} \iff \begin{cases} x_h = x_0 - \delta A_1 \\ y_h = y_0 - \delta B_1 \\ z_h = z_0 - \delta C_1 \end{cases}$$

Заместваме в уравнението на α и намираме δ :

$$A_1(x_0 - \delta A_1) + B_1(y_0 - \delta B_1) + C_1(z_0 - \delta C_1) + D_1 = 0 \iff A_1 x_0 + B_1 y_0 + C_1 z_0 + D_1 - \underbrace{\delta(A_1^2 + B_1^2 + C_1^2)}_{\delta \cdot 1} = 0$$

Получаваме: $\delta(M_0, \alpha) = \frac{Ax_0+By_0+Cz_0+D}{\sqrt{A^2+B^2+C^2}}$ е ориентираното разстояние от точката M_0 до правата α .

28.6 Полупространства.

28.7 Уравнение на права в пространството.

Уравнение на права като пресечница на две равнини: Нека равнините α_1, α_2 са зададени с общите си уравнения:

$$\begin{aligned} \alpha_1 : A_1x + B_1y + C_1z + D_1 &= 0 \\ \alpha_2 : A_2x + B_2y + C_2z + D_2 &= 0 \end{aligned}$$

се пресичат в права g . Тогава координатите (x, y, z) на всяка точка от g удовлетворяват системата:

$$\begin{cases} \alpha_1 : A_1x + B_1y + C_1z + D_1 = 0 \\ \alpha_2 : A_2x + B_2y + C_2z + D_2 = 0 \end{cases}$$

И обратно, ако координатите на една точка удовлетворяват системата, то тази точка лежи на g .

29 Итерационни методи за решаване на нелинейни уравнения.

29.1 Метод на свиващите изображения

Разглеждаме $f(x)$, $x \in [a, b]$ и искаме да решим $f(x) = 0$. Записваме в еквивалентен вид: $x = \varphi(x)$

Неподвижна точка: x е неподвижна точка за изображението $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ ако $\varphi(x) = x$

Следователно, за да намерим корен на уравнението $f(x) = 0$ трябва да намерим неподвижна точка $\xi = \varphi(\xi)$

Избираме начално приближение $x_0 \in [a, b]$. И строим редицата $\{x_n\}$ по правилото $x_{n+1} = \varphi(x_n)$, $n = 1, 2, \dots$
При определени предположения за φ има редица, за която $\lim_{n \rightarrow \infty} x_n \rightarrow \xi$

Лема: Ако $\varphi : [a, b] \rightarrow [a, b]$ е непрекъсната в $[a, b]$, то φ има неподвижна точка в $[a, b]$

Доказателство: Ако $a = \varphi(a)$, то a е неподвижна точка. Аналогично за $b = \varphi(b)$

Да допуснем, че $\varphi(a) \neq a$, $\varphi(b) \neq b$. Тъй като $\varphi : [a, b] \rightarrow [a, b]$, то $\varphi(a) \in [a, b]$, $\varphi(b) \in [a, b]$. Оттук $a < \varphi(a)$, $b > \varphi(b)$. Функцията $r(x) := x - \varphi(x)$ е непрекъсната в $[a, b]$ и

$$r(a) = a - \varphi(a) < 0, \quad r(b) = b - \varphi(b) > 0$$

Затова съществува точка ξ от $[a, b] : r(\xi) = 0$, т.е. $\xi = \varphi(\xi)$ □

Липшицова функция: функцията φ е Липшицова в $[a, b]$, ако съществува константа $q \in [0, 1]$:

$$|\varphi(x) - \varphi(y)| \leq q|x - y|, \quad \forall x, y \in [a, b]$$

Свиващо изображение: Изображение φ , което изпълнява условието на Липшиц с константа $q < 1$ се нарича свиващо. При него разстоянието между образите $\varphi(x)$, $\varphi(y)$ е строго по-малко от разстоянието между образите x, y , т.е. изображението φ "свива" разстоянията.

Теорема: Ако $\varphi : [a, b] \rightarrow [a, b]$ е непрекъсната и Липшицова с константа $q < 1$ (φ е свиващо), тогава:

- $x = \varphi(x)$ има единствен корен ξ в $[a, b]$
- Редицата $\{x_n\}$ клони към ξ при $n \rightarrow \infty$ и е изпълнено:

$$|x_n - \xi| \leq (b - a)q^n, \quad \forall n \in \mathbb{N}$$

Доказателство: Знаем, че φ има поне една неподвижна точка. Да допуснем, че те са повече. Нека $\xi_1 = \varphi(\xi_1)$, $\xi_2 = \varphi(\xi_2)$ за $\xi_1, \xi_2 \in [a, b]$. Тогава при $\xi_1 \neq \xi_2$:

$$\begin{aligned} |\xi_1 - \xi_2| &= |\varphi(\xi_1) - \varphi(\xi_2)| \\ &\leq q|\xi_1 - \xi_2| \text{ (Липшицова)} \\ &< |\xi_1 - \xi_2| \text{ (защото } q < 1) \end{aligned}$$

Получихме, че $|\xi_1 - \xi_2| < |\xi_1 - \xi_2|$, с което имаме, че неподвижната точка е единствена.

Имаме:

$$\begin{aligned} |x_n - \xi| &= |\varphi(x_{n-1}) - \varphi(\xi)| \leq q|x_{n-1} - \xi| \\ &= q|\varphi(x_{n-2}) - \varphi(\xi)| \leq q^2|x_{n-2} - \xi| \\ &= \vdots \\ &\leq q^n|x_0 - \xi| \end{aligned}$$

Тъй като $\xi_0 \in [a, b]$ и $\xi \in [a, b]$, то $|x_0 - \xi| < b - a$.

Оттук $|x_n - \xi| < q^n(b - a)$, но $q < 1$, откъдето $|x_n - \xi| \xrightarrow{n \rightarrow \infty} 0$ и $x_n \xrightarrow{n \rightarrow \infty} \xi$

Следствие: Ако ξ е корен на $x = \varphi(x)$ и φ има непрекъсната производна в околност $\mathcal{U}$ на ξ , таква че $|\varphi'(\xi)| < 1$. Тогава при достатъчно добро начално приближение ξ_0 итерационният процес, породен от φ , е сходящ със скоростта на геометрична прогресия, т.е. съществуват константи $C > 0$ и $0 < q < 1$, такива че:

$$|x_n - \xi| \leq Cq^n, \quad \forall n$$

Доказателство: Тъй като $\varphi'(t)$ е непрекъснатата функция в $\mathcal{U}$ и $|\varphi'(\xi)| < 1$, то съществуват $q < 1$ и $\varepsilon > 0$:

$$|\varphi'(t)| \leq q, \text{ за всяко } t \in [\xi - \varepsilon, \xi + \varepsilon]$$

Освен това, при $t \in [\xi - \varepsilon, \xi + \varepsilon]$ имаме:

$$|\varphi(t) - \xi| \leq q|t - \xi| \leq q\varepsilon < \varepsilon,$$

т.е. $\varphi(t) \in [\xi - \varepsilon, \xi + \varepsilon]$. Следователно φ е свиващо изображение на интервала $[\xi - \varepsilon, \xi + \varepsilon]$ в себе си. Тогава са изпълнени условията от предишната теорема, откъдето:

$$|x_n - \xi| \leq ((\xi + \varepsilon) - (\xi - \varepsilon))q^n = 2\varepsilon q^n \stackrel{C:=2\varepsilon>0}{=} Cq^n \text{ за всяко } n$$

Ред на сходимост: Казваме, че итерационният процес $x_0, x_1, \dots$ има ред на сходимост p , ($p > 1$) ако съществуват положителни константи C и $q < 1$:

$$|x_n - \xi| \leq Cq^{p^n} \text{ за всяко } n$$

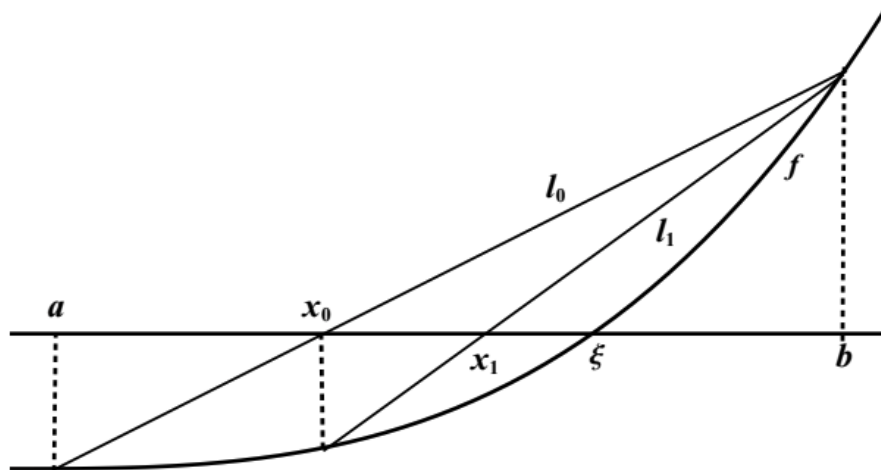
29.2 Метод на хордите

Метод на Хордите: Нека $[a, b]$ е даден интервал и $f(x)$ е два пъти диференцируема в него функция, която удовлетворява условията:

- $f(a)f(b) < 0$
- $f'(x)f''(x) \neq 0, \forall x \in [a, b]$

Методът на хордите е итерационен процес, в който се построява редица от последователни приближения $x_0, x_1, \dots$ на корен ξ на уравнението $f(x) = 0$. Прекарва се права линия l_0 през точките $(a, f(a)), (b, f(b))$ (хордата към дъгата от графиката). Тя пресича оста x в точката x_0 - началното ни приближение.

Точката x_{n+1} се получава като пресечна точка на оста x с хордата l_{n+1} , свързваща точките $x_n, f(x_n)$ и $b, f(b)$ - ако е изпъкнала (иначе $(a, f(a))$). Методът е илюстриран:



Фигура 1: Геометрична илюстрация на метода на хордите.

Формула за последователните приближения на метода на хордите: Нека за определеност $f''(x) > 0$, като на чертежа. Правата $y = l_{n+1}(x)$ има уравнение:

$$l_{n+1}(x) = f(x_n) + \frac{f(x_n) - f(b)}{x_n - b}(x - x_n)$$

Приближението x_{n+1} е корен на уравнението $l_{n+1} = 0$ и следователно:

$$x_{n+1} = x_n - \frac{f(x_n)}{f(x_n) - f(b)}(b - x_n)$$

Сходимость на метода на хордите: При направените предположения за f имаме, че f е изпъкнала, откъдето $\{x_n\}$ е монотонна и ограничена редица, следователно е сходяща. Нека $\lim_{n \rightarrow \infty} x_n = \alpha$. Тогава:

$$\alpha = \alpha - \frac{f(\alpha)}{f(b) - f(\alpha)}(b - \alpha)$$

Получаваме, че $f(\alpha) = 0$, следователно $\alpha = \xi$, откъдето е доказана сходимостта на x_n към ξ .

Сега ще разгледаме скоростта на сходимост. Ще докажем, че сходимостта на метода на хордите е със скоростта на геометричната прогресия. (Ще приложим следствието).

Имаме:

$$\varphi(x) = x - \frac{f(x)}{f(b) - f(x)}(b - x)$$

Разглеждаме $\varphi'(\xi)$:

$$\varphi'(\xi) = 1 - \frac{f'(\xi)}{f(b) - f(\xi)}(\xi - b)$$

Но имаме, че $f(\xi) = 0$, значи

$$\varphi'(\xi) = 1 - \frac{f'(\xi)}{f(b)}(\xi - b) = \frac{f(b) - f'(\xi)(b - \xi)}{f(b)}$$

Като заместим $f(b)$ по формулата на Тейлър имаме:

$$\begin{aligned} f(b) &= f(\xi) + f'(\xi)(b - \xi) + \frac{f''(\eta_1)}{2}(b - \xi)^2 \text{ (в числителя)} \\ f(b) &= f(\xi) + f''(\eta_2)(b - \xi) \text{ (в знаменателя)} \end{aligned}$$

$$\varphi'(\xi) = \frac{f''(\eta_1)(b - \xi)}{2f'(\eta_2)}$$

Нека $M := \max_{t \in [a, b]} |f''(t)|$ и $m := \min_{t \in [a, b]} |f'(t)|$. По условие имаме, че $f'(t) > 0$ в $[a, b]$, следователно $m > 0$. Оттук:

$$|\varphi'(\xi)| \leq \frac{M}{2m}|b - \xi|$$

Ако $b - \xi$ е достатъчно малко, то $|\varphi'(\xi)|$ може да стане по-малко от произволно, предварително избрано $q < 1$. Но $b - \xi$ е достатъчно малко, стига интервалът $[a, b]$ да е достатъчно малък. И така ако ξ е в достатъчно малък интервал $[a, b]$, то $|\varphi'(\xi)| < q < 1$ и от следствието имаме, че итерационният процес породен от φ е сходящ със скорост на геометричната прогресия.

29.3 Метод на секущите

Метод на секущите: Нека $[a, b]$ е даден интервал и $f(x)$ е два пъти диференцируема в него функция, която удовлетворява условията:

- $f(a)f(b) < 0$
- $f'(x)f''(x) \neq 0, \forall x \in [a, b]$

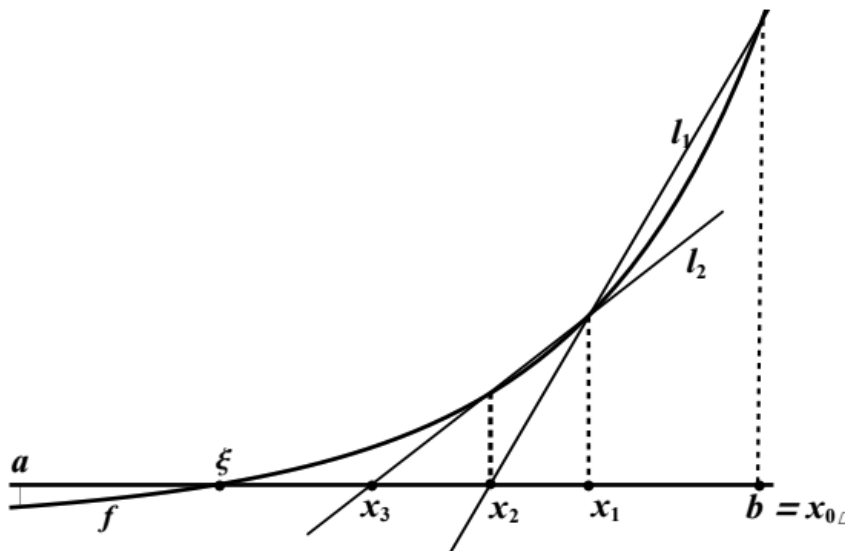
Означаваме $M := \max_{t \in [a, b]} |f''(t)|$ и $n := \min_{t \in [a, b]} |f'(t)|$

Избираме $x_0 = a$ или $x_0 = b$, така че $f(x_0)f''(x_0) > 0$ След това избираме точка $x_1 : \xi < x_1 < x_0$. Това става, когато x_0 и x_1 са от една и съща страна на ξ , т.с.т.к. $f(x_0)f(x_1) > 0$.

Построяваме x_2 като пресечната точка на секущата l_1 минаваща през точките $(x_0, f(x_0)), (x_1, f(x_1))$ и оста x .

Формула за последователните приближения: По формулата на Нютон имаме:

$$l_n(x) = f(x_n) + \frac{f'(x_n)}{f(x_{n-1}) - f(x_n)}(x - x_n)$$



Фигура 2: Геометрична илюстрация на метода на секущите.

x_{n+1} се определя от уравнението $l_n(x) = 0$. Оттук формулата а пресмятане на x_{n+1} чрез x_n е:

$$x_{n+1} = x_n - \frac{f(x_n)}{f(x_{n-1}) - f(x_n)}(x_{n-1} - x_n)$$

Ред на сходимост:

$$|x_n - \xi| \leq Cq^{r^n}, \forall n$$

Тук $r = \frac{1+\sqrt{5}}{2}$

29.4 Метод на Нютон

Метод на Нютон (на допирателните): Нека $[a, b]$ е даден интервал и $f(x)$ е два пъти диференцируема в него функция, която удовлетворява условията:

- $f(a)f(b) < 0$
- $f'(x)f''(x) \neq 0, \forall x \in [a, b]$

Означаваме $M := \max_{t \in [a, b]} |f''(t)|$ и $n := \min_{t \in [a, b]} |f'(t)|$

Както при метода на секущите избираме приближение $x_0 = a$ или $x_0 = b$, така че $f(x_0)f''(x_0) > 0$.

Следващото приближение x_1 се намира като пресечната точка на оста x с допирателната t_0 към правата $y = f(x)$ в точката x_0 .

Формула за последователните приближения: Имаме, че x_{n+1} е нулата на допирателната t_n към f в точката x_n . От условието $l_n(x_{n+1}) = 0$, където:

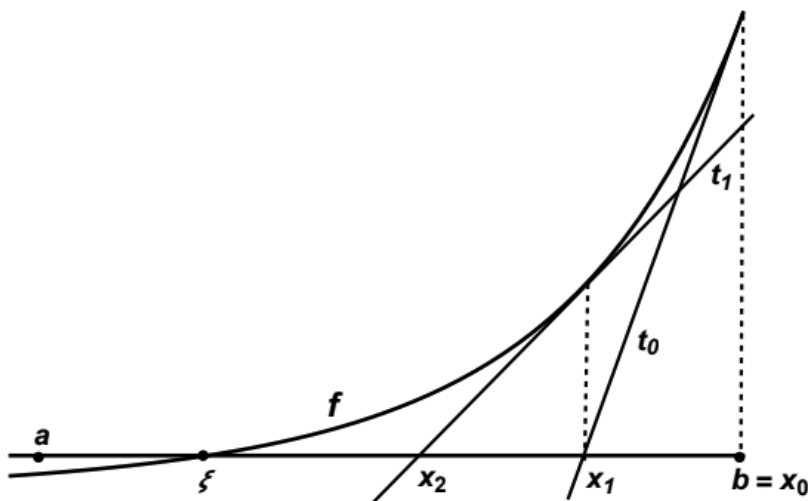
$$l_n(x) = f(x_n) - f'(x_n)(x - x_n)$$

намираме формулата за получаване на x_{n+1} от x_n :

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Ред на сходимост:

$$|x_n - \xi| \leq Cq^{2^n}, \quad \forall n, q \in (0, 1)$$



Фигура 3: Геометрична илюстрация на метода на Нютон.

30 Дискретни разпределения. Равномерно, биномно, геометрично и Пуасоново разпределение. Задачи, в които възникват. Моменти - математическо очакване и дисперсия.

Ще се дадат две разпределения, които трябва да се опишат.

30.1 Дискретно вероятностно разпределение на случайна величина

Сигма алгебра: Нека Ω е произволно множество и нека $\mathcal{A} \in \mathcal{P}(\otimes)$. Казваме, че $\mathcal{A}$ е сигма алгебра, ако са изпълнени условията:

- $\emptyset \in \mathcal{A}$
- $\Omega \in \mathcal{A}$
- Ако $A \in \mathcal{A}$, то $\bar{A} \in \mathcal{A}$
- Ако $A_1, A_2, \dots \in \mathcal{A}$, то $\bigcup_{i=1}^{\infty} A_i \in \mathcal{A}$ и $\bigcap_{i=1}^{\infty} A_i \in \mathcal{A}$

Вероятност: Вероятността P е функция, дефинирана върху сигма алгебрата $\mathcal{A}$ от подмножества на Ω , която удовлетворява аксиомите:

- Неотрицателност: $P(A) \geq 0$, за всяко събитие $A \in \mathcal{A}$
- Нормираност: $P(\Omega) = 1$
- Адитивност: Ако $AB = \emptyset$, то $P(A \cup B) = P(A) + P(B)$
- Непрекъснатост (монотонност): За всяка монотонно намаляваща редица $A_1 \supset A_2 \supset \dots \supset A_n \supset \dots$, клоняща към празното множество е изпълнено $\lim_{n \rightarrow \infty} P(A_n) = 0$

Случайна величина: Случайна величина наричаме функция $X : \Omega \rightarrow \mathbb{R}$

Дискретна случайна величина: Нека $\Omega = \bigcup_i H_i : H_i \cap H_j = \emptyset, i \neq j$, а x_i са произволни реални числа.

Дискретна случайна величина наричаме:

$$X(\omega) = \sum_i x_i I_{H_i}(\omega)$$

Където $I_{H_i}(\omega) = \begin{cases} 1, \omega \in H_i \\ 0, \omega \notin H_i \end{cases}$ е индикатор на множеството H_i

X	0	1
P	1-p	p

Математическо очакване: Нека X е дискретна случайна величина. Математическо очакване наричаме числото:

$$\mathbb{E}X = \sum_j x_j p_j, \text{ където } p_j = P(X = x_j)$$

Дисперсия: Нека X е произволна случайна величина. Дисперсия DX се дефинира с равенството:

$$DX = \mathbb{E}(X - \mathbb{E}X)^2$$

Корен от дисперсията наричаме стандартно отклонение.

Пораждаща функция: Нека X е случайна величина, чийто стойности са цели положителни числа. Пораждаща функция на X наричаме полином, в който пред k -тата степен на коефициента s стои вероятността $P(X = k)$:

$$g_X(s) = \sum_{k=0}^{\infty} P(X = k) s^k$$

Свойства на пораждащата функция:

- $\mathbb{E}X = g'_X(1)$
- $DX = g''_X(1) + g'_X(1) - (g'_X(1))^2$

30.2 Разпределение на Бернули

Разпределение на Бернули: Нека X е случайна величина. X има разпределение на Бернули, ако прием две стойности: "1" при успех и "0" при неуспех. Може да се използва за хвърляне на монета - задаваме стойността "1" на ези, а "0" на тура.

Разпределението има вида:

Математическо очакване:

$$\mathbb{E}X = 0(1 - p) + 1.p = p$$

$$\mathbb{E}X^2 = 0.(1 - p)^2 + 1.p^2 = p^2$$

Дисперсия:

$$DX = \mathbb{E}X^2 - (\mathbb{E}X)^2 = p - p^2 = p(1 - p) = pq$$

30.3 Биномно разпределение

Биномно разпределение: Извършват се n последователни, независими Бернулиеви опити, като вероятността за успех p на всеки опит е една и съща.

Случайната величина X равна на броят на успехите, наричаме биномно разпределена и записваме: $X \in Bi(n, p)$. Вероятността за k успеха при n опита получаваме чрез:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Имаме:

- p^k : Вероятността да имаме k успеха при n опита.
- $(1 - p)^{n-k}$: Вероятността да имаме $n - k$ неуспеха при n опита.
- $\binom{n}{k}$: Броят различни подредби на успешните и неуспешните опити.

Коректност: Формулата за бинома на Нютон ни дава:

$$\sum_{k=1}^n P(X = k) = \sum_{k=1}^n \binom{n}{k} p^k (1-p)^{n-k} = (p + (1-p))^n = 1$$

Пораждаща функция:

$$g_X(s) = \sum_{k=1}^n P(X = k) s^k = \sum_{k=1}^n \binom{n}{k} p^k (1-p)^{n-k} s^k = \sum_{k=1}^n \binom{n}{k} (ps)^k (1-p)^{n-k} = (ps + (1-p))^n$$

Математическо очакване:

$$\mathbb{E}X = (n(ps + (1-p))^{n-1} p) \stackrel{s=1}{=} n(p + 1-p)^{n-1} p = np$$

Дисперсия:

$$\begin{aligned} DX &= g_X''(1) + g_X'(1) - [g_X'(1)]^2 = [n(n-1)(ps + (1-p))^{n-2} p^2 + np - n^2 p^2] \\ &\stackrel{s=1}{=} n(n-1)(p + 1-p)^{n-2} p^2 + np - n^2 p^2 = \\ &= n(n-1)p^2 + np - (np)^2 = np^2 - np^2 + np - (np)^2 = np(1-p) = npq \end{aligned}$$

Чрез биномното разпределение можем да пресметнем броя на шестите при хвърляне на 4 зара.

30.4 Геометрично разпределение

Нека p е вероятността за успех, $q = 1 - p$ е вероятността за неуспех.

Геометрично разпределение: Разглеждаме схема на Бернули с неограничен брой опити. Случайната величина X равна на "броя на неуспехите до достигане на първи успех" наричаме геометрично разпределена случайна величина и записваме $X \in Ge(p)$. Вероятността да получим успех при k -тия опит получаваме пресмятайки:

$$P(X = k) = (1-p)^{k-1} p$$

Коректност: Прилагаме формулата за безкрайна геометрична прогресия:

$$\sum_{k=0}^{\infty} P(X = k) = \sum_{k=0}^{\infty} (1-p)^k p = \frac{p}{1 - (1-p)} = 1$$

Пораждаща функция:

$$g_X(s) = \sum_{k=0}^{\infty} P(X = k) s^k = \sum_{k=0}^{\infty} q^k p s^k = p \sum_{k=0}^{\infty} (qs)^k = \frac{p}{1 - qs}$$

Математическо очакване:

$$\mathbb{E}X = g_X'(1) = p \left(\frac{q}{(1-qs)^2} \right) \stackrel{s=1}{=} \frac{pq}{(1-q)^2} = \frac{pq}{p^2} = \frac{p}{q}$$

Дисперсия:

$$\begin{aligned} g_X''(1) &= \left(\frac{pq}{(1-qs)^2} \right)' \stackrel{s=1}{=} \frac{2pq^2}{(1-q)^3} = \frac{2pq^2}{p^3} = \frac{2q^2}{p^2} \\ DX &= g_X''(1) + g_X'(1) - (g_X'(1))^2 = \frac{2q^2}{p^2} + \frac{p}{q} - \frac{q^2}{p^2} = \frac{q(q+p)}{p^2} = \frac{q}{p^2} \end{aligned}$$

Чрез отрицателното разпределение можем да пресметнем броя хвърляния на зар, докато не се падне шестца.

30.5 Отрицателно биномно разпределение

Отрицателно биномно разпределение: Разглеждаме схема на Бернули с неограничен брой опити. Случайната величина X равна на "броя на неуспехите до достигане на r -тия успех" наричаме отрицателно биномно разпределена и записваме $X \in NB(r, p)$. За да пресметнем вероятността да сме получили r успеха след k опита пресмятаме:

$$P(X = k) = \binom{r+k-1}{k} p^r q^k = \binom{r+k-1}{r-1} p^r q^k$$

Имаме:

- $\binom{r+k-1}{r-1}$: Броят различни начини, по който може да сме получили k неуспеха и $r-1$ успеха.
- $p^{r-1} q^k$: Вероятността да сме получили k неуспеха и $r-1$ успеха.
- Умножаваме всичко по p - вероятността r -тия (последния) опит да е успех.

Коректност:

$$\sum_{k=0}^{\infty} P(X = k) = p^r \sum_{k=0}^{\infty} \binom{r+k-1}{r-1} q^k = \frac{p^r}{(1-q)^r} = 1$$

За да пресметнем очакването и дисперсията ще въведем случайните величини:

- X_1 броят на неуспехите до първия успех.
- X_2 броят на неуспехите между първия и втория успех.
- ...
- X_r броят на неуспехите между $r-1$ -вия и r -тия успех.

Тъй като опитите са независими, то и случайните величини са независими. Имаме, че $X_i \in Ge(p)$.

Имаме, че $X = X_1 + \dots + X_r$, откъдето можем да разгледаме отрицателното биномно разпределение като сума от геометрични разпределения.

Знаем, че пораждащата функция на X_i е:

$$g_{X_i}(s) = \frac{p}{1-qs}$$

За пораждащата функция на X получаваме:

$$g_X(s) = g_{X_1+\dots+X_r}(s) = \prod_{i=1}^r g_{X_i}(s) = \frac{p^r}{(1-qs)^r}$$

Прилагаме формулата за отрицателен бином и имаме:

$$\frac{p^r}{(1-qs)^r} = p^r \sum_{k=0}^{\infty} \binom{r+k-1}{k} q^k s^k = \sum_{k=0}^{\infty} P(X = k) s^k$$

Виждаме, че $X \in NB(r, p)$. От свойствата на очакването и дисперсията имаме:

Математическо очакване:

$$\mathbb{E}X = \mathbb{E}(X_1 + \dots + X_r) = r\mathbb{E}X_1 = r\frac{p}{q}$$

Дисперсия:

$$DX = r\frac{q}{p^2}$$

С отрицателното биномно разпределение можем да пресметнем броя хвърляния на монета, докато не сме хвърлили 10 езита.

30.6 Поасоново разпределение

Поасоново разпределение: Поасоновото разпределение се използва, когато имаме голям брой събития с малка вероятност за успех, частен случай на биномното при $n \rightarrow \infty, p \rightarrow 0$. Казваме, че случайната величина X равна на броя на успехите е поасоново разпределена и пишем $X \in Po(\lambda)$. Пресмятаме вероятността за k успеха чрез:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

Тук $\lambda > 0$ е константа.

Коректност:

$$\sum_{k=0}^{\infty} \frac{\lambda^k e^{-\lambda}}{k!} = e^{-\lambda} \sum_{k=0}^{\infty} \frac{\lambda^k}{k!} = e^{-\lambda} e^{\lambda} = 1$$

Пораждаща функция:

$$g_X(s) = \sum_{k=0}^{\infty} P(X = k) s^k = \sum_{k=0}^{\infty} \frac{\lambda^k e^{-\lambda}}{k!} s^k = e^{-\lambda} \sum_{k=0}^{\infty} \frac{(\lambda s)^k}{k!} = e^{-\lambda} e^{\lambda s} = e^{\lambda(s-1)}$$

Математическо очакване:

$$\mathbb{E}X = g'_X(1) = \lambda e^{\lambda(s-1)} \Big|_{s=1} \stackrel{s=1}{=} \lambda$$

Дисперсия:

$$DX = g''_X(1) + g'_X(1) - [g'_X(1)]^2 = \lambda^2 e^{\lambda(s-1)} + \lambda - \lambda^2 \stackrel{s=1}{=} \lambda^2 + \lambda - \lambda^2 = \lambda$$

Пример: Произвеждаме 1000 детайла на ден. Вероятността един детайл да е дефектен е $1/1000$. Получаваме $\lambda = np = 1$. Вероятността два детайла да се дефектни е:

$$P(X = 2) = \frac{\lambda^2 e^{-\lambda}}{2!} = \frac{1e^{-1}}{2} = \frac{1}{2e}$$